

UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Tecnologías SDN e Inteligencia Artificial como Mecanismo de Mitigación de Incidentes de Ciberseguridad

Autora
Clara García Fuentes

Director
Dr. José Antonio Gómez Hernández



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Granada, noviembre de 2026

Declaración de autoría del presente trabajo

Clara García Fuentes

Yo, Clara García Fuentes, declaro que el presente Trabajo de Fin de Grado, titulado *Tecnologías SDN e Inteligencia Artificial como Mecanismo de Mitigación de Incidentes de Ciberseguridad*, ha sido realizado por mí bajo la supervisión del director indicado, y que su contenido es original salvo las referencias explícitamente citadas a lo largo del documento.

Durante su desarrollo he utilizado herramientas de Inteligencia Artificial generativa como apoyo en determinadas tareas. Concretamente, se han utilizado en los siguientes contextos:

- **Revisión y redacción de partes del documento:** algunas secciones del texto han sido reescritas con asistencia de modelos de lenguaje para adecuar el lenguaje a una base técnica apropiada para el trabajo. El contenido, los datos y los resultados son míos.
- **Escritura de fragmentos de código:** en algunos módulos del sistema se ha recurrido a herramientas de generación de código como apoyo en la implementación, siempre bajo mi criterio en cuanto al diseño, la integración y la verificación del funcionamiento.
- **Resolución de errores:** durante el desarrollo y las pruebas se ha utilizado para diagnosticar y corregir errores, como recurso de depuración complementario a mi propio análisis.

En todos los casos, el uso de estas herramientas ha sido instrumental y supervisado por mí, y subordinado a mis propias decisiones técnicas y metodológicas.

La responsabilidad sobre el diseño del sistema, la interpretación de los resultados, las conclusiones alcanzadas y la integridad global del trabajo es exclusivamente mía.

Granada, noviembre de 2026.

Tecnologías SDN e Inteligencia Artificial como Mecanismo de Mitigación de Incidentes de Ciberseguridad

Clara García Fuentes

Palabras clave: redes definidas por software, SDN, aprendizaje automático, detección de intrusiones, mitigación de ataques, DDoS, *spoofing*, escaneo de puertos, Random Forest, OpenFlow.

Resumen

Ataques informáticos como el escaneo de puertos, la suplantación de identidad (*spoofing*) o la denegación de servicio distribuida (DDoS) son cada vez más frecuentes y exigen respuestas más rápidas de lo que permite la intervención manual. Este trabajo explora cómo combinar las Redes Definidas por Software (SDN), que ofrecen una visión global de la red y permiten actuar sobre su tráfico, con el aprendizaje automático, que permite reconocer qué tráfico es un ataque, para detectar y bloquear estas amenazas en tiempo de ejecución, mientras el tráfico se genera.

Para ello se construye una red virtual emulada, controlada por un único controlador SDN, en la que se genera un conjunto de datos propio con tráfico normal y de ataque. Con él se comparan tres modelos de aprendizaje automático y se despliega el mejor, Random Forest, en el controlador, que analiza cada flujo de tráfico y bloquea de forma temporal las comunicaciones que identifica como ataque.

En las pruebas, el sistema bloquea 390 de las 395 conversaciones de ataque (98,7 %) unos 2 segundos después de que empiece el ataque y con un consumo de CPU del controlador reducido (entre el 3,5 % y el 6,7 %). En cuanto a los falsos positivos, alrededor de una de cada cinco conversaciones legítimas de las pruebas (28 de 123) se corta temporalmente por error. Los resultados se han obtenido en un entorno emulado y sin evaluar ataques de baja tasa; si bien no son directamente extrapolables a redes reales, ofrecen una aproximación a un entorno real.

SDN Technologies and Artificial Intelligence as a Mechanism for Mitigating Cybersecurity Incidents

Clara García Fuentes

Keywords: software-defined networking, SDN, machine learning, intrusion detection, attack mitigation, DDoS, spoofing, port scanning, Random Forest, OpenFlow.

Abstract

Cyberattacks such as port scanning, spoofing and distributed denial of service (DDoS) are increasingly frequent and call for faster responses than manual intervention can provide. This work explores how to combine Software-Defined Networking (SDN), which gives a global view of the network and the ability to act on its traffic, with machine learning, which can recognise which traffic is an attack, in order to detect and block these threats during execution, as the traffic is generated.

To do so, an emulated virtual network with a single SDN controller is built, and a custom dataset of normal and attack traffic is generated on it. Three machine learning models are compared and the best one, Random Forest, is deployed in the controller, which analyses every traffic flow and temporarily blocks the communications it identifies as attacks.

In the tests, the system blocks 390 out of 395 attack conversations (98.7 %) about 2 seconds after the attack starts and with low controller CPU usage (between 3.5 % and 6.7 %). The cost is that about one in five legitimate conversations in the tests (28 out of 123) is temporarily cut by mistake. The results were obtained in an emulated environment and without evaluating low-rate attacks; although they are not directly extrapolable to real networks, they offer an approximation of a real environment.

Yo, **Clara García Fuentes**, estudiante de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75934790Z, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Clara García Fuentes

GARCIA FUENTES CLARA - 75934790Z

Firmado digitalmente por GARCIA FUENTES CLARA - 75934790Z
Fecha: 2026.11.10 09:42:58 +01'00'

Granada a 10 de noviembre de 2026.

D. **José Antonio Gómez Hernández**, Profesor del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Tecnologías SDN e Inteligencia Artificial como Mecanismo de Mitigación de Incidentes de Ciberseguridad***, ha sido realizado bajo su supervisión por **Clara García Fuentes**, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 10 de noviembre de 2026.

El director:

Firmado por GOMEZ HERNANDEZ JOSE ANTONIO - ***5964** el día 10/11/2026 con un certificado emitido por AC FNMT Usuarios

José Antonio Gómez Hernández

Agradecimientos

A mi hermano, por haber recorrido conmigo estos años de carrera y por su disposición a echarme una mano. Gracias por tu apoyo, por comprender las dificultades de este proceso y por estar siempre a mi lado.

A mi mejor amigo, por tantos años de amistad y por seguir siendo esa persona en la que puedo confiar y a la que contar todo, sin importar la distancia. Gracias por estar siempre ahí.

A mi pareja, por acompañarme durante todo este camino, por su paciencia y por su apoyo constante. Gracias por ayudarme siempre que lo he necesitado, por entender los momentos de estrés y por hacer que esta última etapa haya sido mucho más llevadera.

Y, por último, a mi familia y a mis compañeros de carrera y amigos, por haber formado parte de estos años y por haber contribuido, cada uno a su manera, a hacer posible llegar hasta aquí.

Índice general

1 Motivación **1**
- 1.1 Contexto . . . 1
  - 1.1.1 Un panorama de amenazas en crecimiento . . . 1
  - 1.1.2 Redes definidas por software y seguridad . . . 2
  - 1.1.3 Aprendizaje automático para detectar y mitigar ataques en SDN . . . 2
  - 1.1.4 Planteamiento del trabajo . . . 3
- 1.2 Objetivos . . . 4
  - 1.2.1 Objetivo general . . . 4
  - 1.2.2 Objetivos específicos . . . 4
- 1.3 Organización de la memoria . . . 5

2 Planificación **7**
- 2.1 Metodología de planificación . . . 7
- 2.2 Etapas del proyecto . . . 7
  - 2.2.1 Análisis preliminar y revisión bibliográfica . . . 7
  - 2.2.2 Diseño de la arquitectura y estrategia de defensa . . . 8
  - 2.2.3 Implementación y desarrollo de pruebas . . . 8
  - 2.2.4 Experimentación y análisis de resultados . . . 8
  - 2.2.5 Redacción, revisión y cierre del proyecto . . . 9
- 2.3 Recursos utilizados . . . 9
  - 2.3.1 Recursos humanos . . . 9
  - 2.3.2 Recursos hardware . . . 9
  - 2.3.3 Recursos software . . . 10
- 2.4 Planificación temporal . . . 11
- 2.5 Presupuesto . . . 11
  - 2.5.1 Costes de personal . . . 11
  - 2.5.2 Costes de hardware . . . 12
  - 2.5.3 Costes de software . . . 12
  - 2.5.4 Resumen del presupuesto . . . 12

3 Estado del arte **15**
- 3.1 Redes definidas por software . . . 15
  - 3.1.1 Del modelo tradicional a SDN . . . 15

3.1.2 Arquitectura 16
3.1.3 OpenFlow 17
3.1.4 Controladores y entornos de emulación 19
3.2 Seguridad en redes definidas por software 19
3.2.1 Una doble cara 19
3.2.2 Amenazas por plano e interfaz 20
3.2.3 Los ataques considerados 21
3.2.4 Seguridad automatizada 21
3.3 Aprendizaje automático para detectar y mitigar ataques en SDN 22
3.3.1 Panorama 22
3.3.2 Conjuntos de datos 23
3.3.3 Características y obtención de estadísticas 23
3.3.4 Evaluación y métricas 24
3.3.5 Detección en tiempo de ejecución y mitigación 24
3.4 Trabajos relacionados 25
3.5 Carencias en el estado del arte 27
3.6 Posicionamiento del trabajo 28

4 Diseño **31**
4.1 Arquitectura general 31
4.2 Entorno experimental 32
4.2.1 Topología 32
4.2.2 Controlador y observación de los flujos 33
4.2.3 Identidad de los *hosts* 33
4.3 Generación del conjunto de datos 33
4.3.1 Variantes de tráfico 34
4.3.2 Etiquetado 34
4.4 Características y preprocesado 35
4.4.1 Características de cada flujo 35
4.4.2 Preprocesado 36
4.4.3 Selección de características 36
4.5 Modelos y evaluación sin conexión 37
4.5.1 Modelos 37
4.5.2 Protocolo de validación 37
4.5.3 Métricas y selección del mejor modelo 38
4.6 Detección y mitigación en tiempo de ejecución 38
4.6.1 Clasificación en tiempo de ejecución 38
4.6.2 Mitigación 38
4.7 Diseño de la evaluación en tiempo de ejecución 39
4.8 Resumen de las decisiones de diseño 40

5 Implementación **41**
5.1 Organización del código, configuración y ejecución 41

5.2 Entorno de ejecución 43
5.2.1 Plataforma 43
5.2.2 Instalación automatizada 44
5.3 Fase 1: generación del conjunto de datos 45
5.3.1 Red emulada 45
5.3.2 Controlador de monitorización 45
5.3.3 Coordinación entre procesos 46
5.3.4 Generador de tráfico 46
5.3.5 Orquestación 48
5.4 Fase 2: aprendizaje automático 48
5.5 Fase 3: detección y mitigación en tiempo de ejecución 49
5.5.1 Controlador de defensa 49
5.5.2 Clasificador en tiempo de ejecución 50
5.5.3 Mitigación 50
5.5.4 Registro de eventos y medidas 51
5.5.5 Orquestación de las pruebas 52

6 Experimentación 53
6.1 Condiciones experimentales 53
6.2 Fase 1: conjunto de datos 54
6.2.1 Composición 54
6.2.2 Saturación del entorno de emulación 55
6.3 Fase 2: evaluación de los modelos sin conexión 55
6.3.1 Comparación de los modelos 55
6.3.2 Resultados por clase y errores 56
6.3.3 Importancia y selección de características 57
6.3.4 Variación entre generaciones del conjunto de datos . . 58
6.4 Fase 3: detección y mitigación en tiempo de ejecución 59
6.4.1 Detección en tiempo de ejecución 59
6.4.2 Mitigación 60
6.4.3 Efecto del bloqueo sobre la detección por flujo 61
6.4.4 Impacto en la infraestructura 62
6.5 Discusión 63
6.5.1 El modelo y sus errores 63
6.5.2 Resultados sin conexión y en tiempo de ejecución . . . 63
6.5.3 Detección por flujo y mitigación por conversación . . . 64
6.5.4 El coste de los falsos bloqueos 65
6.5.5 Coste para la infraestructura 65
6.5.6 Comparativa con trabajos previos 66
6.5.7 Alcance de los resultados 66

7 Conclusiones y trabajo futuro 67
7.1 Conclusiones 67
7.2 Limitaciones 69

7.3 Trabajo futuro 72

Anexos **73**

A Código de las fases 1 y 2 **73**
A.1 Etiquetado por flujo 73
A.2 Ventana temporal 74
A.3 Validación cruzada agrupada por fase 75

B Código de la fase 3 **77**
B.1 Clasificación en lote 77
B.2 Mitigación 78

C Resultados complementarios **81**
C.1 Matrices de confusión de los otros modelos 81
C.2 Rendimiento sin conexión y en tiempo de ejecución por clase . 82
C.3 Resultados por ejecución de la batería 82
C.4 Ejecución representativa de cada tipo de tráfico 83
C.5 CPU y latencia del plano de control 84

Glosario **87**

Bibliografía **93**

Índice de figuras

1.1 Visión general de las etapas propuestas para el TFG. 4

2.1 Diagrama de Gantt del proyecto, con las horas estimadas de cada etapa. 11

3.1 Arquitectura SDN: planos e interfaces. 16

4.1 Arquitectura del sistema. El controlador comparte entre las fases 1 y 3 la conmutación y el sondeo; la fase 2 se ejecuta sin conexión. 32
4.2 Topología en árbol de la red emulada: un *switch* raíz, cuatro *switches* de acceso y 16 *hosts*. Todos los *switches* se conectan al controlador Ryu, que no se dibuja. 32

5.1 Ejecución del sistema. 42
5.2 Procesamiento de cada respuesta de estadísticas en el controlador de defensa y puntos de medida. 51

6.1 Matriz de confusión de Random Forest con validación cruzada agrupada por fase. 56
6.2 Importancia de las 15 características seleccionadas, según un Random Forest entrenado con todo el conjunto de datos. . . . 57
6.3 Matriz de confusión de la detección en tiempo de ejecución, con las diez ejecuciones de la batería. 60
6.4 Tasa de paquetes de una prueba de DDoS y reglas de descarte instaladas (líneas discontinuas), en la ejecución representativa, la que tiene el F1 macro más cercano a la media. La escala vertical es logarítmica. 61

C.1 Matrices de confusión de la regresión logística y del árbol de decisión. 81
C.2 Exhaustividad (*recall*) y F1 por clase de Random Forest sin conexión (fase 2) y en tiempo de ejecución (fase 3). 82
C.3 Prueba de *scanning*: flujos de ataque acumulados y flujos detectados por el modelo, con las reglas de descarte instaladas. . 83

C.4 Prueba de *spoofing*: flujos de ataque acumulados y flujos detectados por el modelo, con las reglas de descarte instaladas. . 84
C.5 Prueba de tráfico normal: tasa de paquetes y reglas de descarte instaladas. Al no haber ataque, toda regla de descarte sería un falso bloqueo; en esta ejecución no se produjo ninguno, aunque en cada una de las otras nueve hubo entre 1 y 7. Entre los 10s y los 24 s no se registran flujos, un hueco que no aparece en las demás ejecuciones (en ellas, el mayor es de 5s) y que es compatible con la saturación puntual del entorno (Sección 6.2.2). 84
C.6 CPU de Ryu y latencia del plano de control por tipo de tráfico en la ejecución representativa. En las pruebas de ataque, el tramo central sin datos corresponde a la vigencia de las primeras reglas de descarte, durante la que no aparecen flujos nuevos; en la de tráfico normal, al hueco descrito en la Figura C.5. 85

Índice de tablas

2.1 Equipo empleado en el desarrollo y la experimentación. 9

2.2 Herramientas software empleadas y versiones del entorno con el que se obtuvieron los resultados. 10

2.3 Coste de la estudiante por etapas. 12

2.4 Resumen del presupuesto. 13

3.1 Campos de una entrada de flujo en OpenFlow 1.3, según [9]. . 17

3.2 Amenazas por plano o interfaz en SDN. 20

3.3 Trabajos relacionados con el presentado en este TFG. LR: regresión logística; DT: árbol de decisión; RF: Random Forest; NB: *naive Bayes*; KNN: k vecinos. 26

4.1 Variantes de tráfico que genera cada clase. 34

4.2 Características de cada flujo, agrupadas por la información que aportan. 35

4.3 Decisiones principales del diseño y carencia de los trabajos previos citados que atienden. 40

5.1 Organización del código fuente. 42

5.2 Parámetros principales de `config.py`. 43

5.3 Ficheros mediante los que se coordinan los procesos, todos en el directorio `runtime/`. 47

6.1 Composición del conjunto de datos: flujos por clase, fases de cada tipo que se lanzaron y fases en las que se registró algún flujo de su clase, y reparto de los flujos por protocolo. Las fases lanzadas de cada tipo se obtienen del registro de la generación de tráfico. 54

6.2 Resultados de los tres modelos con validación cruzada agrupada por fase (cinco particiones). Precisión (Prec.), exhaustividad (Exh.) y F1 son macro. La desviación (Desv.) es la desviación típica de los F1 macro de las cinco particiones, mientras que el F1 se calcula sobre las predicciones reunidas. El tiempo de entrenamiento es el del modelo con todo el conjunto de datos, y el de inferencia (Inf.), por flujo y en lotes grandes. 56
6.3 Indicadores de calidad de las dos generaciones del conjunto de datos y F1 macro de Random Forest con cada una. La fila de fases tras un DDoS sin límite considera las tres fases siguientes a uno; en las demás fases la proporción es del 3,4 % en ambas generaciones. Los dos valores de la última fila de indicadores corresponden a las dos intensidades limitadas. 58
6.4 Métricas por clase de Random Forest sin conexión (validación cruzada agrupada por fase, 254.283 flujos) y en tiempo de ejecución (diez ejecuciones de la batería, 14.788 flujos). En tiempo de ejecución, la fila macro agrupa los flujos de todas las ejecuciones, del mismo modo que sin conexión agrupa las predicciones de las cinco particiones; la media por ejecución se recoge en la Tabla C.1. 59
6.5 Mitigación por conversación en las diez ejecuciones de la batería. Conv.: conversaciones observadas. Ataque: conversaciones de ataque. Bloq.: de ataque bloqueadas. Falsos: conversaciones legítimas bloqueadas. Bloqueos: decisiones de bloqueo, cada una de las cuales instala la regla de descarte en los cinco *switches*. 1.º: tiempo medio desde el primer flujo del ataque hasta el primer bloqueo. 61
6.6 Exhaustividad (Exh.) por flujo de cada ataque antes y después del primer bloqueo de su prueba, en las diez ejecuciones. . . . 62
6.7 Impacto en la infraestructura por tipo de tráfico. La CPU se expresa respecto de un núcleo, por lo que puede superar el 100 %, y su media se calcula sobre los eventos registrados. La latencia del plano de control solo existe para los flujos que provocan un bloqueo. La inferencia es por flujo, dentro de un lote. 63
6.8 Resumen del coste para la infraestructura. El rango de la media recorre los cuatro tipos de tráfico, y el máximo es el mayor de los cuatro. 65

7.1 Objetivos específicos y resultado de cada uno. 69

C.1 Métricas macro de la detección en tiempo de ejecución en cada ejecución de la batería. Prec.: precisión. Exh.: exhaustividad. 83

Capítulo 1

Motivación

1.1. Contexto

1.1.1. Un panorama de amenazas en crecimiento

Las redes de comunicaciones son hoy un componente esencial de empresas, centros educativos y hogares [1], de modo que protegerlas frente a ataques es una necesidad cotidiana. La literatura señala que el problema va a más: la frecuencia de los ataques de denegación de servicio distribuida (DDoS) ha aumentado y su intensidad ha crecido hasta pasar de los Gbps a los Tbps [2], mientras que las amenazas en general son cada vez más sofisticadas [3]. Frente a este escenario, la operación manual de la seguridad se queda corta: depender de personal técnico ralentiza la detección y la corrección de los incidentes, y favorece errores de configuración y una aplicación irregular de las políticas de seguridad [3]. De ahí el interés por automatizar la respuesta.

Este trabajo se centra en tres familias de ataques que afectan a distintos aspectos de la seguridad de una red:

- ***Scanning.*** Sondeo sistemático de equipos y puertos para descubrir qué servicios hay expuestos, habitualmente como paso previo a otros ataques.
- ***Spoofing.*** Suplantación de identidad mediante la falsificación de direcciones. Se consideran el *spoofing* del protocolo de resolución de direcciones (*Address Resolution Protocol*, ARP), en el que se difunden respuestas ARP falsas para asociar la dirección IP (*Internet Protocol*) de un equipo legítimo a la dirección MAC (*Media Access Control*) del atacante, una amenaza señalada también en redes SDN [4], y el *spoofing* de IP, que falsifica la dirección IP de origen [5].
- **DDoS.** Ataque en el que múltiples orígenes saturan los recursos de la víctima hasta degradar o interrumpir su servicio [5, 2].

1.1.2. Redes definidas por software y seguridad

En las redes tradicionales, las funciones de los planos de datos, control y gestión están acopladas verticalmente en los propios dispositivos de reenvío, lo que dificulta su gestión [6]. Las Redes Definidas por Software (*Software-Defined Networking*, SDN), un paradigma de arquitectura de red que permite gestionar y programar su comportamiento mediante software, rompen este acoplamiento: separan el plano de control del plano de datos y centralizan lógicamente el control en un controlador software [6, 1]. Ese controlador dispone de una visión global de la red y ofrece interfaces programables y estandarizadas [7]. En la práctica, el controlador se comunica con los *switches* mediante un protocolo como OpenFlow [8], que ofrece una interfaz estándar para añadir y eliminar entradas en la tabla de flujos de los *switches*; cada entrada asocia un patrón de cabeceras con una acción, entre las que se cuenta descartar los paquetes del flujo, algo que los propios autores del protocolo señalan como útil para frenar ataques de denegación de servicio [8]. En este trabajo se emplea la versión 1.3 de la especificación [9], en la que el controlador puede gestionar esas entradas tanto de forma reactiva, en respuesta a los paquetes, como proactiva.

Estas características resultan atractivas para la seguridad. La visión global permite monitorizar el tráfico desde un único punto y la programabilidad permite desplegar políticas de seguridad de forma dinámica en toda la red, lo que habilita respuestas automatizadas ante las amenazas [10]. SDN facilita, además, implementar sistemas de detección de amenazas como aplicaciones software sobre interfaces abiertas [11].

Sin embargo, la misma centralización introduce riesgos nuevos. Se amplía la superficie de ataque y aparece la posibilidad de que falle el control centralizado [10]. Un ataque de denegación de servicio contra un controlador que gestiona muchos dispositivos resulta más destructivo que uno dirigido contra un único *router* [1], y de hecho el controlador es el principal objetivo de los DDoS en SDN [2]. A ello se suma que la programabilidad puede ampliar la superficie de ataque al introducir software potencialmente vulnerable en el controlador, las interfaces de programación y las aplicaciones [12], y que se han identificado ataques en los planos de aplicación, control y datos [13]. Por tanto, SDN ofrece herramientas adicionales para defender la red, pero obliga a proteger también la propia infraestructura SDN.

1.1.3. Aprendizaje automático para detectar y mitigar ataques en SDN

El aprendizaje automático y el aprendizaje profundo se emplean ampliamente para reforzar la seguridad en SDN, ya que permiten identificar patrones de ataque en grandes volúmenes de datos y apoyar una toma de decisiones rápida [14]. La revisión sistemática de Doğan et al. [14], que parte

de 433 artículos publicados entre 2020 y 2024 y selecciona 163 para su taxonomía, concluye que la calidad de los conjuntos de datos es determinante para la robustez de los modelos y que siguen sin resolverse retos como la detección en tiempo real, la escalabilidad en entornos con varios controladores y la falta de conjuntos de datos completos específicos de SDN.

Los trabajos concretos son variados. Alzahrani y Alenazi [15] generan dos conjuntos de datos con Mininet [16] y Ryu [17] que contienen tráfico normal y distintos ataques, entre ellos inundaciones y escaneos de puertos, y entrenan con ellos varios clasificadores supervisados. Hasan et al. [18] aplican selección de características y comparan modelos de aprendizaje automático para detectar DDoS, complementando el análisis con una red emulada con Mininet y Ryu, y ALesawi y Alawadi [19] proponen detectar y mitigar DDoS en tiempo real combinando SDN, aprendizaje automático y selección de características. En el ámbito del *spoofing*, Gowda y Dayananda [4] detectan ataques ARP en un controlador Ryu comparando las direcciones MAC e IP de los remitentes con una asociación previamente aprendida, y bloquean los paquetes de la MAC atacante. Por último, Linn et al. [20] generan tráfico en fases temporales y emplean validación cruzada agrupada para evitar fugas de datos al evaluar un detector de DDoS.

1.1.4. Planteamiento del trabajo

De este panorama se desprenden varias ideas que motivan el trabajo. En primer lugar, buena parte de los trabajos consultados se centra en un único tipo de ataque, sobre todo el DDoS [2, 7, 11, 18, 21, 20, 19], cuando un sistema de defensa real ha de distinguir entre amenazas de naturaleza muy distinta y diferenciarlas, a su vez, del tráfico legítimo. En segundo lugar, el realismo de los datos condiciona la validez de los resultados: los conjuntos más utilizados para evaluar detectores de DDoS en SDN son sintéticos de carácter privado o poco realistas [7], y se echan en falta conjuntos específicos de SDN [14]. En tercer lugar, la evaluación exige cuidado para no sobreestimar el rendimiento, por ejemplo evitando que muestras de una misma fase de tráfico caigan a la vez en entrenamiento y prueba [20]. Por último, un detector solo reduce el riesgo si se integra con una respuesta efectiva, y la detección en tiempo real sigue siendo un reto abierto [14].

Con esta motivación, el presente trabajo explora el uso combinado de SDN e Inteligencia Artificial para detectar y mitigar los tres tipos de ataque descritos, mediante un análisis teórico y una implementación práctica en un entorno emulado. La propuesta es un sistema completo, organizado en tres fases (Figura 1.1). En la primera, se genera un conjunto de datos propio, con tráfico normal y de ataque etiquetado por flujo, sobre una red emulada con Mininet y controlada con Ryu. En la segunda, se entrenan y comparan tres modelos de aprendizaje automático con scikit-learn [22], evaluados de forma que se eviten las fugas de información. En la tercera, el controlador

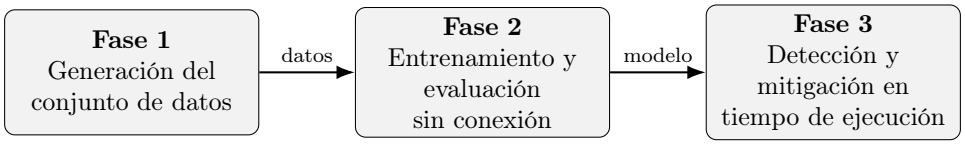


Figura 1.1: Visión general de las etapas propuestas para el TFG.

clasifica en tiempo de ejecución cada flujo con el mejor modelo y bloquea las conversaciones de ataque mediante reglas OpenFlow. Aquí, «en tiempo de ejecución» significa que la clasificación se realiza sobre el tráfico mientras este se genera, con sondeos sobre estadísticas del *switch* cada segundo; no debe entenderse como una respuesta instantánea.

El trabajo tiene un alcance acotado. El sistema se evalúa en una red emulada, con un único controlador y una topología fija en árbol de 5 *switches* y 16 *hosts*, dimensionada teniendo en cuenta los recursos disponibles en el equipo utilizado para realizar la emulación, y considera las tres familias de ataque mencionadas. No pretende ser un sistema de detección de intrusiones listo para producción, sino un banco de pruebas con el que estudiar hasta dónde llega la combinación de SDN y aprendizaje automático y cuáles son sus límites, que se analizan en el Capítulo 7.

1.2. Objetivos

1.2.1. Objetivo general

El objetivo general de este trabajo es explorar el uso combinado de la Inteligencia Artificial y de las Redes Definidas por Software para detectar y mitigar en tiempo de ejecución ataques de *scanning*, *spoofing* y DDoS, y evaluar la capacidad de esta combinación para responder ante ellos con un tiempo de respuesta realista, mediante un análisis teórico y una implementación práctica en un entorno emulado.

1.2.2. Objetivos específicos

El objetivo general se concreta en los siguientes objetivos específicos:

- **OBJ_1.** Revisar el estado del arte sobre la arquitectura SDN, las amenazas de seguridad que la afectan y el uso del aprendizaje automático para detectar y mitigar ataques.

- **OBJ_2.** Diseñar y construir un entorno experimental reproducible, formado por una red emulada con Mininet, un controlador Ryu con OpenFlow 1.3 y un procedimiento de instalación automatizado.

- **OBJ_3.** Generar un conjunto de datos propio, etiquetado por flujo, con tráfico normal y de ataque de las tres familias consideradas: *scanning*, *spoofing* (ARP e IP) y DDoS.
- **OBJ_4.** Definir el preprocesado y las características de cada flujo, incluidas las que describen patrones entre flujos en una ventana temporal, de modo que se calculen igual en el entrenamiento y en la detección en tiempo de ejecución.
- **OBJ_5.** Entrenar y comparar tres modelos de aprendizaje automático —regresión logística, árbol de decisión y Random Forest— con una validación que evite fugas de información entre entrenamiento y prueba, y seleccionar el mejor.
- **OBJ_6.** Integrar el modelo seleccionado en el controlador para clasificar en tiempo de ejecución cada flujo observado.
- **OBJ_7.** Diseñar un mecanismo de mitigación basado en reglas OpenFlow que bloquee las conversaciones de ataque de forma temporal y limite el impacto de los falsos positivos.
- **OBJ_8.** Evaluar por separado la detección y la mitigación, y medir el impacto del sistema en la infraestructura (uso de CPU del controlador, latencia del plano de control y tiempo de inferencia).
- **OBJ_9.** Analizar los aspectos positivos así como las limitaciones del enfoque y las líneas de trabajo futuro.

1.3. Organización de la memoria

La memoria se organiza en siete capítulos, además de los anexos, el glosario y la bibliografía:

- El **Capítulo 1**, el presente, expone el contexto, los objetivos y la estructura del documento.
- El **Capítulo 2** describe la planificación del proyecto: la metodología, las etapas, los recursos empleados, la planificación temporal y el presupuesto.
- El **Capítulo 3** revisa el estado del arte sobre SDN, su seguridad y la detección y mitigación de ataques con aprendizaje automático.
- El **Capítulo 4** presenta el diseño del sistema y justifica las decisiones tomadas: arquitectura, generación de datos, características, modelos, evaluación y estrategia de mitigación.

- El **Capítulo 5** explica cómo se ha implementado ese diseño: el entorno, el controlador, el generador de tráfico, el *pipeline* de aprendizaje automático y la defensa en tiempo de ejecución.
- El **Capítulo 6** recoge la experimentación: primero los resultados de las tres fases y después su discusión.
- El **Capítulo 7** presenta las conclusiones, las limitaciones del trabajo y las líneas de trabajo futuro.
- Los **anexos** reúnen el código relevante y otro material complementario para quien desee profundizar, el **glosario** define los términos y siglas empleados y la **bibliografía** recoge las fuentes citadas.

Capítulo 2

Planificación

Este capítulo describe cómo se organizó el trabajo: la metodología seguida, las etapas en que se dividió, los recursos empleados, su distribución en el tiempo y el coste que habría supuesto en un entorno profesional. El Trabajo de Fin de Grado tiene una carga de 300 horas (12 créditos ECTS) y se desarrolló entre mayo y octubre de 2026.

2.1. Metodología de planificación

El trabajo se planificó por etapas con un orden natural de dependencia entre ellas. La generación del conjunto de datos produce el material con el que se entrenan los modelos, y el mejor de ellos es el que el controlador despliega en la detección y mitigación en tiempo de ejecución (Figura 1.1). De ahí que el diseño, la implementación y la experimentación se agrupasen en torno a esas tres piezas, precedidos por una etapa de análisis y seguidos por la redacción de la memoria.

2.2. Etapas del proyecto

El proyecto se dividió en cinco etapas. La Figura 2.1 de la Sección 2.4 muestra su duración y su distribución en el tiempo.

2.2.1. Análisis preliminar y revisión bibliográfica

La primera etapa, de unas 50 horas, se dedicó a comprender el problema y a situar el trabajo en la literatura. Se estudió la arquitectura SDN y el protocolo OpenFlow [8, 9], las amenazas de seguridad que afectan a este tipo de redes y las propuestas que emplean aprendizaje automático para detectarlas y mitigarlas. La revisión se apoyó en una veintena de artículos, entre revisiones sistemáticas y trabajos con propuestas concretas, y en un libro de texto de referencia sobre redes [5].

De esta etapa salieron las ideas que motivan el trabajo y que se recogen en el Capítulo 1: la tendencia a estudiar un único tipo de ataque, la importancia del realismo de los datos, el riesgo de fugas de información en la evaluación y la dificultad de la detección en tiempo de ejecución. Con ellas se fijaron el alcance (los ataques de *scanning*, *spoofing* y DDoS), los objetivos y el conjunto de herramientas a emplear. Su resultado más directo es el estado del arte del capítulo 3.

2.2.2. Diseño de la arquitectura y estrategia de defensa

La segunda etapa, de unas 40 horas, definió cómo resolver el problema antes de programarlo. Se estableció la arquitectura en tres fases del sistema (conjunto de datos, aprendizaje automático y defensa en tiempo de ejecución), la topología de la red emulada, el modo en que el controlador observa el tráfico, las características que describen cada flujo y el criterio con el que se etiqueta. Se decidió también cómo evaluar los modelos, de forma que no hubiera fugas de información entre entrenamiento y prueba, y cómo mitigar: bloquear de forma temporal la conversación de ataque, tras confirmarla varias veces, para acotar el efecto de un falso positivo.

Estas decisiones y su justificación se desarrollan en el Capítulo 4.

2.2.3. Implementación y desarrollo de pruebas

La etapa más extensa, de unas 100 horas, construyó el sistema. Incluyó el procedimiento de instalación automatizado, que parte de una máquina limpia y deja el entorno listo; la topología Mininet [16]; el controlador Ryu [17] que observa los flujos y los etiqueta; el generador de tráfico normal y de ataque; y el flujo de trabajo de aprendizaje automático, desde el preprocesado hasta la evaluación. Se desarrolló además el controlador de defensa, con la clasificación en tiempo de ejecución y la mitigación, y el código que genera las tablas y las figuras de resultados. El diseño que implementa se describe en el capítulo 4, y su construcción, en el capítulo 5.

2.2.4. Experimentación y análisis de resultados

La cuarta etapa, de unas 40 horas, recoge la ejecución de los experimentos, el cálculo de las métricas y el análisis de sus resultados. Se solapó con la implementación: los primeros experimentos se hicieron con la generación del conjunto de datos, antes de que el sistema estuviera terminado. Se generaron dos conjuntos de datos independientes con la misma configuración, de unas seis horas de duración cada uno, para conocer cuánto varían los resultados entre generaciones. Con los dos se entrenaron y evaluaron los tres modelos; con el primero, que muestra menos signos de saturación del entorno (Sección 6.3.4), se entrenó el modelo desplegado, y con él se realizaron diez repeticiones de la batería de pruebas.

Componente	Característica
Equipo	Portátil ASUS ROG Strix G15 (G513RM)
Procesador	AMD Ryzen 7 6800H, 8 núcleos y 16 hilos, con una frecuencia máxima de 4,785 GHz
Memoria RAM	16 GiB (dos módulos SODIMM de 8 GiB)
Almacenamiento	SSD NVMe de 1 TB (Micron 2450); el sistema Linux ocupa una partición ext4 de unos 242 GB
Sistema operativo	Ubuntu Desktop 22.04.5 LTS, instalado de forma nativa junto a Windows

Tabla 2.1: Equipo empleado en el desarrollo y la experimentación.

También forma parte de esta etapa el estudio de las limitaciones de los resultados, como el sesgo que introduce la propia mitigación en la medición de la detección. Los resultados y su discusión se presentan en el Capítulo 6.

2.2.5. Redacción, revisión y cierre del proyecto

La última etapa, de unas 70 horas, ocupó el mes de octubre. Comprende la redacción de la memoria, su revisión con el director y el cierre del proyecto. La memoria se ha escrito en LaTeX [23].

2.3. Recursos utilizados

2.3.1. Recursos humanos

Participaron dos personas:

- **La estudiante**, Clara García Fuentes, autora del trabajo, que realizó todas sus etapas: el análisis, el diseño, la implementación, la experimentación y la redacción de la memoria, con una dedicación total de 300 horas.
- **El director**, José Antonio Gómez Hernández, que orientó y supervisó el trabajo y revisó la memoria, con una dedicación estimada de 20 horas.

2.3.2. Recursos hardware

Todo el trabajo se realizó en un único equipo portátil. En él se ejecutaron la red emulada, el controlador y la generación de tráfico, además del entrenamiento de los modelos. No se empleó ningún dispositivo de red físico. Las características del equipo se resumen en la Tabla 2.1.

Herramienta	Versión	Uso en el proyecto
Entorno y lenguaje		
Ubuntu Desktop [24]	22.04.5 LTS	Sistema operativo
Python [25]	3.10.12	Lenguaje de todo el sistema
Eventlet [26]	0.35.2	Dependencia de Ryu
Redes definidas por software		
Mininet [16]	2.3.0	Emulación de la red
Open vSwitch [27]	2.17.12	*Switches* virtuales de la red emulada
Ryu [17]	4.34	Controlador SDN (OpenFlow 1.3)
Generación de tráfico		
Scapy [28]	2.7.0	Envenenamiento ARP (*spoofing*)
`hping3` [29]	3.a2	Ataques DDoS, sondas ACK y suplantación de IP
Nmap [30]	7.80	Escaneo de puertos
iPerf [31]	2.1.5	Tráfico normal TCP y UDP
Datos y aprendizaje automático		
scikit-learn [22]	1.7.2	Modelos, validación y métricas
pandas [32]	2.3.3	Manejo del conjunto de datos
NumPy [33]	1.26.4	Cálculo numérico
Joblib [34]	1.6.0	Guardado y carga de los modelos
Matplotlib [35]	3.5.1	Gráficas de resultados
psutil [36]	7.2.2	Medición de la CPU del controlador
Desarrollo, documentación y control de versiones		
Visual Studio Code [37]	1.133.0	Editor de código y de la memoria
LaTeX [23]	TeX Live 2022	Redacción de la memoria
Git [38]	2.34.1	Control de versiones
GitHub [39]	—	Repositorio remoto del proyecto (servicio web)

Tabla 2.2: Herramientas software empleadas y versiones del entorno con el que se obtuvieron los resultados.

2.3.3. Recursos software

El sistema se apoya íntegramente en software libre o gratuito. La Tabla 2.2 recoge las herramientas empleadas, con las versiones fijadas en el entorno con el que se obtuvieron los resultados. Las herramientas de sistema (Mininet, Open vSwitch, Nmap, `hping3` e iPerf) proceden de los repositorios de Ubuntu, y las bibliotecas de Python se instalan en un entorno virtual aislado.

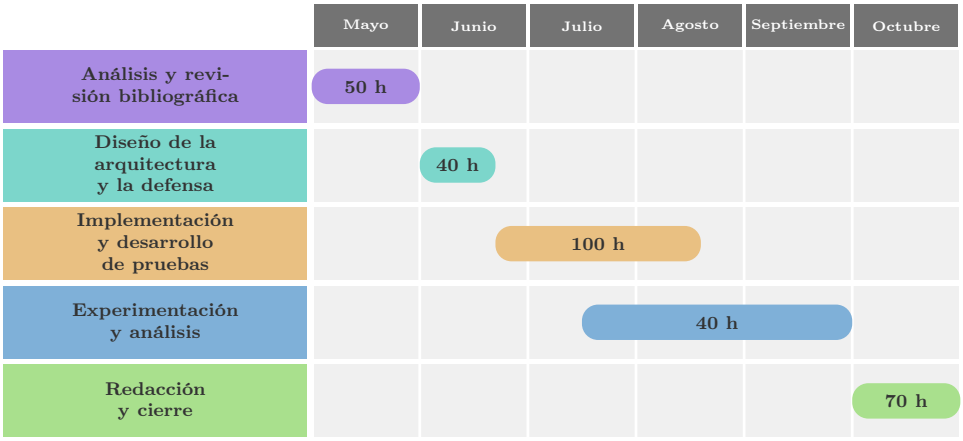


Figura 2.1: Diagrama de Gantt del proyecto, con las horas estimadas de cada etapa.

2.4. Planificación temporal

La Figura 2.1 muestra la distribución de las cinco etapas a lo largo de los seis meses, con las horas estimadas de cada una. El solape entre la implementación y la experimentación se debe a que los primeros experimentos se realizaron con la generación del conjunto de datos, antes de terminar el sistema.

Sobre estas horas estimadas, la dedicación media fue de unas 10 horas semanales entre mayo y septiembre (230 horas en torno a 22 semanas) y de unas 16 horas semanales en octubre (70 horas en algo más de cuatro semanas).

2.5. Presupuesto

El trabajo no tuvo coste económico real para la estudiante. Este presupuesto estima lo que habría costado realizarlo en un entorno profesional, con cuatro conceptos: el trabajo de la estudiante, la dirección del trabajo, el uso del equipo y el software. No se imputan otros gastos de menor entidad, como la electricidad o la conexión a Internet. Los valores de referencia que se emplean son supuestos razonables, y no cifras reales de contratación.

2.5.1. Costes de personal

Estudiante. Para estimar el coste horario se parte de un salario anual de 30.000 €, el límite inferior del rango de 30.000 a 35.000 € que se indica para un ingeniero informático junior en España [40]. Se supone que las cotizaciones a cargo de la empresa añaden en torno a un 30 %, lo que da un coste anual

Etapa	Horas	Tarifa (€/h)	Coste (€)
Análisis preliminar y revisión bibliográfica	50	22	1.100
Diseño de la arquitectura y la estrategia de defensa	40	22	880
Implementación y desarrollo de pruebas	100	22	2.200
Experimentación y análisis de resultados	40	22	880
Redacción, revisión y cierre	70	22	1.540
Total	**300**		**6.600**

Tabla 2.3: Coste de la estudiante por etapas.

para la empresa de unos 39.000 €, y una jornada de 1.800 horas al año. El coste resultante es de unos 21,7 € por hora, que se redondea a 22 €.

Con esa tarifa, las 300 horas de trabajo suponen 6.600 €, repartidos entre las etapas como indica la Tabla 2.3.

Director. Se estima una dedicación de 20 horas a lo largo del proyecto, que comprende las tutorías y las revisiones de la implementación y de la memoria. Para su coste horario se toma el límite inferior del rango de 60.000 a 80.000 € anuales que se indica para un perfil sénior [40], con las mismas cotizaciones y la misma jornada anual: 60.000 € × 1,3 / 1.800 h ≈ 43,3 € por hora, que se redondea a 43 €. La dirección supone así 20 h × 43 €/h = 860 €.

2.5.2. Costes de hardware

El equipo se utilizó durante los seis meses del proyecto. Su coste se imputa mediante una amortización lineal sobre una vida útil de cuatro años (48 meses). Con un precio de compra de 1.200 €, el coste atribuible al proyecto es de $1.200 \times 6/48 = 150$ €. Se amortiza en lugar de imputar el precio completo porque el equipo conserva su utilidad una vez terminado el trabajo.

2.5.3. Costes de software

Todas las herramientas empleadas (Tabla 2.2) son de software libre o gratuito, por lo que no suponen coste de licencia.

2.5.4. Resumen del presupuesto

La Tabla 2.4 reúne los cuatro conceptos. El coste total estimado asciende a 7.610 €, del que los costes de personal son, con diferencia, la parte principal.

Concepto	**Coste (€)**
Personal: estudiante (300 h a 22 €/h)	6.600
Personal: director (20 h a 43 €/h)	860
Hardware (amortización del portátil durante 6 meses)	150
Software (licencias)	0
Total	**7.610**

Tabla 2.4: Resumen del presupuesto.

Capítulo 3

Estado del arte

Este capítulo sitúa el trabajo en la literatura. Se organiza de lo general a lo particular. Primero, se abordan las Redes Definidas por Software y el protocolo OpenFlow (Sección 3.1). A continuación, se revisan las amenazas de seguridad que las afectan (Sección 3.2). Se describe el uso del aprendizaje automático para detectarlas y mitigarlas (Sección 3.3). Para finalizar, se revisan los trabajos concretos más cercanos al presente (Sección 3.4). El capítulo se cierra con un análisis de las carencias que se repiten en esos trabajos (Sección 3.5) y con el posicionamiento de este (Sección 3.6).

3.1. Redes definidas por software

3.1.1. Del modelo tradicional a SDN

En una red tradicional, el plano de control y el plano de datos están mezclados en cada dispositivo: el control es distribuido y se ejecuta en los propios equipos, cuya funcionalidad se implementa en gran medida en hardware dedicado [41]. Los planos de datos, de control y de gestión quedan así acoplados verticalmente [6], y esa rigidez hace que las redes IP resulten complejas de gestionar y de evolucionar [42].

Las Redes Definidas por Software (SDN) responden a ese problema separando el plano de control del de datos. Según la revisión histórica de Waseem et al. [42], esa idea se gestó en tres etapas: las redes activas, desde mediados de los años noventa, en las que los nodos de la red podían realizar operaciones personalizadas sobre los paquetes [6]; la separación de los planos de control y de datos, entre 2001 y 2007; y la aparición de la interfaz OpenFlow y de los sistemas operativos de red, entre 2007 y 2010. El propio artículo que presentó OpenFlow [8] partía de una dificultad práctica: el parque instalado de equipos y la reticencia a experimentar con tráfico de producción suponían una barrera muy alta para probar ideas nuevas, hasta el punto de que la infraestructura se percibía como osificada. Su propuesta fue aprovechar

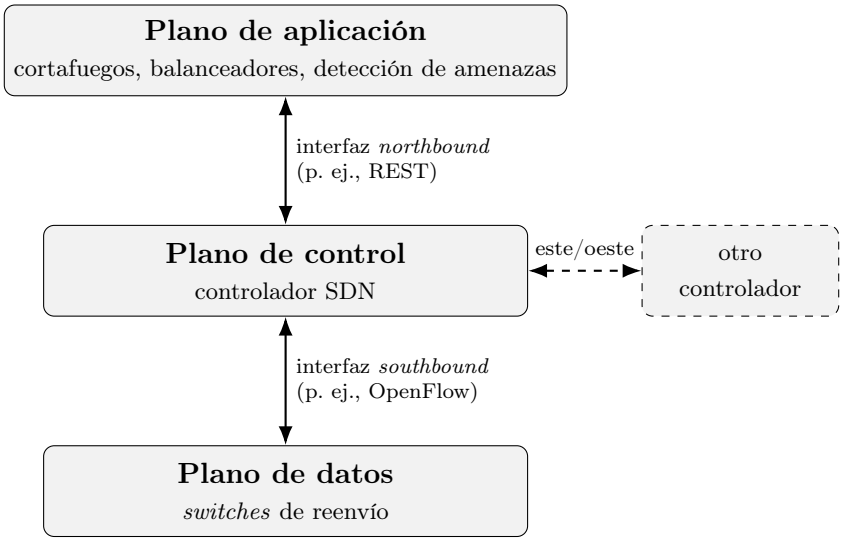


Figura 3.1: Arquitectura SDN: planos e interfaces.

una función que los *switches* y *routers* ya incorporaban, las tablas de flujos, y ofrecer un protocolo abierto para programarlas.

Los mismos autores [42] resumen SDN en cinco pilares: la separación de los planos de control y de datos, la abstracción de flujo, la programabilidad de la red, la neutralidad respecto al fabricante y el control centralizado. Con ello, los dispositivos de reenvío quedan dedicados a reenviar paquetes y las decisiones se toman en un controlador [41].

Cabe preguntarse si este cambio tiene un coste en rendimiento. Sadiku et al. [41] compararon en Mininet una red tradicional, una red SDN y una red que combinaba ambas, con enlaces de 10 Mbps, y midieron ancho de banda, caudal (*throughput*), latencia y *jitter*. En su experimento, la red tradicional y la SDN presentaron valores muy próximos (una latencia media de 0,152ms frente a 0,161ms), mientras que la red combinada fue la peor en las tres primeras magnitudes (0,247ms de latencia). Se trata de un único experimento de laboratorio con una topología concreta, que no permite generalizar, pero que indica que la separación de planos no implica por sí misma una degradación apreciable en ese escenario.

3.1.2. Arquitectura

La arquitectura SDN se describe habitualmente en tres niveles: 1) el plano de datos, formado por los dispositivos de reenvío, 2) el plano de control, donde reside el controlador, que actúa como «cerebro» de la red, y 3) el plano de aplicación, donde se ejecutan las aplicaciones de red [43, 6]. La Figura 3.1 los representa junto a las interfaces que los comunican.

La comunicación entre planos se realiza a través de interfaces. La interfaz

Campo	**Descripción**
Campos de coincidencia	Puerto de entrada, cabeceras del paquete y, opcionalmente, metadatos de una tabla anterior
Prioridad	Precedencia de la entrada en la coincidencia
Contadores	Se actualizan con los paquetes que coinciden con la entrada
Instrucciones	Modifican el conjunto de acciones o el procesamiento en el *pipeline*
Timeouts	Tiempo máximo, o de inactividad, antes de que el *switch* expire la entrada
Cookie	Valor opaco elegido por el controlador, que no se usa al procesar paquetes

Tabla 3.1: Campos de una entrada de flujo en OpenFlow 1.3, según [9].

southbound, entre el controlador y los dispositivos, está dominada por OpenFlow, que es la opción más reconocida, aunque existen alternativas como OVSDB o NETCONF [43, 6]. La interfaz *northbound*, entre el controlador y las aplicaciones, carece de estandarización, y la más extendida es REST [43, 6]. Las interfaces *este/oeste* comunican controladores entre sí para compartir información, lo que permite desplegar varios controladores manteniendo una visión lógicamente centralizada de la red [42, 43].

3.1.3. OpenFlow

OpenFlow define el modo en que el controlador programa los *switches*. Un *switch* OpenFlow consta, al menos, de una tabla de flujos, un canal seguro hacia el controlador y el propio protocolo [8]. En la versión 1.3 de la especificación [9], que es la que se emplea en este trabajo, el *switch* puede contar con varias tablas de flujos organizadas en una secuencia (*pipeline*) y el controlador puede añadir, modificar y eliminar entradas tanto de forma reactiva, en respuesta a paquetes, como proactiva. Los paquetes se comparan con las entradas por orden de prioridad y se aplica la primera que coincide [9].

Cada entrada de flujo consta de los campos de la Tabla 3.1. Para este trabajo son especialmente relevantes seis de ellos: 1) los campos de coincidencia, que definen a qué paquetes se aplica la entrada y, por tanto, qué se considera un flujo y qué conversación se bloquea, 2) la prioridad, que decide qué entrada se aplica cuando coinciden varias, 3) los contadores, que permiten conocer cuánto tráfico ha coincidido con cada entrada, 4) las instrucciones, que definen qué se hace con los paquetes que coinciden, 5) los *timeouts*, que determinan cuándo expira una entrada, y 6) la *cookie*, que facilita la gestión de las entradas de flujo.

De la especificación [9] interesan las características que sustentan lo que

hace el sistema: observar el tráfico, mitigar los ataques, conmutar de forma reactiva y comunicarse con el controlador. Las dos primeras actúan sobre las entradas de flujo (Tabla 3.1), la tercera sobre la tabla de flujos del *switch* y la cuarta sobre el canal que lo une al controlador.

Observar el tráfico.

- **Contadores.** Para cada entrada, el *switch* mantiene el número de paquetes y de bytes recibidos y la duración de la entrada. El controlador los consulta mediante una petición de estadísticas de flujo, que puede filtrarse por campos de coincidencia, y con ellos calcula tasas de paquetes y de bytes.

- **Campos de coincidencia.** Entre los campos que pueden emplearse figuran las direcciones MAC e IP, los puertos TCP y UDP y los campos del protocolo ARP, que en la mayoría de los casos requieren que el tipo de protocolo coincida previamente con el esperado.

Mitigar.

- **Descarte.** No existe una acción explícita de descarte: se descartan los paquetes cuyo conjunto de acciones no contiene ninguna acción de salida, por ejemplo porque la lista de instrucciones está vacía. El artículo original ya señalaba que descartar los paquetes de un flujo permite, entre otras cosas, frenar ataques de denegación de servicio [8].

- **Expiración.** Un *hard timeout* elimina la entrada tras un número de segundos, tanto si ha coincidido con paquetes como si no, y un *idle timeout* la elimina cuando no ha coincidido con ninguno durante ese tiempo. Si ambos son cero, la entrada es permanente.

Conmutar de forma reactiva. Los paquetes que no coinciden con ninguna entrada se tratan según la entrada de fallo de coincidencia (*table-miss*), que puede enviarlos al controlador, descartarlos o dirigirlos a otra tabla. Si esta entrada no existe, la especificación establece que los paquetes sin coincidencia se descartan.

Comunicarse con el controlador. El canal entre *switch* y controlador suele cifrarse con TLS, pero puede funcionar directamente sobre TCP. La especificación recomienda además limitar la tasa de paquetes enviados al controlador para evitar su denegación de servicio.

3.1.4. Controladores y entornos de emulación

Existen decenas de controladores. Hussain et al. [6] catalogan en torno a treinta plataformas, y Gupta et al. [43] comparan los más extendidos (OpenDaylight, ONOS, POX, Ryu, Beacon, Floodlight, entre otros) en cuanto a lenguaje, arquitectura y soporte de interfaces. Ryu [17] está escrito en Python [25] y, según el catálogo de Hussain et al., es un controlador centralizado con interfaz REST que admite las versiones de OpenFlow de la 1.0 a la 1.5; en la clasificación por sencillez que esos mismos autores proponen figura en primer lugar, por delante de Floodlight, ONOS y OpenDaylight [6]. En la práctica, es también uno de los más utilizados en la literatura sobre detección con aprendizaje automático: la revisión de Doğan et al. [14] lo encuentra en 37 de los 163 estudios que analiza, por delante de Floodlight y ONOS (15 estudios cada uno). Bahashwan et al. [7], que se limitan a los trabajos sobre DDoS, señalan en cambio a POX como el controlador más empleado.

Para evaluar estos sistemas sin una red física, la herramienta más habitual es Mininet [16], que levanta una red virtual de *hosts* y *switches* en una sola máquina. Se habla de un entorno *emulado* y no *simulado* porque los *hosts* ejecutan aplicaciones reales y los *switches*, normalmente Open vSwitch [27], un software de reenvío real, de modo que el tráfico que se genera es tráfico de red real y no el resultado de un modelo. Bahashwan et al. [7] confirman que Mininet es el emulador empleado en los trabajos de su revisión, y que para generar los ataques se recurre a herramientas como `hping3` [29] y Scapy [28]; en esa misma línea, Alzahrani y Alenazi [15] combinan `hping3`, Scapy, Nmap [30] e iPerf [31] para generar tráfico de ataque y de fondo.

Este entorno tiene limitaciones que la literatura señala. Una red de Mininet no puede superar el ancho de banda de la máquina que la aloja [6], la validación en una sola máquina con topologías pequeñas es un reto habitual [13], y su comportamiento puede diferir del de redes reales [2]. Además, Doğan et al. [14] observan que solo cuatro de los estudios revisados emplearon infraestructura física, y que en 31 de ellos se usó un único controlador. Estas limitaciones se tendrán presentes al interpretar los resultados del Capítulo 6.

3.2. Seguridad en redes definidas por software

3.2.1. Una doble cara

La centralización y la programabilidad de SDN tienen una lectura favorable para la seguridad. La visibilidad global y la capacidad de desplegar políticas de forma dinámica permiten monitorizar el tráfico y reaccionar ante él desde un único punto [10]. Maleh et al. [1] apuntan que la programabilidad y la centralización mejoran la eficacia de las herramientas de seguridad, y que buena parte de las amenazas de SDN se heredan de las redes tradicionales, de modo que los sistemas de detección tradicionales pueden emplearse

Plano o interfaz	Amenazas señaladas	Fuentes
Aplicación	Autenticación, autorización y control de acceso débiles; aplicaciones maliciosas o con errores; inserción fraudulenta de reglas de flujo	[1, 12]
Northbound	Interfaz no estandarizada; autenticación débil de las API; inyección de código	[1, 12]
Control	Denegación de servicio y saturación del controlador; secuestro; acceso no autorizado; fallos en cascada	[1, 13, 12]
Southbound (controlador–switch)	En OpenFlow 1.3 el cifrado TLS no es necesariamente obligatorio, lo que habilita escucha y ataques de intermediario; envenenamiento ARP entre *switch* y controlador si no se usa SSL	[1, 13]
Datos	Tablas de flujos de tamaño limitado, con riesgo de desbordamiento; saturación del *buffer*; *flooding* y *spoofing*	[1, 13]

Tabla 3.2: Amenazas por plano o interfaz en SDN.

sin grandes modificaciones.

La lectura desfavorable es la ya apuntada en el Capítulo 1: las mismas características amplían la superficie de ataque y concentran el riesgo en el controlador [10, 42]. Un atacante que manipule el programa del controlador puede cambiar el comportamiento de toda la red subyacente [42].

3.2.2. Amenazas por plano e interfaz

Las revisiones sistemáticas coinciden en ordenar las amenazas según el plano o la interfaz que atacan. La Tabla 3.2 resume las principales a partir de Maleh et al. [1], Farooq et al. [13] y Diouf et al. [12].

Hay dos aspectos que ayudan a delimitar el alcance de este trabajo. El primero es que existe un eje de amenazas centrado en el *software* de SDN (controlador, API y aplicaciones), al que Diouf et al. [12] dedican una revisión de 58 publicaciones, en la que el controlador es el componente más estudiado (más de la mitad de los trabajos) y en la que señalan que explotar vulnerabilidades en el plano de aplicación o en una interfaz puede acabar comprometiendo el plano de control. Esa línea, centrada en vulnerabilidades del *software*, es distinta de la que se aborda aquí, que parte del tráfico de red. El segundo es que la propia revisión de Farooq et al. [13] apunta que la mayoría de las soluciones que analiza se centra en el DDoS contra el plano de control.

3.2.3. Los ataques considerados

Los tres ataques que aborda este trabajo aparecen en esas revisiones con distinta profundidad.

DDoS. Es el más estudiado. Kaur et al. [2] proponen clasificarlo en SDN según el plano atacado (control, datos, aplicación o canal de comunicación), la vulnerabilidad aprovechada (sobrecarga del agente OpenFlow, del *buffer*, de la tabla de flujos o saturación del controlador, entre otras), su intensidad (alta o baja tasa) y su objetivo. Farooq et al. [13] describen variantes de *flooding* (SYN, UDP y mensajes *packet-in*, con direcciones IP falsas) y advierten de que desplegar varios controladores no resuelve el problema, ya que el ataque puede acabar afectando a todos ellos.

Spoofing. Farooq et al. [13] distinguen el *spoofing* de ARP, de IP y de *switch*. En el de ARP, la dirección IP de un equipo legítimo queda asociada a la dirección MAC del atacante. Las defensas se agrupan, según la clasificación que recogen, en las basadas en patrones de tráfico, en la vinculación entre IP y MAC y en grafos de flujos. En Maleh et al. [1] figuran, por ejemplo, soluciones para el *spoofing* de IP (VAVE), comprobadores de *hosts* frente al envenenamiento ARP y arquitecturas que limitan los mensajes ARP y DHCP.

Scanning. Es el que menos atención recibe. Farooq et al. [13] le dedican apenas dos frases, en las que lo describen como el sondeo remoto de las direcciones IP de la red por parte de un intruso, y Maleh et al. [1] señalan que los ataques de escaneo suelen ser precursores de otros dirigidos, como la denegación de servicio. Entre las soluciones que recogen, el sistema de Giotis et al., basado en OpenFlow y sFlow, es capaz de detectar escaneos de puertos además de DDoS y gusanos [1].

En conjunto, Maleh et al. [1] advierten de un problema general: las soluciones de protección propuestas suelen diseñarse para un único tipo de ataque, y son muy pocas las que incluyen mecanismos para recuperar el estado de la red tras un ataque exitoso.

3.2.4. Seguridad automatizada

Yungaicela-Naula et al. [3] definen la automatización de la seguridad como la ejecución, por parte de máquinas, de acciones de seguridad para detectar, investigar, mitigar y prevenir amenazas con la mínima intervención humana. Su revisión permite situar el presente trabajo en cuatro aspectos.

En primer lugar, miden el nivel de automatización mediante cuatro propiedades (autorreparación, autoadaptación, autoconfiguración y autooptimización) y clasifican las soluciones reactivas en estadísticas, de aprendizaje

automático, de aprendizaje profundo, de adaptación en línea, de aprendizaje por refuerzo y de aprendizaje adversario. Este trabajo se sitúa entre las soluciones reactivas de aprendizaje automático: el modelo se entrena sin conexión y no se adapta en línea, de modo que la automatización se limita a detectar y mitigar sin intervención humana. En segundo lugar, señalan estrategias para que estas soluciones escalen: muestrear el tráfico, delegar decisiones en los *switches*, repartir la detección entre un análisis grueso en los *switches* y uno fino en el controlador, y distribuir los controladores. Este trabajo no aplica ninguna de ellas: un único controlador sondea y clasifica todos los flujos de la red, y solo la clasificación en lote (Sección 4.6) reduce la carga. Es una opción razonable para una red emulada pequeña, pero no permite extrapolar el comportamiento a redes mayores (Capítulo 7). En tercer lugar, advierten de que la mitigación más básica, bloquear o descartar, asume un rendimiento de detección del 100 %, y plantean alternativas como retrasar el tráfico o redirigirlo a un análisis más profundo. Este trabajo adopta el descarte, pero, para no depender de una detección perfecta, lo hace de forma temporal y solo tras tres confirmaciones (Sección 4.6), de modo que un bloqueo erróneo expira por sí mismo. Las alternativas que proponen no se evalúan y quedan como trabajo futuro (Sección 7.3).

Por último, recomiendan evaluar estas soluciones con más que métricas de clasificación, que ofrecen una valoración parcial: proponen medir la escalabilidad (retardo, CPU y memoria del controlador, ocupación de la tabla de flujos), la complejidad de la implementación y el coste, y añaden que es difícil encontrar estudios cuya evaluación sea a la vez exhaustiva y realista. Este trabajo sigue esa recomendación en parte: además de las métricas de clasificación, mide la CPU del controlador, la latencia del plano de control, el tiempo de inferencia y los bloqueos erróneos (Sección 4.7), pero no mide la memoria ni la ocupación de la tabla de flujos.

3.3. Aprendizaje automático para detectar y mitigar ataques en SDN

3.3.1. Panorama

La revisión de Doğan et al. [14] recoge 163 estudios de 2020 a 2024. Los algoritmos de aprendizaje automático clásico más empleados son las máquinas de vectores soporte (en 55 estudios) y Random Forest (en 53), mientras que árboles de decisión, k vecinos y *naive Bayes* se prefieren por su sencillez; el aprendizaje profundo aparece en 60 estudios, con redes convolucionales en 15 y LSTM en 9. Los autores señalan que los métodos de aprendizaje automático suelen preferirse a los de aprendizaje profundo por requerir menos recursos computacionales. En la revisión de Bahashwan et al. [7], centrada en DDoS y con 70 estudios, el 53 % emplea aprendizaje automático, el 38 %

aprendizaje profundo y el 9 % ambos.

3.3.2. Conjuntos de datos

Los datos son el punto más criticado. Según Doğan et al. [14], los conjuntos más frecuentes son los preparados por los propios autores, seguidos de NSL-KDD, InSDN y CICIDS2017, y muchos trabajos siguen empleando KDD'99 y NSL-KDD, que pueden no representar el panorama de amenazas actual. Convertir conjuntos de redes tradicionales a flujos SDN no siempre es práctico, y los ataques específicos de SDN no están representados en los conjuntos públicos. Bahashwan et al. [7] observan que los conjuntos privados sintéticos, seguidos de los no realistas, son los más utilizados, y que solo tres conjuntos realistas específicos de SDN son públicos. Kaur et al. [2] afirman que no existe un conjunto completo para el entorno SDN, que casi todos los investigadores emplean uno sintético y que la mayoría de los conjuntos presenta fuerte desequilibrio de clases.

El problema se observa en la práctica. El Sayed et al. [11] entrenan un detector con CICIDS2017 y lo prueban sobre otros conjuntos: con todas sus características, el exactitud es del 99,61 % sobre CICIDS2018, pero del 55,18 % sobre InSDN, y con un subconjunto de diez características el modelo no llega a identificar ningún DDoS en InSDN. Kaur et al. [2] recogen, por su parte, una red convolucional cuyo exactitud desciende del 95,18 % en la fase de diseño al 89,61 % en la de prueba sobre CICIDS2017.

La reproducibilidad es otra carencia: salvo dos estudios, la mayoría no comparte sus conjuntos de datos, y solo uno comparte el código [14].

3.3.3. Características y obtención de estadísticas

Un detector en SDN necesita describir cada flujo con características que el controlador pueda calcular. Las entradas de flujo ofrecen los contadores de paquetes y bytes y la duración (Sección 3.1.3), a partir de los cuales se derivan tasas y medias. Los trabajos difieren en la frecuencia con la que sondean al *switch* y en el número de características:

- El Sayed et al. [11] mapean las características de la herramienta CICFlowMeter a las que un controlador puede calcular, obtienen 48 y eliminan las direcciones IP de origen y destino para evitar el sobreajuste.
- Alzahrani y Alenazi [15] construyen dos conjuntos: uno basado en características de *host*, con 84 columnas reducidas a 51, y otro basado en el controlador, que sondea las estadísticas de flujo cada segundo con un *idle timeout* de 2 segundos y utiliza solo tres características (paquetes por segundo, bytes por segundo y bytes por paquete).
- Hasan et al. [18] sondean cada 10 segundos y calculan 22 características por flujo.

- Linn et al. [20] sondean cada 5 segundos, obtienen 20 características por flujo y retiran diez que consideran propensas a fugas de información, como la marca de tiempo o los identificadores de *switch*.

Doğan et al. [14] no recogen una lista concreta de características, pero sí señalan que la selección de características se hace por métodos muy variados: manual, análisis de componentes principales, correlación, chi-cuadrado o importancia en Random Forest, entre otros.

3.3.4. Evaluación y métricas

La métrica más habitual es la exactitud (*accuracy*): Bahashwan et al. [7] la encuentran en 58 de los 70 estudios, frente a 50 de exhaustividad (*recall*), 47 de precisión y 40 de F1, y recomiendan evaluar con métricas diversas. Doğan et al. [14] advierten de que el desequilibrio de clases produce métricas infladas, como el exactitud, y observan que la mayoría de los estudios informa de exactitudes superiores al 90 %. Las métricas computacionales son minoritarias: tiempo de entrenamiento o prueba en diez estudios, uso de CPU en ocho de los estudios [7].

El protocolo de partición condiciona la validez de los resultados. Ninguna de las cuatro revisiones sobre aprendizaje automático consultadas [14, 2, 7, 3] aborda de forma explícita las fugas de información entre entrenamiento y prueba. Los trabajos concretos reflejan la diversidad de enfoques (Sección 3.4): la mayoría parte los flujos de forma aleatoria, de modo que flujos de un mismo instante o de una misma conversación pueden caer a ambos lados de la partición, y solo uno de ellos agrupa por ventana temporal [20].

3.3.5. Detección en tiempo de ejecución y mitigación

Detectar un ataque no es lo mismo que mitigarlo. Doğan et al. [14] observan que la mayoría de los estudios de su taxonomía no incluye mitigación; entre las que sí lo hacen, mencionan descartar los flujos hacia el destino más atacado, limitar la tasa con *meters* de OpenFlow, redirigir a señuelos o aplicar listas de control de acceso y reglas de cortafuegos, y recogen respuestas de aproximadamente un segundo. Bahashwan et al. [7] añaden que la mayoría de los detectores se ejecutan dentro del controlador, lo que añade carga, y que solo unos pocos estudios combinan detección y mitigación.

Los propios autores de las revisiones identifican esta cuestión como un reto abierto: Doğan et al. [14] señalan la mitigación escalable y en tiempo real entre los retos pendientes, y Bahashwan et al. [7] reclaman prevención y no solo detección.

3.4. Trabajos relacionados

De la bibliografía consultada se seleccionan siete trabajos concretos, por ser los más próximos a este: uno sobre *spoofing* ARP [4], uno que cubre inundaciones y escaneos [15] y cinco centrados en DDoS [11, 18, 21, 20, 19]. La Tabla 3.3 los resume, junto con el presente trabajo, y a continuación se describen. Los resultados numéricos de cada trabajo se recogen tal como los comunican sus autores. No son comparables entre sí ni con los de esta memoria, ya que cada uno emplea un conjunto de datos, un protocolo y unas métricas distintos. A continuación, se pasa a describir los trabajos más relevantes y un resumen de sus principales características en la Tabla 3.3.

Gowda y Dayananda [4]. Detectan y previenen ataques ARP en un controlador Ryu con Open vSwitch sobre Mininet. No emplean aprendizaje automático: el controlador construye una tabla que asocia MAC, IP y puerto a partir de los paquetes ARP y bloquea al origen cuyos datos no coinciden con la entrada almacenada. Las pruebas consisten en un único escenario. Las únicas cifras que dan en el texto son el uso de CPU del controlador (2,0 % durante el ataque y 0,7 % con la solución); definen un tiempo de detección, pero solo lo muestran en una gráfica, sin cuantificarlo. Proponen aplicar aprendizaje automático como trabajo futuro.

Alzahrani y Alenazi [15]. Generan dos conjuntos con Mininet y Ryu y comparan ocho clasificadores binarios (ataque o normal). Con tres características obtenidas del controlador, el árbol de decisión alcanza un exactitud de 0,9992 en su partición aleatoria estratificada, y el modelo se integra en una aplicación de Ryu que bloquea los flujos clasificados como ataque; la mediana del caudal del tráfico normal pasa de 163 a 329 Mbps una vez bloqueado el ataque. No miden CPU ni latencia del controlador, ni analizan los bloqueos erróneos. Incluyen sondeos y escaneos de puertos entre sus ataques, pero no *spoofing*.

El Sayed et al. Proponen un detector de DDoS basado en un LSTM-autoencoder con selección de características, entrenado y probado sin conexión [11]. Es el único de estos trabajos que somete el modelo a una prueba entre conjuntos, con los resultados descritos en la Sección 3.3.2. Miden además el efecto sobre el controlador: Ryu sin modelo queda limitado a unos 3.800 paquetes por segundo, y la incorporación del modelo reduce su caudal entre un 2,8 % y un 6,4 % según el conjunto y el número de *switches*. La mitigación se describe en el marco teórico (descartar o redirigir a un señuelo), pero no se implementa.

Trabajo	Ataque	Datos	Modelos	Partición	En tiempo de ejecución y mitigación
Gowda y Dayananda [4]	*Spoofing* ARP	Sin conjunto de datos	Reglas (tabla MAC–IP–puerto)	No aplica	Sí, en Ryu; bloqueo del atacante
Alzahrani y Alenazi [15]	Inundaciones y sondeos	Propios (dos conjuntos)	KNN, SVM, DT, RF, AdaBoost, XGBoost, NB, MLP	Aleatoria estratificada 80/10/10	Prototipo en Ryu; bloqueo del flujo
El Sayed et al. [11]	DDoS	InSDN, CICIDS2017 y 2018	LSTM-autoencoder	Aleatoria 70/30 y entre conjuntos	No; sin mitigación
Hasan et al. [18]	DDoS	CICIoT2023 y propio	RF, KNN, DT, LR, NB	Aleatoria 80/20	Prueba «en tiempo real» con KNN; sin mitigación
Lai [21]	DoS	KDD99	LR, DT, RF	Validación cruzada sin detallar	No; sin mitigación
Linn et al. [20]	DDoS	Propio, por fases	SVM lineal, RF, XGBoost	Por fases y `GroupKFold` por ventana	No; sin mitigación
ALesawi y Alawadi [19]	DDoS	Propio (1.034.669 filas)	KNN, NB, RF	Aleatoria 70/30	Sí; bloqueo con reglas de flujo
Este trabajo	*Scanning*, *spoofing* y DDoS	Propio (254.283 flujos, 904 fases)	LR, DT, RF	`GroupKFold` por fase	Sí; reglas DROP temporales

Tabla 3.3: Trabajos relacionados con el presentado en este TFG. LR: regresión logística; DT: árbol de decisión; RF: Random Forest; NB: *naive Bayes*; KNN: k vecinos.

Hasan et al. Combinan el conjunto público CICIoT2023, de tráfico IoT y no específico de SDN, con un conjunto propio generado con Mininet y Ryu a partir de estadísticas del controlador [18]. Con tres características elegidas por importancia en Random Forest, el árbol de decisión alcanza 0,9997 de exactitud y Random Forest 0,9980, mientras que la regresión logística se queda en 0,4040 y *naive Bayes* en 0,2538. La prueba «en tiempo real» con KNN, de la que informan un exactitud del 100 %, no indica el número de muestras. No incluyen mitigación, que dejan como trabajo futuro.

Lai. Compara regresión logística, árbol de decisión y Random Forest, entre otros [21]. El resumen habla de un entorno SDN simulado, pero los resultados se presentan sobre KDD99, un conjunto que no es específico de SDN, y no

se indican controlador ni topología. Se informa del área bajo la curva ROC (por ejemplo, 0,66 de media macro para la regresión logística y 1,00 para Random Forest) y no de otras métricas. No hay integración en el controlador ni mitigación.

Linn et al. Es el trabajo más próximo en cuanto a metodología de evaluación [20]. Generan en Mininet y Ryu tráfico a lo largo de 1.200 segundos en seis fases (tráfico benigno disperso, ruido con ataques de baja tasa, inundación sostenida, ráfagas multivector, recuperación y de nuevo tráfico benigno disperso), sondean los flujos cada 5 segundos y eliminan características que consideran propensas a fugas. Entrenan con las tres primeras fases, prueban con las tres últimas y emplean validación cruzada agrupada por ventana temporal de 5 segundos. El mejor resultado que comunican es un F1 del 95,52 % con una tasa de falsas alarmas del 37,25 %, que los propios autores consideran una preocupación para un despliegue real. La detección se realiza sin conexión y no hay mitigación ni medida de sobrecarga.

ALesawi y Alawadi. Detectan y mitigan DDoS (inundaciones UDP, TCP SYN, ICMP y ataques LAND) con Ryu, OpenFlow 1.3 y una red de seis *switches* y 18 *hosts* [19]. Generan 1.034.669 filas con 22 columnas, seleccionan diez características mediante la intersección de tres rankings y comparan KNN, *naive Bayes* y Random Forest con una partición aleatoria 70/30, con exactitudes de hasta el 100 %. La aplicación del controlador clasifica el tráfico de forma continua y, ante un ataque, bloquea mediante reglas de flujo; según los autores, detecta y bloquea todos los ataques, incluido un Smurf no visto en el entrenamiento. Sin embargo, no describen la regla de bloqueo, no cuantifican la latencia de detección ni el uso de CPU, no informan de falsos positivos en tiempo de ejecución y sus resultados se basan en una única matriz de confusión.

3.5. Carencias en el estado del arte

Del análisis anterior se desprenden varias carencias, algunas señaladas por las propias revisiones y otras visibles al contrastar los trabajos entre sí:

1. **Cobertura de ataques.** Las soluciones se diseñan para un único tipo de ataque [1], y las revisiones sobre aprendizaje automático se centran en el DDoS [2, 7]. De los siete trabajos de la tabla 3.3, cinco se limitan al DDoS o al DoS, uno al *spoofing* ARP y uno incluye sondeos y escaneos, pero ninguno cubre a la vez *scanning*, *spoofing* y DDoS. Además, el escaneo apenas recibe atención en las revisiones de seguridad [13].

2. **Realismo de los datos.** Predominan los conjuntos sintéticos o no específicos de SDN [7, 2, 14], y los resultados sobre un conjunto pueden

no trasladarse a otro [11].

3. **Protocolo de evaluación.** Las revisiones no tratan las fugas de información [14, 2, 7, 3], y en la Tabla 3.3 cuatro trabajos parten los datos de forma aleatoria. Solo uno agrupa por ventana temporal [20]. Con particiones aleatorias, las puntuaciones cercanas al 100 % resultan difíciles de interpretar sin más información sobre el conjunto.

4. **Métricas.** La exactitud domina [7] y es sensible al desequilibrio de clases [14]. Las métricas que dan cuenta de lo que cuesta un bloqueo erróneo, como la tasa de falsas alarmas, aparecen en pocos trabajos, y en uno de ellos es elevada [20].

5. **Detección en tiempo de ejecución y mitigación.** La mayoría de los estudios no incluye mitigación [14], y pocos combinan detección y mitigación [7]. En la Tabla 3.3, tres trabajos la implementan [4, 15, 19], y ninguno cuantifica las conversaciones legítimas bloqueadas por error. Bloquear sin más supone dar por buena la detección [3].

6. **Coste para la infraestructura.** Las métricas de coste computacional son minoritarias [7], y la evaluación exhaustiva y realista escasea [3]. De los trabajos de la tabla, solo dos informan de alguna medida sobre el controlador (CPU o caudal) [4, 11].

7. **Entorno de validación.** Predominan los entornos virtuales de pequeño tamaño y los despliegues con un único controlador [14, 13].

3.6. Posicionamiento del trabajo

El presente trabajo no pretende superar las cifras de los trabajos anteriores, sino cubrir de forma conjunta varias de las carencias de la sección anterior en un mismo sistema de extremo a extremo. Para ello, el trabajo realizado se ha propuesto:

- abordar a la vez *scanning*, *spoofing* y DDoS, y distinguirlos del tráfico normal en una clasificación multiclase;

- generar un conjunto de datos propio, etiquetado por flujo y organizado por fases, con Mininet, el emulador más habitual en la literatura [7], y Ryu, uno de los controladores más empleados [14];

- evaluar los modelos con validación cruzada agrupada por fase, en la línea de lo propuesto por Linn et al. [20], y con métricas por clase y F1 macro, que no dependen de la clase mayoritaria como la exactitud;

- integrar el mejor modelo en el controlador y mitiga con reglas OpenFlow de descarte, de forma temporal y tras varias confirmaciones, de modo que un bloqueo erróneo expire por sí mismo, en lugar de dar por buena la primera detección [3];
- evaluar por separado la detección y la mitigación, cuantificar las conversaciones legítimas bloqueadas por error y medir el uso de CPU del controlador, la latencia del plano de control y el tiempo de inferencia, como recomiendan las revisiones [3, 7].

Los límites que presenta la propuesta vienen dados por los propios de un entorno emulado, ya señalados en la Sección 3.1: una red emulada con un único controlador y una topología fija. El Capítulo 4 detalla cómo se traducen estas decisiones en el diseño del sistema.

Capítulo 4

Diseño

Este capítulo presenta el diseño del sistema y justifica las decisiones que lo definen. Parte de las carencias identificadas en la Sección 3.5 y del alcance fijado en el Capítulo 1, y se organiza siguiendo el recorrido de los datos: primero la arquitectura general y el entorno experimental, después la generación del conjunto de datos, las características, los modelos y su evaluación, y por último la detección y la mitigación en tiempo de ejecución. La implementación del sistema se explica en el Capítulo 5, y las medidas obtenidas, en el Capítulo 6.

4.1. Arquitectura general

El sistema se organiza en las tres fases de la Figura 1.1. Cada fase produce lo que consume la siguiente: la generación de datos entrega un conjunto de datos etiquetado, el aprendizaje automático lo convierte en un modelo y la defensa en tiempo de ejecución despliega ese modelo en el controlador. De este encadenamiento se deriva el principio que más condiciona el diseño: *lo que el modelo ve al clasificar debe calcularse igual que lo que vio al entrenar*. Si el controlador obtuviera en tiempo de ejecución los contadores, las tasas o las características temporales con un procedimiento distinto del usado para construir el conjunto de datos, el modelo recibiría entradas de otra distribución y su rendimiento medido sin conexión dejaría de ser representativo. Por eso las fases 1 y 3 comparten la topología, el tráfico, la conmutación y el sondeo del controlador, el cálculo de los contadores y las tasas y el criterio de etiquetado, y las fases 2 y 3, el cálculo de las características de ventana temporal y el preprocesado.

La Figura 4.1 muestra los componentes y sus relaciones. La red emulada con Mininet [16] se conecta mediante OpenFlow 1.3 [9] a un controlador Ryu [17], único y remoto. El controlador tiene tres funciones: conmutar el tráfico con reglas por flujo, consultar periódicamente las estadísticas de los *switches* y, según la fase, registrarlas en el conjunto de datos o clasificarlas y actuar.

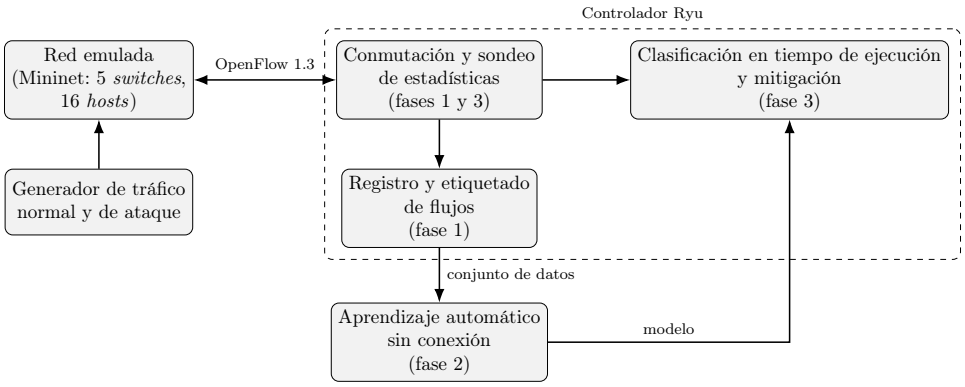


Figura 4.1: Arquitectura del sistema. El controlador comparte entre las fases 1 y 3 la conmutación y el sondeo; la fase 2 se ejecuta sin conexión.

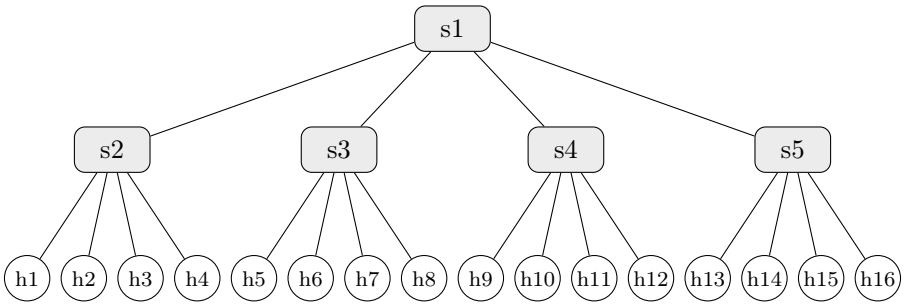


Figura 4.2: Topología en árbol de la red emulada: un *switch* raíz, cuatro *switches* de acceso y 16 *hosts*. Todos los *switches* se conectan al controlador Ryu, que no se dibuja.

El generador de tráfico se ejecuta sobre los *hosts* de la red y es el mismo en las fases 1 y 3. El aprendizaje automático de la fase 2 trabaja sin conexión sobre el conjunto de datos y devuelve el modelo, junto con los parámetros de preprocesado, que carga el controlador de la fase 3.

4.2. Entorno experimental

4.2.1. Topología

La red tiene forma de árbol de profundidad 2 y ramificación 4: un *switch* raíz, cuatro *switches* de acceso y cuatro *hosts* por *switch* de acceso, es decir, 5 *switches* y 16 *hosts*, con enlaces de 10 Mbps (Figura 4.2). La topología en árbol es una de las habituales en Mininet y permite que el tráfico entre *hosts* de distintas ramas atraviese varios *switches*, de modo que cada flujo se observa en más de un punto. La topología es fija y única, lo que acota el alcance de las conclusiones, como se señaló en la Sección 3.6.

4.2.2. Controlador y observación de los flujos

El controlador instala reglas reactivas y granulares: ante un paquete desconocido, crea una regla específica para ese flujo, definida por los campos de la capa 2, la capa 3 y la capa 4, o por los campos ARP, en lugar de reglas agregadas por destino. De este modo cada conversación genera entradas de flujo propias y separables, que son la unidad que se observa y se clasifica.

La observación se basa en las estadísticas que el propio *switch* mantiene para cada regla. El controlador las solicita cada segundo y, a partir de los contadores acumulados, calcula tasas de paquetes y de bytes por diferencia entre sondeos consecutivos. Los tiempos de vida de las reglas se fijaron en función de lo que se necesita observar. Una regla expira tras 5s sin tráfico y, en cualquier caso, a los 10s. Los flujos más cortos, como una sonda de un escaneo o un paquete falsificado, son precisamente la firma de algunos ataques, y un sondeo por segundo da más oportunidades de capturarlos antes de que expiren. El límite máximo de 10s evita que los flujos largos acumulen contadores de minutos, que no serían comparables con los del resto.

4.2.3. Identidad de los *hosts*

La detección de *spoofing* exige una referencia contra la que contrastar la asociación entre direcciones IP y MAC. Se toma de la propia emulación: la topología escribe la pareja legítima de cada *host*, y el controlador la carga en cuanto está disponible, tras la comprobación inicial de conectividad y antes de la primera fase de tráfico. Aprender esa asociación del tráfico fallaría si el primer paquete de un *host* ya fuera un ARP falsificado. En una red real esta referencia procedería de otra fuente, como un servidor DHCP o un inventario de equipos, lo que es una limitación del entorno emulado (Capítulo 7).

4.3. Generación del conjunto de datos

El conjunto de datos se genera sobre la red anterior, alternando fases de tráfico de un único tipo (normal, *scanning*, *spoofing* o DDoS), de 10 a 25s de duración, cuyo tipo se sortea. Al terminar cada fase, el generador la cierra y el controlador vacía las tablas de flujo, de modo que los flujos residuales de una fase, ya sin tráfico, no se registran con su etiqueta. El tráfico de transición entre fases, incluido el que prepara la siguiente, se etiqueta como tráfico fuera de fase y se descarta, igual que el de arranque. Cada fase recibe un identificador, que no es una característica del modelo, sino el criterio con el que se agrupan los datos en la validación (Sección 4.5). La generación se detiene al alcanzar del orden de 300.000 filas, lo que supone en torno a 900 fases. Se fija además un tope aproximado de 500 filas por fase, para que ninguna domine el conjunto; como se comprueba mientras la fase está en curso, en torno a una de cada cinco fases lo supera. Al tratarse

Clase	Variantes
Normal	Sesiones de `ping` de 2 a 6 paquetes, transferencias TCP con iPerf, y transferencias UDP con iPerf a 200 Kbps, 500 Kbps, 1 Mbps o 5 Mbps, entre *hosts* elegidos al azar.
Scanning	Un atacante contra *hosts* de la red, con Nmap (escaneo SYN, de conexión TCP o UDP sobre los ocho puertos más habituales, o rangos de puertos, con tres perfiles de velocidad) y con sondas TCP que solo llevan el indicador ACK, enviadas con `hping3`.
Spoofing	*Spoofing* ARP, con un atacante que envenena, en los dos sentidos, la caché de entre dos y cuatro víctimas y la de otro *host* al que suplanta, de modo que el tráfico entre ellos se dirija a su MAC (con Scapy), o *spoofing* de IP, con paquetes TCP SYN con la IP de origen falsificada (con `hping3`).
DDoS	Entre dos y cuatro atacantes contra una víctima, con paquetes SYN, UDP o ICMP (con `hping3`), a unos 500 paquetes por segundo por atacante, a unos 1000, o sin límite de tasa, y con puerto destino variable.

Tabla 4.1: Variantes de tráfico que genera cada clase.

de un proceso estocástico, se generan dos conjuntos de datos independientes con la misma configuración, para estimar cuánto varía el resultado entre generaciones (Capítulo 6).

4.3.1. Variantes de tráfico

Para que el conjunto varíe también dentro de cada clase, y no solo entre ellas, cada tipo de tráfico incluye variantes que se sortean en cada fase. La Tabla 4.1 las resume. El tráfico normal se genera con `ping` e iPerf [31]; los ataques, con Nmap [30], `hping3` [29] y Scapy [28].

El tráfico normal incluye a propósito transferencias de tasa alta. Sin ellas, el modelo podría aprender el atajo «tasa alta, luego ataque», que no se sostendría ante una transferencia legítima grande. Con las variantes de la tabla, la tasa por sí sola no basta para separar las clases.

4.3.2. Etiquetado

La etiqueta se asigna por flujo, no por franja temporal. Cada fase de ataque declara sus actores (atacante, víctimas e identidad suplantada) y solo se etiquetan como ataque los flujos que involucran a alguno de ellos, en cualquiera de los dos sentidos. El resto del tráfico presente durante la fase, como los protocolos de resolución de direcciones entre otros *hosts*, se etiqueta como normal. En el *scanning* el único actor es el atacante, porque un escaneo apunta a casi todos los *hosts* y, de incluir a los objetivos, se etiquetaría como ataque todo el tráfico hacia cualquiera de ellos.

Este criterio tiene una consecuencia que conviene tener presente: como

Grupo	Características	N.º
Protocolo y puertos	Tipo de trama, protocolo IP, puertos TCP y UDP de origen y destino, indicadores TCP del primer paquete, código de operación ARP	8
Identidad	Coincidencia entre la IP declarada y la MAC conocida del *host* (`ip_mac_consistent`); respuesta ARP que nadie había pedido (`arp_unsolicited_reply`)	2
Contadores y tasas	Duración (segundos y nanosegundos), paquetes, bytes, paquetes por segundo, bytes por segundo y tamaño medio de paquete	7
Estado del *switch*	Número de flujos activos en el *switch* (`flow_count_per_dpid`)	1
Ventana temporal	Puertos distintos por origen, destinos distintos por origen, flujos hacia el mismo destino y puerto, y orígenes distintos hacia el mismo destino y puerto	4
Total		**22**

Tabla 4.2: Características de cada flujo, agrupadas por la información que aportan.

la víctima es un actor, un flujo entre la víctima y un *host* ajeno al ataque también se etiqueta como ataque. Es una simplificación del criterio de etiquetado, no de la medición, y se retoma entre las limitaciones (Capítulo 7). Las fases de ataque se generan sin tráfico legítimo concurrente.

4.4. Características y preprocesado

4.4.1. Características de cada flujo

Cada flujo se describe con 22 características, que el controlador puede calcular a partir de lo que observa del *switch* y del primer paquete de cada flujo. La Tabla 4.2 las agrupa por la información que aportan.

Las dos características de identidad se orientan al *spoofing*. La primera detecta que el origen declarado no se corresponde con la MAC real del *host*, y la segunda, una respuesta ARP sin petición previa en los 5 segundos anteriores, que es la firma del envenenamiento de la caché ARP. Un ARP no solicitado también existe de forma legítima, por lo que es un indicio y no una prueba: el modelo lo combina con el resto de características.

Las características de ventana temporal describen patrones que un flujo aislado no revela: un origen que prueba muchos puertos o muchos destinos apunta a un *scanning*, y muchos orígenes hacia un mismo destino y puerto, a un DDoS distribuido. Se calculan con una ventana deslizante de 5s que mira solo hacia atrás, para no filtrar información futura, y que se reinicia en cada fase, de modo que nunca mezcla eventos de dos fases distintas. Esa

duración es del orden del tiempo de inactividad de las reglas, de modo que abarca la vida típica de un flujo, y queda por debajo de la duración mínima de una fase. El origen y el destino se identifican por la IP en el tráfico IP y por la MAC y la IP objetivo en el ARP, y el destino se cuenta junto con el puerto para no confundir con un DDoS el tráfico hacia otro servicio de una misma IP. El cálculo lo realiza un único módulo, usado tanto sin conexión, sobre el conjunto de datos completo, como en tiempo de ejecución, evento a evento, para que ambos modos obtengan los mismos valores. En tiempo de ejecución, la ventana se reinicia cada vez que el controlador vacía las tablas por un cambio de fase.

4.4.2. Preprocesado

El preprocesado deja el conjunto limpio y codificado, pero sin escalar ni seleccionar características, pasos que se hacen dentro de cada partición de la validación (Sección 4.5) para no filtrar información entre fases. Las decisiones principales son las siguientes.

- **Se excluyen los identificadores del laboratorio.** Las direcciones IP y MAC, el instante, el identificador del *switch* y el identificador de fase permitirían al modelo memorizar qué *host* o qué momento participó en cada ataque, en lugar de aprender patrones de tráfico. Con solo 16 *hosts* el riesgo es real. Tampoco se usan como característica los tiempos de vida de las reglas, que valen lo mismo en todas las filas porque proceden de la configuración y no del tráfico.

- **Se tratan las categóricas como tales.** El tipo de trama, el protocolo, el código ARP y los indicadores TCP son códigos, no cantidades: el protocolo 17 (UDP) no es «mayor» que el 6 (TCP). Se codifican como categorías para que el modelo no los interprete como magnitudes.

- **Se distingue el dato ausente del dato que no aplica.** Un flujo UDP no tiene puertos TCP, ni un flujo IP campos ARP. Esos huecos son estructurales, no datos perdidos, y se rellenan con cero. En cambio, una tasa indeterminada (infinita o no numérica) es una indeterminación aritmética y se elimina la fila.

- **Se descartan las filas fuera de fase y los duplicados.** El tráfico de comprobación de conectividad que precede a la primera fase y el de transición entre fases no pertenecen a ninguna clase.

4.4.3. Selección de características

De las 22 características se conservan las 15 más importantes, para eliminar las poco informativas sin perder rendimiento. La importancia la calcula

un modelo auxiliar de tipo Random Forest, un método habitual en la literatura [14], independiente de los tres modelos que se comparan y común a todos ellos. En la validación, la selección se hace dentro de cada partición con solo sus datos de entrenamiento, y en el modelo desplegado, una vez con todo el conjunto de datos. Al basarse en árboles, este criterio podría favorecer a los modelos de ese tipo frente a la regresión logística, algo que no se ha medido.

El número de 15 es un umbral de diseño: con la selección sobre todo el conjunto de datos, las 7 descartadas suman un 5,8 % de la importancia total (Sección 6.3.3). No se ha optimizado: no se ha comparado el rendimiento con otros valores. Reducir las características por su importancia es una práctica habitual; ALesawi y Alawadi, por ejemplo, pasan de 22 columnas a 10 combinando varios rankings [19].

4.5. Modelos y evaluación sin conexión

4.5.1. Modelos

Se comparan tres clasificadores de scikit-learn [22]: regresión logística, árbol de decisión y Random Forest, con 200 árboles. Cubren tres niveles de complejidad: un modelo lineal, un modelo no lineal interpretable y un conjunto de árboles. Los tres se entrenan con ponderación de clases balanceada, que penaliza más los errores en las clases minoritarias sin duplicar ni eliminar filas, y con una semilla fija para que la fase 2 sea reproducible. El modelo desplegado debe además clasificar cada flujo con un coste que no retrase al controlador, por lo que el coste computacional de entrenamiento y de inferencia se mide junto con el rendimiento.

4.5.2. Protocolo de validación

Las filas de una misma fase están muy correlacionadas: comparten atacante y víctima y se generan con segundos de diferencia. Si se repartieran al azar entre entrenamiento y prueba, quedarían filas casi gemelas a ambos lados y el modelo «reconocería» en la prueba fragmentos que ya vio, lo que inflaría las métricas sin medir generalización. Esto es lo que ocurre con las particiones aleatorias que la literatura emplea con frecuencia (Sección 3.5). La validación se hace por ello con `GroupKFold` de cinco particiones, agrupando por fase, de modo que todas las filas de una fase van juntas al entrenamiento o a la prueba. El escalado y la selección de características se ajustan solo con la parte de entrenamiento de cada partición, para que la fuga de información no reaparezca por esa vía. Se emplean varias particiones y no una sola porque un único reparto agrupado es muy inestable según qué fases caigan en la prueba.

4.5.3. Métricas y selección del mejor modelo

Las métricas se calculan sobre las predicciones de cada fila, obtenidas por un modelo que no vio su fase. Las predicciones de las cinco particiones se reúnen y las métricas se calculan una sola vez sobre el conjunto reunido, promediadas en forma macro para que las cuatro clases pesen lo mismo. La dispersión que se informa es la desviación típica de los F1 macro de cada una de las cinco particiones. Se informa de precisión, exhaustividad y F1 por clase, de la matriz de confusión de cada modelo y del coste computacional. El mejor modelo se elige por F1 macro, y se prefiere sobre la exactitud porque este depende de la clase mayoritaria. El modelo final se entrena con todo el conjunto de datos, ya que en ese paso no se mide el rendimiento y cuantos más datos tenga, mejor.

4.6. Detección y mitigación en tiempo de ejecución

4.6.1. Clasificación en tiempo de ejecución

En la fase 3, el controlador conmuta y sondea exactamente igual que en la fase 1, con las mismas reglas, tiempos de vida y cálculo de contadores. Cualquier diferencia cambiaría las características respecto al entrenamiento, y en especial el número de flujos activos en el *switch*, que depende de cuánto viven las reglas. En cada respuesta de estadísticas de un *switch*, el controlador calcula las características de todos sus flujos y los clasifica en una sola llamada al modelo, con el mismo preprocesado que al entrenar. La clasificación en lote evita que el coste fijo de cada llamada, multiplicado por el número de flujos de cada respuesta, deje atrás al controlador. El etiquetado de la verdad de referencia utiliza el mismo criterio que el conjunto de datos, para que las métricas en tiempo de ejecución se midan igual que las obtenidas sin conexión.

4.6.2. Mitigación

Cuando el modelo clasifica un flujo como ataque (*scanning*, *spoofing* o DDoS), el controlador puede bloquear la conversación correspondiente instalando una regla OpenFlow de descarte. El diseño de esa regla responde a cuatro decisiones.

- **La unidad de bloqueo es la conversación, el par ordenado de direcciones MAC origen y destino.** El modelo afirma que un flujo pertenece a un ataque, no que un *host* sea el atacante, y el conjunto de datos etiqueta como ataque también las respuestas de la víctima. Bloquear la IP de origen cortaría a las víctimas, y en el *spoofing* dejaría sin servicio al *host* suplantado, porque la IP de origen es falsa. La MAC de origen es la del remitente real, y un falso positivo corta una

conversación, no un *host* entero. La regla solo corta ese sentido de la comunicación: el sentido contrario es otra conversación, que se bloquea si a su vez se confirma como ataque.

- **El bloqueo se instala en todos los *switches*.** Hacerlo solo en el que informó del flujo no cortaría el tráfico de la conversación que no pase por él.
- **La regla es temporal, de 20s.** La red no queda bloqueada por reglas residuales, un falso positivo se corrige por sí solo y un ataque que continúa se vuelve a detectar y a bloquear. Es un valor de diseño, no optimizado, que busca un compromiso: es muy superior al tiempo de confirmación (unos 2s) y al tiempo máximo de las reglas de reenvío (10s), por lo que el bloqueo es efectivo, pero lo bastante corto para que un bloqueo erróneo no corte una conversación legítima durante demasiado tiempo. Tiene una prioridad muy superior a las reglas de reenvío, para prevalecer sobre ellas, y en OpenFlow 1.3 se expresa como una regla sin acciones [9].
- **El bloqueo requiere tres confirmaciones.** Una conversación se bloquea cuando sale clasificada como ataque en tres sondeos distintos dentro de una ventana de 5s. Un ataque real es sostenido y aparece sondeo tras sondeo, mientras que un error aislado del modelo no suele repetirse, de modo que el umbral filtra los falsos positivos sin retrasar apenas la respuesta: con un sondeo por segundo, tres confirmaciones se acumulan en unos 2s. Es un compromiso: subir el umbral reduciría los bloqueos erróneos, a costa de retrasar la respuesta ante un ataque real.

Estos cuatro parámetros (unidad de bloqueo, alcance, duración y umbral de confirmación) son los que definen la política de mitigación. Los resultados de detección en tiempo de ejecución dependen de ella, porque cuanto antes se bloquee una conversación, menos flujos de ese ataque llega a observar el controlador. Esta relación se discute en el Capítulo 6.

4.7. Diseño de la evaluación en tiempo de ejecución

La evaluación de la fase 3 reproduce, sobre la red emulada y con el modelo desplegado, una batería formada por los cuatro tipos de tráfico, de 30s cada uno, con el mismo generador que el conjunto de datos. Las cuatro pruebas duran lo mismo para que sus resultados sean comparables. Cada ejecución sortea atacantes, víctimas y variantes, por lo que el resultado de una ejecución aislada varía mucho. Por eso se repite la batería diez veces, y además de las métricas con todas las ejecuciones se informa de su media y su desviación entre ejecuciones. En la batería, el DDoS se lanza sin la

Decisión de diseño	Carencia a la que responde
Tres clases de ataque y tráfico normal, en una clasificación multiclase	Cobertura de ataques
Conjunto de datos propio, etiquetado por flujo, con variantes dentro de cada clase	Realismo de los datos
Validación cruzada agrupada por fase, con escalado y selección dentro de cada partición	Protocolo de evaluación
F1 macro, métricas por clase y falsos bloqueos medidos por conversación	Métricas
Mismo cálculo de características y de etiquetas en entrenamiento y en tiempo de ejecución	Detección en tiempo de ejecución
Bloqueo temporal, por conversación y tras tres confirmaciones	Detección en tiempo de ejecución y mitigación
CPU del controlador, latencia del plano de control y tiempo de inferencia	Coste para la infraestructura

Tabla 4.3: Decisiones principales del diseño y carencia de los trabajos previos citados que atienden.

variante sin límite de tasa (*flood*): esa variante satura la máquina que aloja la emulación mientras el modelo clasifica en tiempo de ejecución, de modo que lo que se saturaría sería el entorno y no el sistema de defensa. Se mantienen las dos intensidades limitadas, de unos 500 y 1000 paquetes por segundo.

La detección y la mitigación se miden por separado. La detección se mide por flujo, con el mismo criterio de etiqueta y las mismas métricas que la fase 2, para poder contrastar ambos resultados. La mitigación se mide por conversación, que es la unidad en la que decide el controlador: cuántas conversaciones de ataque se bloquearon, cuántas conversaciones legítimas se bloquearon por error y cuánto tarda en llegar el primer bloqueo. Se separan porque el propio bloqueo altera el tráfico que el controlador observa a continuación, de modo que medir ambas cosas sobre la misma unidad mezclaría el rendimiento del modelo con el efecto de la mitigación.

Además, se mide el impacto en la infraestructura: el uso de CPU del proceso del controlador, muestreado con psutil [36], la latencia del plano de control desde que llegan las estadísticas hasta que se envía la orden de bloqueo, y el tiempo de inferencia del modelo por flujo.

4.8. Resumen de las decisiones de diseño

La Tabla 4.3 resume las decisiones principales del diseño y las carencias del estado del arte (Sección 3.5) a las que responden.

Capítulo 5

Implementación

Este capítulo explica cómo se ha construido el sistema descrito en el Capítulo 4. Sigue el mismo orden: primero la organización del código y el entorno de ejecución, y después cada una de las tres fases. Se describen los mecanismos que no son evidentes a partir del diseño, como el cálculo de los contadores, la coordinación entre procesos o la integración del modelo en el controlador. Los fragmentos de código más relevantes se reúnen en los anexos A y B, y las referencias a ellos se indican donde procede. El código completo está disponible en un repositorio público de GitHub [39]: `https://github.com/clara99gf/sdn-cybersecurity-ml`.

5.1. Organización del código, configuración y ejecución

El sistema está escrito en Python 3.10 [25], el lenguaje en el que está escrito Ryu [17] y para el que existen Mininet [16], Scapy [28] y las bibliotecas de aprendizaje automático empleadas (scikit-learn [22], pandas [32] y NumPy [33]). El código se organiza por fases y por función, como resume la Tabla 5.1. Las fases comparten el módulo de configuración y, según el caso, el generador de tráfico, el módulo de ventana temporal y el criterio de etiquetado, de modo que el principio de diseño de que lo que se calcula al entrenar se calcule igual en tiempo de ejecución se cumple en esos puntos sin depender de mantener dos versiones, porque es el mismo código y no una copia que pudiera acabar siendo distinta. El controlador de defensa es una aplicación de Ryu propia, porque la del monitor, al arrancar, prepara el fichero del conjunto de datos y se dedica a escribir filas etiquetadas. Por eso reproduce la lógica de conmutación y sondeo del monitor, con los mismos parámetros de `config.py`, y del monitor solo importa la función de etiquetado, que no depende de su estado.

Todos los parámetros se definen en `config.py`, y los módulos de las tres fases importan de ahí sus valores, de forma que cada uno está definido en un

Fichero o directorio	Contenido
`config.py`	Parámetros y rutas de las tres fases, con una sección por fase.
`setup.sh`, `setup.py`, `requirements.txt`	Instalación del entorno y de las dependencias.
`run_01_dataset.py`	Orquestador de la fase 1.
`mininet_lab/`	Topología (`topology.py`), generador de tráfico normal y de ataque (`traffic_generator.py`) y *spoofing* ARP (`arp_spoof.py`).
`controller/sdn_monitor.py`	Controlador de la fase 1: conmutación, sondeo, cálculo de características y registro etiquetado.
`feature_windows.py`	Ventana temporal compartida por las fases 2 y 3.
`ml/`	Preprocesado (`preprocessing.py`), entrenamiento (`train.py`), evaluación (`evaluate.py`), guardado y carga de datos y modelos (`utils.py`) y orquestador de la fase 2 (`run_02_ml.py`).
`controller/sdn_defense.py`, `controller/live_classifier.py`	Controlador de la fase 3 y clasificador en tiempo de ejecución.
`defense/`	Tráfico de la fase 3 (`traffic.py`) y generación de tablas y figuras (`plots.py`).
`run_03_defense.py`	Orquestador de la fase 3.

Tabla 5.1: Organización del código fuente.

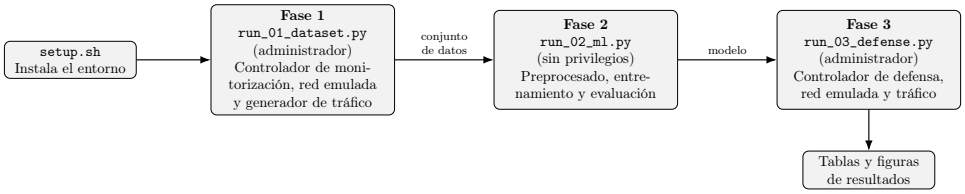


Figura 5.1: Ejecución del sistema.

único lugar. La Tabla 5.2 recoge los principales.

La Figura 5.1 resume cómo se ejecuta el sistema. Tras instalar el entorno una sola vez con `setup.sh` (Sección 5.2), cada fase se lanza con un único *script* y consume lo que produjo la anterior, por lo que deben ejecutarse en orden: la fase 2 necesita el conjunto de datos de la fase 1, y la fase 3, el modelo de la fase 2. Las fases 1 y 3 levantan la red emulada y el controlador, y requieren privilegios de administrador; la fase 2 trabaja solo sobre ficheros y no los necesita. Las fases 1 y 3 se ejecutan indicando explícitamente el intérprete del entorno virtual, porque `sudo` ignora un entorno virtual activado, y al terminar devuelven al usuario la propiedad de los ficheros que crean.

Parámetro	Valor	Significado (Unidad)
`TOPO_DEPTH`, `TOPO_FANOUT`	2, 4	Profundidad y ramificación del árbol (5 *switches*, 16 *hosts*)
`LINK_BANDWIDTH_MBPS`	10	Ancho de banda de cada enlace (Mbps)
`POLL_INTERVAL`	1	Intervalo entre sondeos de estadísticas (s)
`FLOW_IDLE_TIMEOUT`	5	Inactividad tras la cual expira una regla (s)
`FLOW_HARD_TIMEOUT`	10	Permanencia máxima de una regla (s)
`TARGET_ROWS`	300.000	Filas objetivo del conjunto de datos
`MIN_PHASE_DURATION`, `MAX_PHASE_DURATION`	10, 25	Rango de duración de una fase de tráfico (s)
`MAX_ROWS_PER_PHASE`	500	Tope aproximado de filas por fase
`PHASE_SETTLE_SECONDS`	1,5	Pausa entre el cierre de una fase y el inicio de la siguiente (s)
`WINDOW_SECONDS`	5	Ventana temporal de las características de patrón entre flujos (s)
`N_FEATURES`	15	Características que se conservan tras la selección
`RANDOM_STATE`	42	Semilla de la fase 2
`DEFENSE_PHASE_DURATION`	30	Duración de cada prueba de la fase 3 (s)
`DEFENSE_DROP_TIMEOUT`	20	Duración de cada regla DROP (s)
`DEFENSE_MITIGATION_CONFIRMATIONS`	3	Sondeos en que debe salir como ataque una conversación
`DEFENSE_CONFIRM_WINDOW_S`	5	Ventana de esas confirmaciones (s)
`DEFENSE_BATTERY_RUNS`	10	Repeticiones de la batería de la fase 3
`DEFENSE_DDOS_ALLOW_FLOOD`	`False`	Incluye el DDoS sin límite de tasa en la batería de la fase 3

Tabla 5.2: Parámetros principales de `config.py`.

5.2. Entorno de ejecución

5.2.1. Plataforma

El entorno de referencia es Ubuntu 22.04 LTS [24], con las versiones de las herramientas que recoge la Tabla 2.2. La versión de Python no se puede

elegir libremente: Ryu [17] solo funciona con las versiones de la 3.8 a la 3.10, porque en las posteriores es incompatible con Eventlet [26], la biblioteca de concurrencia en la que se apoya. Por ello, el sistema fija Python 3.10 e instala los paquetes en un entorno virtual propio.

5.2.2. Instalación automatizada

El *script* `setup.sh` instala todo el entorno con un solo comando a partir de una máquina Ubuntu limpia, sin pasos previos ni posteriores. Sus pasos son los siguientes.

1. Instala con el gestor de paquetes del sistema las herramientas que no son bibliotecas de Python: Mininet [16], Open vSwitch [27], Nmap [30], `hping3` [29], iPerf [31] y `ping`.

2. Busca un intérprete de Python compatible con Ryu en el directorio de sistema, y si no lo hay, instala Python 3.10 desde un repositorio de paquetes adicional. Se busca solo en el directorio de sistema para que el resultado no dependa de otros gestores de entornos que cada usuario pudiera tener instalados.

3. Crea un entorno virtual aislado, sin acceso a los paquetes del intérprete base, para que el entorno no dependa de lo que haya instalado en cada máquina. Si ya existe uno que no cumple esas condiciones, lo recrea.

4. Da acceso a Mininet dentro del entorno virtual. Mininet se instala con el gestor del sistema y queda fuera del entorno virtual, por lo que se enlaza únicamente ese paquete, en lugar de abrir todo el entorno a los paquetes del sistema.

5. Instala Ryu con un procedimiento especial. Su versión 4.34 no se construye con las versiones actuales de las herramientas de empaquetado, por lo que se instala una anterior y se desactiva el aislamiento de la construcción.

6. Instala el resto de las dependencias de Python y el propio proyecto en modo editable. Por defecto se instalan las versiones exactas de `requirements.txt`, con las que se obtuvieron los resultados de esta memoria. Mediante una variable de entorno, se puede optar por los rangos de versiones definidos en `setup.py`, lo que permite comprobar la compatibilidad con versiones más recientes de las dependencias.

7. Aplica un parche mínimo a Ryu. Ryu importa de Eventlet un símbolo que las versiones compatibles ya no exportan, y sin corregirlo no arranca. El parche envuelve ese *import* en un bloque de captura de errores, y se aplica de forma idempotente, de modo que puede repetirse.

8. Termina con una batería de comprobaciones: que los módulos se importan, que el parche de Ryu está aplicado, que `ryu-manager` arranca y que las dependencias se resuelven dentro del entorno virtual. Si alguna falla, indica cómo reproducir el error.

5.3. Fase 1: generación del conjunto de datos

5.3.1. Red emulada

La topología se construye con la clase de árbol que incluye Mininet, con los parámetros de la Tabla 5.2, enlaces con ancho de banda limitado y un controlador remoto. Se asignan direcciones MAC correlativas y predecibles, lo que permite relacionar cada *host* con su identidad en el conjunto de datos. Tras arrancar la red se desactivan en todas las interfaces la segmentación y el ensamblado de paquetes que realiza la tarjeta de red virtual, y se limita el tamaño máximo de agregación (GSO) al de una trama Ethernet estándar. Sin ello, las interfaces virtuales agrupan paquetes en supertramas de hasta 64 KB y el controlador vería tamaños y tasas que no corresponden al tráfico real de la red. Después se espera a que los *switches* se conecten al controlador y se comprueba la conectividad con un `pingAll`. Por último, se escribe en un fichero la pareja IP y MAC de cada *host*, tomada de la propia API de Mininet, que el controlador carga para conocer la asociación legítima desde el primer paquete de las fases de tráfico (Sección 4.2).

5.3.2. Controlador de monitorización

El controlador de la fase 1 es una aplicación de Ryu para OpenFlow 1.3 [9]. Realiza dos tareas que se describen a continuación.

Conmutación. Al conectarse un *switch*, instala una regla de prioridad mínima que envía al controlador todo paquete desconocido. Cuando llega uno, el controlador aprende el puerto de la MAC de origen, decide el puerto de salida y calcula las señales que solo se pueden obtener del primer paquete: los indicadores TCP, la coincidencia entre la IP declarada y la MAC conocida y, en el ARP, si la respuesta fue solicitada. Si ya conoce el puerto de destino, instala una regla específica para ese flujo, con los tiempos de vida de la Tabla 5.2; si no lo conoce, como ocurre con los paquetes de difusión, lo reenvía por inundación sin instalar regla, de modo que ese paquete no llega a observarse como flujo. En ambos casos, reenvía el paquete. Las respuestas ARP se clasifican como solicitadas si se observó una petición para esa dirección en los 5 segundos anteriores.

Sondeo y registro. Un hilo cooperativo de Ryu pide cada segundo las estadísticas de flujo a todos los *switches*. Por cada flujo de la respuesta (se

descarta la regla de prioridad mínima) calcula las características derivadas y escribe una fila etiquetada en el fichero del conjunto de datos. El cálculo de los contadores tiene tres detalles de implementación:

- **Compensación del primer paquete.** El paquete que provoca el evento de paquete desconocido no lo cuentan las estadísticas del *switch*, porque se reenvía antes de que exista la regla. El controlador lo suma al contador, y mantiene esa compensación mientras la regla viva. Si solo se aplicara en el primer sondeo, el contador acumulado disminuiría en el segundo, lo que es imposible en OpenFlow.
- **Detección de reinstalaciones.** Si la duración de un flujo disminuye entre dos sondeos, el *switch* lo ha eliminado y reinstalado con los contadores a cero. En ese caso se descartan la compensación acumulada y la lectura anterior.
- **Tasas.** Las tasas de paquetes y de bytes se calculan por diferencia entre dos sondeos consecutivos del mismo flujo. Para un flujo recién instalado, que no tiene lectura anterior, se estiman dividiendo el contador entre la edad del flujo.

5.3.3. Coordinación entre procesos

El generador de tráfico y el controlador son procesos independientes, y el controlador no necesita conocer al generador. Se coordinan mediante pequeños ficheros en un directorio común, que resume la Tabla 5.3. El principal es el fichero de etiqueta: antes de cada fase, el generador escribe en él la etiqueta, el identificador de fase y los actores del ataque, y el controlador lo lee en cada sondeo para etiquetar cada fila con el criterio de la Sección 4.3. La función que aplica el criterio por flujo se reproduce en el listado A.1 del apartado de Anexos. El controlador vigila además ese fichero: cuando cambia la fase (comparando etiqueta e identificador, para detectar dos fases consecutivas del mismo tipo) vacía las tablas de flujo y el estado interno por flujo, de modo que una fase no contamine la siguiente. Como esa comprobación se hace cada 0,3s, entre el cambio de etiqueta y el vaciado puede llegar alguna respuesta de estadísticas que describe flujos de la fase anterior; el controlador la descarta, para no registrarlos con la etiqueta de la nueva. Conserva, en cambio, la asociación entre IP y MAC, que es identidad del *host* y no estado transitorio.

5.3.4. Generador de tráfico

El generador del módulo `traffic_generator.py` ejecuta, en los *hosts* de Mininet, las herramientas de la Tabla 4.1: `ping` e iPerf para el tráfico normal, Nmap y `hping3` para los escaneos y el DDoS, `hping3` para el *spoofing* de IP

Fichero	Fases	Función
Etiqueta de la fase `current_label.txt`	1 y 3	El generador escribe etiqueta, identificador de fase y actores; el controlador lo lee para etiquetar.
Petición de vaciado `flush_request.txt`	1	El generador pide vaciar las tablas cuando corta una fase por superar el tope de filas.
Identidades de *hosts* `host_identities.csv`	1 y 3	Pareja IP y MAC legítima de cada *host*, escrita al levantar la red y cargada por el controlador.
Indicador de prueba activa `defense_active.flag`	3	El orquestador lo crea al empezar cada prueba, con el tipo y el número de repetición, y lo borra al terminarla. Fuera de una prueba, el controlador solo conmuta.
Indicador de controlador listo `defense_ready.flag`	3	El controlador lo crea tras vaciar las tablas y descartar las estadísticas en tránsito; el orquestador no lanza el tráfico hasta entonces.
Indicador de limpieza `defense_clean.flag`	3	El controlador lo crea al terminar de limpiar las reglas al final de una prueba.

Tabla 5.3: Ficheros mediante los que se coordinan los procesos, todos en el directorio `runtime/`.

y un guion propio con Scapy [28] para el *spoofing* ARP. Cada tipo de fase es una función que sortea sus variantes y devuelve los actores. Algunos detalles de implementación condicionan el resultado:

- **Procesos en segundo plano.** Las herramientas se lanzan en segundo plano en la shell del *host* y se detienen por su identificador de proceso exacto. Al empezar cada fase se eliminan todos los procesos de tráfico que pudieran seguir vivos.
- **Puerto de origen fijo.** `hping3` se lanza con un puerto de origen fijo, para que cada ataque genere un flujo sostenido y no un flujo por paquete. Las sondas ACK del escaneo, enviadas a un ritmo de unos cinco paquetes en un segundo, se dimensionan para que el flujo sobreviva al menos a un sondeo y llegue al conjunto de datos.
- ***Spoofing* ARP.** El guion envía cada segundo respuestas ARP que nadie ha solicitado, en los dos sentidos: indican a la víctima que la IP suplantada está en la MAC del atacante, y al *host* suplantado, lo mismo de la víctima. Es la firma de la que se alimenta la característica de respuesta no solicitada.
- **Calentamiento ARP.** Antes de etiquetar una fase de escaneo o de DDoS, el generador resuelve por ARP las direcciones de los objetivos.

Ese tráfico legítimo se genera todavía bajo la etiqueta de transición con la que se cerró la fase anterior, por lo que en la fase 1 se descarta y no cuenta como ataque; al escribirse después la etiqueta de la nueva fase, el controlador vacía las tablas y esos flujos desaparecen. En la fase 3, el orquestador fija una etiqueta normal antes de cada prueba, de modo que ese tráfico se evalúa como normal. La diferencia responde al propósito de cada fase: la fase 1 construye los datos de entrenamiento y descarta lo que no pertenece claramente a una clase, mientras que la fase 3 evalúa el sistema sobre todo el tráfico que observa, y ese tráfico es legítimo.

- **Cierre de cada fase.** Al terminar una fase, el generador detiene sus herramientas y escribe la etiqueta de transición, que el preprocesado descarta. El controlador vacía entonces las tablas, de modo que los flujos residuales de la fase, ya sin tráfico, no se registran con su etiqueta, y el generador espera 1,5s antes de empezar la siguiente. En la fase 3 ocurre lo mismo al terminar cada prueba, pero el controlador de defensa no evalúa ese tráfico: no registra sus predicciones ni lo bloquea, por el mismo motivo por el que la fase 1 lo descarta.

- **Tope de filas.** Durante las esperas, el generador comprueba cada pocas décimas de segundo si la fase ha superado el tope de filas, y si es así pide el vaciado de las tablas y la trunca. Dejar de lanzar tráfico no basta, porque los flujos ya instalados seguirían vivos hasta su tiempo máximo y añadirían filas. En la fase 3 esta comprobación se desactiva, porque no hay un fichero creciente que contar.

5.3.5. Orquestación

El *script* `run_01_dataset.py` arranca el controlador en segundo plano, espera a que escuche en el puerto de OpenFlow, levanta la topología y lanza la generación, que se detiene al alcanzar el número objetivo de filas. Al terminar, o si se interrumpe, detiene el controlador, limpia el estado residual de Mininet y devuelve al usuario la propiedad de los ficheros creados. Si el controlador no llega a escuchar, muestra las últimas líneas de su registro. La generación completa tarda unas seis horas en el equipo empleado.

5.4. Fase 2: aprendizaje automático

La fase 2 se ejecuta con un único *script* que encadena el preprocesado, el entrenamiento y la evaluación, y se detiene mostrando la traza completa si alguno de los pasos falla. Utiliza scikit-learn [22], pandas [32] y NumPy [33], y guarda los objetos entrenados con Joblib [34]. Tarda unos tres minutos.

Preprocesado. Sigue los pasos de la Sección 4.4. Primero calcula las cuatro características de ventana temporal con el módulo compartido, que mantiene para cada clave una cola de eventos y el recuento de valores distintos (listado A.2 del apartado de Anexos). Todas las claves incluyen el identificador de fase, de modo que la ventana se reinicia en cada fase. Las filas sin puerto de destino (ARP o ICMP) no alimentan el recuento de puertos distintos, sino que toman el último valor de su origen dentro de la fase. Después reconstruye la fase de cada fila a partir de su identificador, descarta las filas fuera de fase y los duplicados, elimina identificadores y columnas constantes, trata los nulos y codifica las categóricas. Guarda la matriz de características, las etiquetas y los grupos, junto con los codificadores ajustados, que la fase 3 reutiliza para codificar igual en tiempo de ejecución. Además, genera la tabla descriptiva del conjunto de datos.

Entrenamiento. Escala las características con la media y la desviación típica de todo el conjunto, selecciona las 15 más importantes con un Random Forest auxiliar y entrena los tres modelos con ponderación de clases balanceada. Guarda en disco cada modelo, el escalador, la lista de características seleccionadas y los tiempos de entrenamiento, además de la tabla y la figura de importancias. Este modelo final se entrena con todos los datos y no sirve para medir rendimiento.

Evaluación. La evaluación aplica el protocolo de la Sección 4.5. Para cada una de las cinco particiones agrupadas por fase, ajusta el escalador y la selección solo con el conjunto de entrenamiento, entrena el modelo y predice la partición de prueba (listado A.3 del apartado de Anexos). Con las predicciones de todas las particiones, calcula las métricas globales y, por clase, las matrices de confusión y el coste computacional, y escribe las tablas en formato CSV y las figuras. Elige el mejor modelo por F1 macro y lo guarda con un nombre fijo, que es el que carga el controlador de la fase 3.

5.5. Fase 3: detección y mitigación en tiempo de ejecución

5.5.1. Controlador de defensa

El controlador de defensa conmuta y sondea como el de la fase 1, con las mismas reglas, tiempos de vida y cálculo de contadores, y también vacía las tablas en cada cambio de etiqueta y descarta las respuestas que llegan mientras ese vaciado está pendiente. El criterio de etiquetado de la verdad de referencia no se copia: se importa del monitor, para que signifique lo mismo que al entrenar. La clave de flujo, en cambio, forma parte de la lógica de conmutación y sondeo, que se reproduce de forma idéntica (Sección 5.1).

Hay dos diferencias con el monitor. Una es que, además de la regla de prioridad mínima, se excluyen del procesamiento las reglas DROP propias, que no existen en el conjunto de datos. La otra es que el controlador solo clasifica y mitiga mientras hay una prueba en marcha. Esto lo indica el fichero `defense_active.flag` (Tabla 5.3), y fuera de una prueba el controlador se limita a conmutar. Al comenzar una prueba, el controlador vacía las tablas de flujo, descarta durante unos segundos las respuestas de estadísticas en tránsito, que describen flujos anteriores a la prueba, y avisa al orquestador de que está listo. Al terminar, vuelca los eventos pendientes, limpia las reglas, incluidas las DROP, y lo comunica.

5.5.2. Clasificador en tiempo de ejecución

El clasificador carga el modelo y los objetos de preprocesado que generó la fase 2 y reproduce, flujo a flujo, el mismo preprocesado que se aplicó al entrenar: las mismas columnas y en el mismo orden, la misma codificación de las categóricas, el mismo escalado y la misma selección de características. Tres decisiones de implementación evitan que ese cálculo en tiempo de ejecución difiera del de entrenamiento o bloquee al controlador:

- **Codificación por valor numérico.** Pandas leyó como decimales las categóricas con huecos del fichero del conjunto de datos, y los codificadores aprendieron sus clases como texto («6.0»), mientras que en tiempo de ejecución los valores llegan como enteros («6»). El clasificador compara por valor numérico, y asigna a los huecos y a las categorías nunca vistas el código del valor cero, el de «no aplica», igual que el relleno con cero del preprocesado.
- **Ventana temporal.** Las cuatro características de ventana se calculan con el mismo módulo que en el preprocesado, alimentado con todos los flujos de cada respuesta de estadísticas y en orden, como requiere la ventana. Las ventanas se vacían cada vez que el controlador vacía las tablas, es decir, al empezar cada prueba y en cada cambio de etiqueta, igual que el preprocesado las reinicia en cada fase.
- **Clasificación en lote y sin paralelismo.** Todos los flujos de una respuesta de estadísticas se clasifican en una sola llamada al modelo (listado B.1 del apartado de Anexos), y el tiempo de inferencia por flujo es el del lote dividido entre su tamaño. El Random Forest se ejecuta en un único proceso, porque dentro de Ryu, que usa Eventlet con sus hilos parcheados, la predicción en paralelo con Joblib llega a bloquearse.

5.5.3. Mitigación

El controlador cuenta, para cada conversación (par ordenado de MAC de origen y destino), en cuántas rondas de sondeo distintas ha salido clasificada

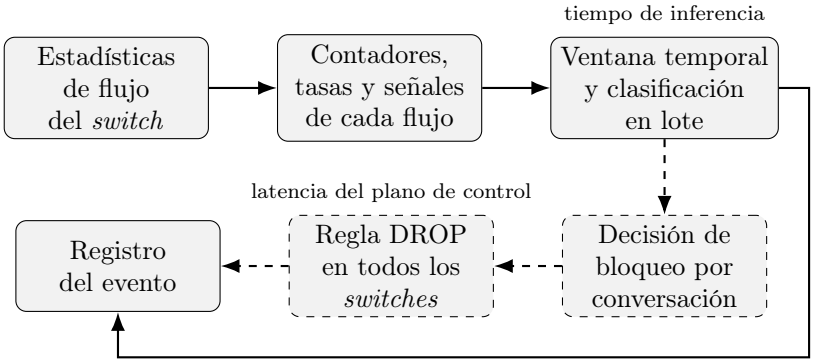


Figura 5.2: Procesamiento de cada respuesta de estadísticas en el controlador de defensa y puntos de medida. En trazo discontinuo, los pasos condicionales: la decisión de bloqueo solo se evalúa para los flujos clasificados como ataque, y la regla DROP solo se instala cuando se alcanza el umbral de confirmación. El evento se registra para todos los flujos clasificados.

como ataque dentro de la ventana de confirmación. Cada ronda se identifica por el segundo en que llega la respuesta de estadísticas. Las respuestas de los distintos *switches* a un mismo sondeo suelen llegar en el mismo segundo, pero si quedan a ambos lados de un cambio de segundo cuentan como dos rondas, de modo que el bloqueo puede producirse antes de completarse tres sondeos. Cuando se alcanza el umbral, instala en todos los *switches* una regla con prioridad muy superior a la de reenvío, que casa con ese par de MAC y no tiene acciones, lo que en OpenFlow 1.3 equivale a descartar. La regla caduca por tiempo máximo a los 20s y lleva una marca propia para poder distinguirla. Mientras la regla está vigente, la conversación no vuelve a contarse, y al instalarla se reinicia su cuenta de confirmaciones. La función que lo implementa se reproduce en el listado B.2 del apartado de Anexos.

5.5.4. Registro de eventos y medidas

Por cada flujo clasificado, el controlador registra un evento con la predicción, la etiqueta real, el tipo de prueba y la repetición, el tiempo de inferencia, la latencia del plano de control, si se aplicó un bloqueo, algunas características de interés y el uso de CPU del proceso. La Figura 5.2 resume lo que ocurre en cada sondeo y de dónde sale cada medida.

La latencia del plano de control se mide con un reloj monotónico desde que se empieza a procesar la respuesta de estadísticas hasta que se envía el bloqueo, y solo existe para los flujos que provocan un bloqueo. La CPU del proceso se muestrea cada segundo con psutil [36], de forma no bloqueante y en un hilo cooperativo aparte, y si esa biblioteca no está disponible se calcula a partir de las estadísticas del propio proceso del sistema. Los eventos se acumulan en memoria y se vuelcan al fichero por lotes de 50, para no bloquear

con escrituras en disco el hilo de Eventlet que procesa las respuestas.

5.5.5. Orquestación de las pruebas

El *script* `run_03_defense.py` levanta el controlador de defensa y la red, con la misma topología y los mismos ajustes de las interfaces que la fase 1, y ofrece un menú. Permite lanzar una prueba de un solo tipo de tráfico, con su resumen y su figura, o la batería completa, que encadena los cuatro tipos de tráfico, la repite el número de veces configurado y al terminar genera todas las tablas y figuras. El tráfico lo genera el mismo módulo que en la fase 1, con las dos diferencias de la Sección 4.7: sin tope de filas y sin la intensidad sin límite de tasa del DDoS. Entre una prueba y la siguiente se desactiva la detección, se detienen los procesos de tráfico y se espera a que el controlador confirme que ha limpiado las reglas, de modo que ninguna prueba empiece con bloqueos de la anterior.

El módulo `plots.py`, que se ejecuta de forma independiente a partir del fichero de eventos, calcula las tablas de detección por flujo, de mitigación por conversación, de la exhaustividad antes y después del primer bloqueo, de las métricas por ejecución y del impacto en la infraestructura, y dibuja las figuras con Matplotlib [35]. Las figuras temporales muestran una sola ejecución, la representativa, definida como aquella cuyo F1 macro está más cerca de la media, porque promediar curvas de ejecuciones con instantes de bloqueo distintos no tendría sentido. Las tablas y la matriz de confusión usan todas las ejecuciones.

Capítulo 6

Experimentación

Este capítulo presenta los resultados de las tres fases del sistema descrito en los capítulos 4 y 5 y los discute. Los resultados se exponen siguiendo el orden de las fases: el conjunto de datos (Sección 6.2), la evaluación de los modelos sin conexión (Sección 6.3) y la detección y la mitigación en tiempo de ejecución (Sección 6.4). La Sección 6.5 los interpreta, los relaciona entre sí y con el estado del arte, y señala qué condiciones los acotan. Las tablas y figuras de menos significativas se reúnen en el anexo C.

6.1. Condiciones experimentales

Todos los experimentos se ejecutaron en el equipo de la Tabla 2.1, con las versiones de software de la Tabla 2.2. La red emulada con Mininet [16] y el controlador Ryu [17] son los descritos en la Sección 4.2: 5 *switches*, 16 *hosts* y enlaces de 10 Mbps.

Los modelos se implementan con scikit-learn [22] y se evalúan con la validación cruzada agrupada por fase de la Sección 4.5. Para la fase 3 se repite diez veces la batería de la Sección 4.7, de modo que cada repetición ejecuta durante 30s cada uno de los cuatro tipos de tráfico. El DDoS de esa batería se lanza sin la variante sin límite de tasa, que sí está en el conjunto de datos, y las fases de ataque no llevan tráfico legítimo concurrente.

Conviene precisar tres convenciones. Primera, se generaron dos conjuntos de datos independientes con la misma configuración. Los resultados de este capítulo corresponden al primero, que es el que entrena el modelo desplegado en la fase 3. El segundo conjunto solo aparece donde se nombra expresamente, sobre todo en la Sección 6.3.4, que compara ambos para estimar la variación entre generaciones y justifica esa elección. Segunda, las cifras de dispersión, tanto entre particiones de la validación cruzada como entre las diez repeticiones de la batería, son desviaciones típicas poblacionales. Tercera, las métricas macro promedian las cuatro clases con el mismo peso, y las de la validación cruzada se calculan sobre las predicciones de las

			Fases		Protocolo (% de flujos)			
Clase	**Flujos**	**%**	Lanzadas	Con flujos	ARP	ICMP	TCP	UDP
Normal	98.474	38,7	236	233	49,5	21,6	18,4	10,5
Scanning	69.545	27,4	234	211	90,9	2,7	5,7	0,6
DDoS	50.563	19,9	221	212	36,3	29,7	23,0	11,1
Spoofing	35.701	14,0	220	210	73,9	4,7	19,5	1,8
Total	254.283	100,0	911	866	61,6	15,7	16,0	6,7

Tabla 6.1: Composición del conjunto de datos: flujos por clase, fases de cada tipo que se lanzaron y fases en las que se registró algún flujo de su clase, y reparto de los flujos por protocolo. Las fases lanzadas de cada tipo se obtienen del registro de la generación de tráfico.

cinco particiones reunidas, de modo que el F1 de un modelo no es la media de los F1 de sus particiones; la desviación sí es la de esos cinco F1.

6.2. Fase 1: conjunto de datos

6.2.1. Composición

La generación ejecutó 911 fases de tráfico y escribió 300.027 filas. De ellas, 45.744 corresponden a tráfico de transición entre fases o de arranque y se descartan en el preprocesado, de modo que el conjunto de datos contiene 254.283 flujos etiquetados, repartidos en 904 fases; las siete fases restantes no registraron ningún flujo (Sección 6.2.2). La Tabla 6.1 recoge su composición. El tráfico normal es la clase mayoritaria, con el 38,7 % de los flujos, y las tres clases de ataque suman entre el 14,0 % y el 27,4 % cada una. El desequilibrio es, por tanto, moderado. Ninguna clase se reequilibró en los datos: en la fase 2 se compensa con la ponderación de clases (Sección 4.5).

Como el tipo de cada fase se sortea, se lanzaron cantidades similares de cada uno. La diferencia entre las dos columnas de fases son las fases en las que no se registró ningún flujo de su clase: tres fases normales, que quedaron vacías, y 42 de ataque, que se analizan en la Sección 6.2.2. Los flujos normales proceden sobre todo de las fases normales, pero también de las de ataque, en las que se etiquetan como normales los flujos que no involucran a ningún actor (Sección 4.3). En cuanto al protocolo, el ARP es el más frecuente en conjunto (61,6 %) y domina en el *scanning* (90,9 %) y en el *spoofing* (73,9 %), mientras que el ICMP y el UDP tienen más peso en el DDoS y en el tráfico normal. La generación de cada conjunto tardó unas seis horas (5,8 h la primera y 6,4 h la segunda).

6.2.2. Saturación del entorno de emulación

De las 675 fases de ataque, 42 (el 6,2 %) no contienen ningún flujo de su clase: 23 de *scanning*, 10 de *spoofing* y 9 de DDoS. Cuatro de ellas, junto con tres fases de tráfico normal, son las siete fases que no registraron ningún flujo. Los registros de la generación muestran que en esas fases las herramientas de ataque sí se lanzaron, pero la red no respondía. Por ejemplo, Nmap informa de que el objetivo no está accesible, a pesar de que se lanza sin descubrimiento previo, lo que indica que ni siquiera se resolvió su dirección por ARP. Al mismo tiempo, el controlador encuentra vacías las tablas de flujo de todos los *switches*, y el conjunto de datos tiene tramos de hasta 35s sin ninguna fila. Estos episodios aparecen sobre todo aislados, aunque una tercera parte se concentra en rachas de dos o tres fases de ataque consecutivas, y son más frecuentes tras un DDoS sin límite de tasa: afectan al 10,0 % de las fases que siguen a uno en las tres fases anteriores, frente al 3,4 % del resto. La propia variante sin límite de tasa deja menos flujos de DDoS por fase (una mediana de 78, frente a unos 330 en las variantes limitadas), lo que indica que el controlador no llega a registrar todos los flujos mientras se ejecuta.

Todo ello apunta a que, en momentos puntuales, el controlador y la red emulada, que comparten una única máquina, se saturan. Lo que se pierde son muestras de ataque: esas fases solo aportan flujos etiquetados como normales, o ninguno. Es una limitación del entorno, que se recoge en el Capítulo 7.

6.3. Fase 2: evaluación de los modelos sin conexión

6.3.1. Comparación de los modelos

La Tabla 6.2 compara los tres modelos con la validación cruzada agrupada por fase. Random Forest obtiene el mejor F1 macro, 0,826, seguido del árbol de decisión (0,758) y de la regresión logística (0,586). Las diferencias entre modelos (0,068 entre el primero y el segundo, y 0,172 entre el segundo y el tercero) son varias veces mayores que la dispersión entre particiones, que es de entre 0,012 y 0,017. Random Forest es también el mejor en precisión, exhaustividad y exactitud.

El coste computacional de Random Forest es el mayor, pero sigue siendo pequeño: 5,5s de entrenamiento con todo el conjunto de datos y 0,0026ms de inferencia por flujo en lote. La regresión logística y el árbol de decisión infieren en torno a 0,0001ms. Estos tiempos de inferencia se miden sobre lotes de decenas de miles de flujos, con la predicción repartida entre todos los núcleos del equipo, y no son comparables con los de la fase 3, donde cada lote tiene pocos flujos y la predicción usa un único proceso (Sección 6.4.4).

Modelo	Exactitud	Prec.	Exh.	F1	Desv.	Entren. (s)	Inf. (ms)
Regresión logística	0,586	0,584	0,633	0,586	0,012	2,2	0,0001
Árbol de decisión	0,763	0,758	0,759	0,758	0,017	1,2	0,0001
Random Forest	**0,831**	**0,848**	**0,810**	**0,826**	**0,016**	**5,5**	**0,0026**

Tabla 6.2: Resultados de los tres modelos con validación cruzada agrupada por fase (cinco particiones). Precisión (Prec.), exhaustividad (Exh.) y F1 son macro. La desviación (Desv.) es la desviación típica de los F1 macro de las cinco particiones, mientras que el F1 se calcula sobre las predicciones reunidas. El tiempo de entrenamiento es el del modelo con todo el conjunto de datos, y el de inferencia (Inf.), por flujo y en lotes grandes.

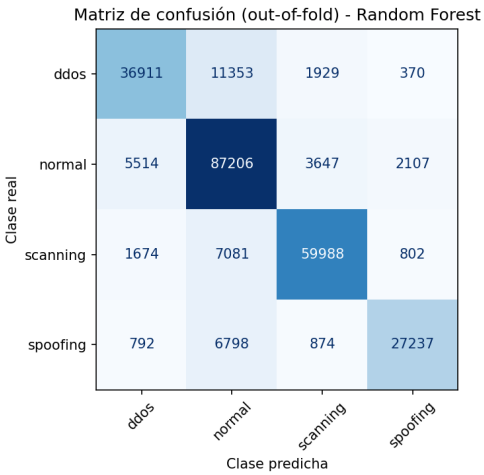


Figura 6.1: Matriz de confusión de Random Forest con validación cruzada agrupada por fase. Cada flujo se predice con un modelo que no vio su fase. Las clases aparecen en orden alfabético.

6.3.2. Resultados por clase y errores

La Figura 6.1 detalla el comportamiento de Random Forest, y sus métricas por clase se recogen en la Tabla 6.4, junto a las obtenidas en tiempo de ejecución (Sección 6.4). El *scanning* es la clase mejor resuelta (F1 de 0,882), y el DDoS la peor (0,773), con una exhaustividad de 0,730. La matriz de confusión muestra que el error dominante es clasificar un ataque como tráfico normal: se confunden con él el 22,5 % de los flujos de DDoS (11.353 de 50.563), el 19,0 % de los de *spoofing* (6.798 de 35.701) y el 10,2 % de los de *scanning* (7.081 de 69.545). En sentido contrario, el 11,4 % de los flujos normales se clasifica como algún ataque (11.268 de 98.474). Los errores entre clases de ataque son mucho menos frecuentes. Las matrices de confusión de los otros dos modelos se recogen en la Figura C.1 del anexo C.

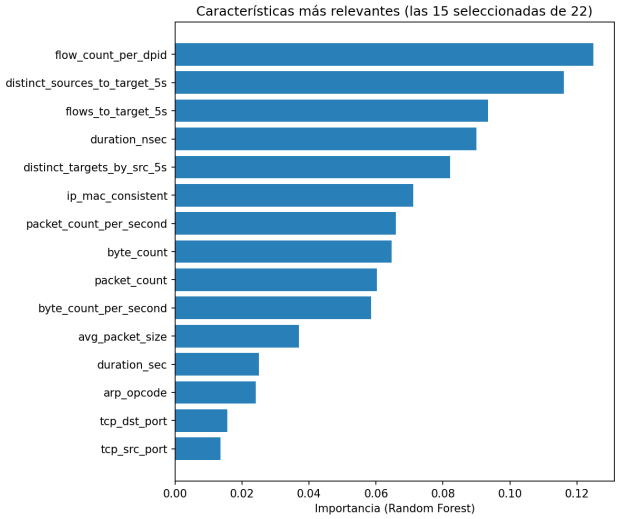


Figura 6.2: Importancia de las 15 características seleccionadas, según un Random Forest entrenado con todo el conjunto de datos.

6.3.3. Importancia y selección de características

La Figura 6.2 muestra la importancia de las 15 características seleccionadas, medida con el Random Forest entrenado con todo el conjunto de datos. Las cuatro primeras son el número de flujos activos en el *switch* (0,125), los orígenes distintos hacia el mismo destino y puerto en la ventana (0,116), los flujos hacia ese destino y puerto (0,094) y la duración en nanosegundos (0,090). Les sigue, con 0,082, el número de destinos distintos por origen. Tres de las cinco primeras son características de ventana temporal. La importancia se mide como la reducción media de impureza que aporta cada característica en los árboles, una medida que tiende a favorecer a las variables continuas con muchos valores distintos, como la duración en nanosegundos, por lo que el orden debe leerse con cautela.

Las siete características descartadas suman el 5,8 % de la importancia total, de modo que las 15 seleccionadas reúnen más del 94 %. Las descartadas son el protocolo IP y los indicadores TCP (0,012 cada una), la respuesta ARP no solicitada (0,010, de identidad), el tipo de trama (0,010), los puertos distintos por origen (0,006, de ventana) y los puertos UDP de destino y de origen (0,004 y 0,003). El número de 15 es un umbral de diseño, elegido para conservar casi toda la importancia con menos variables (Sección 4.4.3), y no un valor optimizado: no se ha evaluado el rendimiento con otros números de características.

Indicador	1.ª generación	2.ª generación
Fases ejecutadas / con flujos	911 / 904	999 / 990
Flujos etiquetados	254.283	257.098
Fases de ataque sin flujos de su clase	42 de 675 (6,2 %)	55 de 718 (7,7 %)
Fases sin flujos de su clase tras un DDoS sin límite	10,0 %	13,5 %
Mediana de la tasa de los flujos de DDoS (paq./s)	13,5	2,0
Mediana de flujos de DDoS por fase de tasa limitada	320–334	168–195
F1 macro de Random Forest	0,826 ± 0,016	0,779 ± 0,015

Tabla 6.3: Indicadores de calidad de las dos generaciones del conjunto de datos y F1 macro de Random Forest con cada una. La fila de fases tras un DDoS sin límite considera las tres fases siguientes a uno; en las demás fases la proporción es del 3,4 % en ambas generaciones. Los dos valores de la última fila de indicadores corresponden a las dos intensidades limitadas.

6.3.4. Variación entre generaciones del conjunto de datos

La segunda generación, con la misma configuración, ejecutó 999 fases y contiene 257.098 flujos en 990 fases. Con ella, Random Forest obtiene un F1 macro de 0,779 ± 0,015, frente a 0,826 ± 0,016 con la primera, y los otros dos modelos también bajan (a 0,700 el árbol de decisión y a 0,558 la regresión logística). La diferencia entre generaciones, de 0,047, es unas tres veces la dispersión entre particiones de cada una, de modo que no se explica por la inestabilidad del modelo dentro de una misma generación. Se concentra en el DDoS, cuya exhaustividad baja de 0,730 a 0,584, y en el *spoofing* (de 0,763 a 0,681).

La Tabla 6.3 compara varios indicadores de la calidad de los datos, que se calculan sin el modelo. La segunda generación tiene más fases de ataque sin flujos de su clase, y los flujos de DDoS que registra son mucho menos intensos: la mediana de su tasa es de 2,0 paquetes por segundo, frente a 13,5 en la primera, y las fases de DDoS de tasa limitada dejan aproximadamente la mitad de flujos de ataque. Es decir, la segunda generación muestra más signos de la saturación descrita en la Sección 6.2.2. Ambas se generaron en el mismo equipo, sin otro uso durante la generación, y no se ha identificado la causa de la diferencia. Unos flujos de DDoS registrados a menor tasa se parecen más al tráfico normal, lo que es coherente con que la exhaustividad del DDoS sea la que más baja.

Por estos indicadores, el modelo desplegado en la fase 3 se entrena con la primera generación, que representa mejor los ataques que pretende generar el diseño. La decisión se tomó conociendo los resultados de las dos, por lo que la cifra de 0,826 debe leerse junto con la de 0,779: el rendimiento

	Sin conexión (fase 2)			**En tiempo de ejecución (fase 3)**		
Clase	Prec.	Exh.	F1	Prec.	Exh.	F1
Normal	0,776	0,886	0,827	0,670	0,915	0,774
Scanning	0,903	0,863	0,882	0,967	0,974	0,970
DDoS	0,822	0,730	0,773	0,876	0,478	0,618
Spoofing	0,893	0,763	0,823	0,871	0,743	0,802
Macro	0,848	0,810	0,826	0,846	0,778	0,791

Tabla 6.4: Métricas por clase de Random Forest sin conexión (validación cruzada agrupada por fase, 254.283 flujos) y en tiempo de ejecución (diez ejecuciones de la batería, 14.788 flujos). En tiempo de ejecución, la fila macro agrupa los flujos de todas las ejecuciones, del mismo modo que sin conexión agrupa las predicciones de las cinco particiones; la media por ejecución se recoge en la Tabla C.1.

sin conexión depende de forma apreciable de las condiciones del entorno durante la generación. Con solo dos generaciones no se puede estimar con rigor la dispersión entre ellas.

6.4. Fase 3: detección y mitigación en tiempo de ejecución

Las diez repeticiones de la batería producen 14.788 flujos clasificados en tiempo de ejecución. Como se decidió en el diseño, la detección se mide por flujo y la mitigación, por conversación.

6.4.1. Detección en tiempo de ejecución

La Tabla 6.4 recoge las métricas por clase de la detección en tiempo de ejecución junto a las obtenidas sin conexión. Para calcularlas igual que las de la fase 2, las de la fase 3 se obtienen sobre todos los flujos reunidos, en este caso los 14.788 de las diez ejecuciones, y con ellos el F1 macro es de 0,791, frente a 0,826 sin conexión. La Figura C.2 del anexo C representa las columnas de exhaustividad y F1 de la tabla. En tiempo de ejecución, el *scanning* vuelve a ser la clase mejor resuelta (F1 de 0,970) y el DDoS, la peor (0,618), con una exhaustividad de 0,478.

La Figura 6.3 muestra la matriz de confusión, también con todos los flujos. De nuevo, el error principal es clasificar como normal un flujo de ataque, que afecta al 49,1 % de los flujos de DDoS (1.460 de 2.973), al 20,6 % de los de *spoofing* (278 de 1.347) y al 1,7 % de los de *scanning* (109 de 6.374). Casi todos los errores del DDoS se producen después del primer bloqueo de la prueba, como se analiza en la Sección 6.4.3.

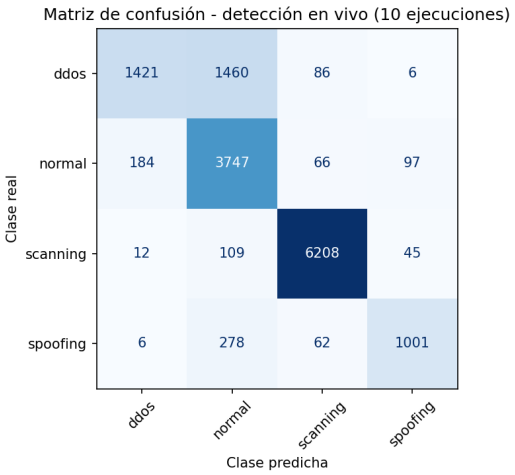


Figura 6.3: Matriz de confusión de la detección en tiempo de ejecución, con las diez ejecuciones de la batería. Las clases aparecen en orden alfabético.

Si el F1 macro se calcula por separado en cada ejecución, su media es de 0,756 ± 0,059, y su valor va de 0,667 a 0,828 (Tabla C.1 del anexo C). La dispersión es grande porque cada ejecución sortea atacantes, víctimas y variantes de ataque, de modo que una ejecución aislada no es representativa. Por eso las figuras temporales de este capítulo y del anexo muestran una sola ejecución, la representativa, que es la novena, con un F1 macro de 0,761, el más cercano a esa media.

6.4.2. Mitigación

La Tabla 6.5 recoge los resultados de la mitigación por conversación, es decir, por par ordenado de direcciones MAC de origen y destino. De las 395 conversaciones de ataque, el sistema bloqueó 390, lo que da una exhaustividad de 0,987. Bloqueó también 28 conversaciones legítimas, todas en las pruebas de tráfico normal, de modo que la precisión del bloqueo es de 0,933 y su F1, de 0,959.

El primer bloqueo llega, de media, entre 2,0s y 2,2s después del primer flujo del ataque, según el tipo, y se produce en las diez ejecuciones de cada tipo de ataque. En el DDoS y el *spoofing* llega a los 2,0s en todas ellas, y en el *scanning*, entre 2,0s y 3,0s. Las cinco conversaciones de ataque que no se bloquearon son una de *scanning* y cuatro de *spoofing*. El número de bloqueos (509) supera al de conversaciones bloqueadas (390 de ataque y 28 legítimas) porque una conversación puede bloquearse más de una vez: la regla dura 20s, y si el ataque continúa cuando caduca, se detecta y se bloquea de nuevo. La Figura 6.4 lo ilustra con una ejecución de DDoS: la tasa de paquetes cae a cero tras el bloqueo, no aparecen flujos nuevos mientras la regla está vigente,

Prueba	Conv.	Ataque	Bloq.	No bloq.	Falsos	Bloqueos	1.º (s)
Normal	123	0	0	0	28	28	—
Scanning	299	281	280	1	0	289	2,21
DDoS	79	62	62	0	0	124	2,00
Spoofing	67	52	48	4	0	68	2,00
Total	568	395	390	5	28	509	—

Tabla 6.5: Mitigación por conversación en las diez ejecuciones de la batería. Conv.: conversaciones observadas. Ataque: conversaciones de ataque. Bloq.: de ataque bloqueadas. Falsos: conversaciones legítimas bloqueadas. Bloqueos: decisiones de bloqueo, cada una de las cuales instala la regla de descarte en los cinco *switches*. 1.º: tiempo medio desde el primer flujo del ataque hasta el primer bloqueo.

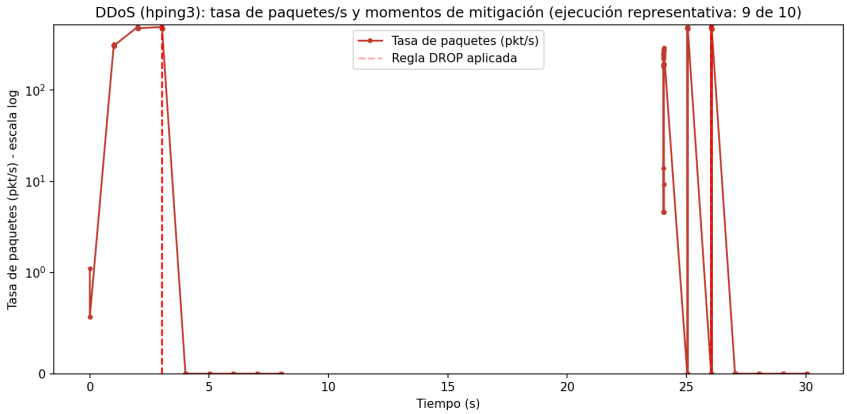


Figura 6.4: Tasa de paquetes de una prueba de DDoS y reglas de descarte instaladas (líneas discontinuas), en la ejecución representativa, la que tiene el F1 macro más cercano a la media. La escala vertical es logarítmica.

el ataque reaparece al caducar y vuelve a cortarse. Las figuras C.3, C.4 y C.5 del anexo C muestran las otras tres pruebas de la misma ejecución.

6.4.3. Efecto del bloqueo sobre la detección por flujo

Como la mitigación corta el ataque, cambia lo que el controlador observa a continuación. La Tabla 6.6 separa la exhaustividad por flujo de cada clase en los flujos anteriores al primer bloqueo de la prueba y en los posteriores. El efecto no es uniforme. En el DDoS, la exhaustividad baja de 0,986 a 0,410, y en el *spoofing*, de 0,911 a 0,714. En el *scanning*, en cambio, sube de 0,792 a 0,981. Los flujos anteriores al primer bloqueo son pocos (entre 202 y 353 por clase) frente a los posteriores (entre 1.145 y 6.143).

El registro de eventos permite descomponer la caída del DDoS. De los

	Antes del primer bloqueo		Después	
Clase	Flujos	Exh.	Flujos	Exh.
Scanning	231	0,792	6.143	0,981
DDoS	353	0,986	2.620	0,410
Spoofing	202	0,911	1.145	0,714

Tabla 6.6: Exhaustividad (Exh.) por flujo de cada ataque antes y después del primer bloqueo de su prueba, en las diez ejecuciones.

2.620 flujos de DDoS posteriores al primer bloqueo, 1.814 tienen una tasa de paquetes nula: son reglas de reenvío anteriores al bloqueo que siguen en la tabla hasta expirar, pero que ya no llevan tráfico porque la regla de descarte, de mayor prioridad, lo intercepta. El modelo solo clasifica como DDoS el 24,3 % de ellos y el 72,7 % como normales (1.318). Los 806 flujos posteriores que sí llevan tráfico, de conversaciones aún no bloqueadas o de ataques que reaparecen al caducar una regla, se detectan en el 78,4 % de los casos. Si se consideran solo los flujos anteriores al primer bloqueo y los posteriores con tráfico, la exhaustividad del DDoS es de 0,846 (980 de 1.159).

6.4.4. Impacto en la infraestructura

La Tabla 6.7 recoge el coste del sistema en el controlador. La CPU del proceso de Ryu, muestreada con psutil [36], tiene una media de entre el 3,5 % y el 6,7 % según el tipo de tráfico. Los máximos son puntuales y mayores: del 12,0 % en el *scanning* y el *spoofing*, del 46,0 % en el tráfico normal y del 68,0 % en el DDoS. La latencia del plano de control, desde que llegan las estadísticas hasta que se envía la orden de bloqueo, tiene una media de entre 8,4 ms y 15,8ms, y su máximo es de 72,7ms, en el *scanning*. La inferencia cuesta entre 0,58ms y 2,98ms por flujo de media, con un máximo de 9,4ms. La Figura C.6 del anexo C muestra la evolución de la CPU y de la latencia en la ejecución representativa.

	CPU de Ryu (%)		**Latencia (ms)**		**Inferencia (ms)**	
Prueba	Media	Máx.	Media	Máx.	Media	Máx.
Normal	5,54	46,0	8,88	10,31	1,76	7,06
Scanning	6,60	12,0	15,76	72,73	0,58	9,44
DDoS	6,67	68,0	9,37	12,67	1,37	6,86
Spoofing	3,50	12,0	8,44	10,76	2,98	7,96

Tabla 6.7: Impacto en la infraestructura por tipo de tráfico. La CPU se expresa respecto de un núcleo, por lo que puede superar el 100 %, y su media se calcula sobre los eventos registrados. La latencia del plano de control solo existe para los flujos que provocan un bloqueo. La inferencia es por flujo, dentro de un lote.

6.5. Discusión

6.5.1. El modelo y sus errores

Random Forest supera con claridad a los otros dos modelos en todas las métricas, y la regresión logística es la que peor se comporta. Que el modelo lineal quede muy por debajo de los basados en árboles es compatible con que las clases no se separen con una frontera lineal sobre estas características, aunque esta explicación no se ha contrastado con un experimento específico.

La mayoría de los errores van en la misma dirección: los ataques que se pierden se clasifican como tráfico normal, y los de DDoS y *spoofing* son los más afectados, con exhaustividades de 0,730 y 0,763. Los datos disponibles no permiten determinar con exactitud la causa. Una posibilidad es que el criterio de etiquetado, que marca como ataque también los flujos de la víctima en ambos sentidos (Sección 4.3), incluya flujos cuyas características individuales no se distinguen de las del tráfico normal. Para comprobarlo habría que medir los errores separando los flujos que parten del atacante de los de la víctima, algo que no se ha hecho ya que excedería el trabajo planteado. Un dato que sí consta es que la característica de respuesta ARP no solicitada, pensada como firma del envenenamiento de la caché, no figura entre las 15 seleccionadas para el modelo desplegado, mientras que la coincidencia entre IP y MAC sí. La comparación entre generaciones (Sección 6.3.4) apunta, además, a que en la exhaustividad del DDoS influye cómo registra el entorno los flujos de ataque.

6.5.2. Resultados sin conexión y en tiempo de ejecución

En este caso, el valor F1 macro en tiempo de ejecución, de 0,791 agrupando los flujos y de 0,756 como media de las ejecuciones, queda por debajo del obtenido sin conexión, de 0,826 (Tabla 6.4). La diferencia no es uniforme: el *scanning* mejora en tiempo de ejecución (F1 de 0,970 frente a 0,882), el

spoofing baja ligeramente (0,802 frente a 0,823) y el DDoS baja con claridad (0,618 frente a 0,773). Los dos números no son directamente comparables, por cuatro razones. Las pruebas en tiempo de ejecución tienen otra composición de clases, pues el tráfico normal es el 27,7 % de los flujos frente al 38,7 % del conjunto de datos. El DDoS en tiempo de ejecución no incluye la variante sin límite de tasa. Las pruebas en tiempo de ejecución duran 30s y no tienen tope de filas, frente a fases de 10 a 25s en el conjunto de datos. Y la propia mitigación modifica qué flujos de ataque llega a observar el controlador. Esta última razón explica la mayor parte de la caída del DDoS: antes del primer bloqueo su exhaustividad es de 0,986, y la caída se concentra en flujos que el bloqueo ha dejado sin tráfico (Sección 6.4.3). A ello se suma la dispersión entre ejecuciones: dos de las diez (con 0,827 y 0,828) alcanzan el valor sin conexión, y las otras ocho quedan por debajo. Lo que sí permite afirmar la comparación es que el modelo, entrenado sobre el conjunto de datos, detecta en tiempo de ejecución los ataques mientras estos llevan tráfico; la detección por flujo, medida tras el bloqueo, refleja sobre todo el efecto de la mitigación.

6.5.3. Detección por flujo y mitigación por conversación

La detección y la mitigación dan cifras distintas porque miden cosas distintas. En el DDoS, la exhaustividad por flujo es de 0,478, pero las 62 conversaciones de DDoS se bloquearon. La explicación está en el diseño de la mitigación: para bloquear una conversación basta con que se clasifique como ataque en tres sondeos distintos dentro de 5s, no con que se acierte en todos sus flujos, y los flujos que el modelo deja de reconocer son en su mayoría posteriores al bloqueo. Un ataque sostenido genera muchos flujos y es suficiente con que el modelo acierte en una parte de ellos. Por el mismo motivo, el tiempo hasta el primer bloqueo (entre 2,0s y 2,2s de media) es coherente con tres confirmaciones a razón de un sondeo por segundo, y la latencia de procesamiento del controlador (milisegundos) es despreciable frente a esa espera. Reducir el tiempo de respuesta exigiría reducir el intervalo de sondeo o el umbral de confirmación, y ninguna de las dos variantes se ha evaluado.

El *spoofing* es la clase con más conversaciones de ataque sin bloquear (4 de 52). El registro de eventos permite identificarlas: las cuatro son conversaciones ARP de pruebas de *spoofing* de IP, entre *hosts* que son actores del ataque. El criterio de etiquetado las marca como ataque por involucrar a un actor, pero son resoluciones ARP ordinarias, y el modelo clasifica como normales 39 de sus 50 flujos. La de *scanning* es también una conversación ARP del atacante, con 14 flujos, 13 de ellos clasificados como normales. Ambos casos apoyan la hipótesis de la sección anterior: una parte de los flujos etiquetados como ataque no se distingue del tráfico normal.

El efecto del bloqueo sobre la exhaustividad por flujo (Tabla 6.6) muestra que la detección en tiempo de ejecución no es independiente de la política de

Magnitud	Media (rango)	Máximo
CPU de Ryu (%)	3,5–6,7	68,0 (DDoS)
Latencia del plano de control (ms)	8,4–15,8	72,7 (*scanning*)
Inferencia por flujo en tiempo de ejecución (ms)	0,58–2,98	9,44 (*scanning*)

Tabla 6.8: Resumen del coste para la infraestructura. El rango de la media recorre los cuatro tipos de tráfico, y el máximo es el mayor de los cuatro.

mitigación. En el DDoS, tras el bloqueo, el controlador sigue viendo sobre todo reglas de reenvío sin tráfico, que el modelo clasifica mayoritariamente como normales. Se puede discutir si esos flujos deben contar como ataque: el criterio de etiquetado los marca así porque involucran a los actores, pero ya no transportan tráfico de ataque, y que el modelo no los reconozca no impide la mitigación, porque la conversación ya está bloqueada. En el *scanning* el efecto es el contrario, y la exhaustividad sube tras el bloqueo, de modo que no hay una explicación única para las tres clases. Las muestras previas al bloqueo son, además, pequeñas.

6.5.4. El coste de los falsos bloqueos

El principal coste del sistema son los falsos bloqueos: 28 de las 123 conversaciones legítimas observadas en las pruebas de tráfico normal, es decir, el 22,8 %, o 2,8 de las 12,3 conversaciones de cada prueba. Cada una sufre un corte de 20s. Según el registro de eventos, 318 de los 3.863 flujos de las pruebas de tráfico normal (8,2 %) se clasifican como ataque, y 75 de las 123 conversaciones tienen al menos un flujo clasificado así. Solo 28 de esas 75 llegan a bloquearse, lo que indica que la confirmación en tres sondeos filtra una parte importante de los errores aislados del modelo. Con ese filtro, la proporción de conversaciones legítimas cortadas sigue siendo alta. El diseño apunta a que subir el umbral reduciría los falsos bloqueos a costa de retrasar la respuesta, pero ese compromiso no se ha medido, porque solo se evaluó un umbral. Tampoco se ha medido el efecto de un corte de 20s en una aplicación real, y la unidad de medida es la conversación, no el servicio.

6.5.5. Coste para la infraestructura

La Tabla 6.8 resume el coste del sistema en el controlador, cuyo detalle por tipo de tráfico recoge la Tabla 6.7.

La CPU media del controlador, del 3,5 % al 6,7 %, y la latencia media de entre 8,4ms y 15,8ms indican que, en este entorno, clasificar cada flujo en tiempo de ejecución y mitigar no supone una carga elevada para el controlador. Los máximos de CPU son puntuales; el mayor, del 68,0 %, se registra en el DDoS. La latencia es mayor en el *scanning* (15,8ms de media), que es

también la prueba con más flujos clasificados (6.559 eventos frente a entre 1.369 y 3.863 en las demás), lo que es compatible con que lotes mayores tarden más en procesarse. Estas cifras se midieron en una única máquina, con una topología pequeña, enlaces de 10 Mbps y sin la variante sin límite de tasa del DDoS. No permiten extrapolar el comportamiento a redes mayores. Tampoco es comparable el tiempo de inferencia en tiempo de ejecución (entre 0,58ms y 2,98ms por flujo) con el medido sin conexión (0,0026ms), porque la diferencia se debe al coste fijo de cada llamada al modelo, que en tiempo de ejecución se reparte entre pocos flujos, y a que sin conexión la predicción se reparte entre todos los núcleos del equipo, y no al modelo.

6.5.6. Comparativa con trabajos previos

Si bien los resultados de otros trabajos no son comparables con las de este (Sección 3.4), los resultados se relacionan aquí con las carencias identificadas en la Sección 3.5. Para tener una visión en cierta medida aproximada de los resultados generados en la experimentación del TFG: el sistema cubre a la vez *scanning*, *spoofing* y DDoS con una clasificación multiclase, y el F1 macro de 0,826 se obtiene con una validación que agrupa por fase, lo que reduce el riesgo de resultados inflados por fuga de información entre entrenamiento y prueba. Mide por separado la detección y la mitigación, y cuantifica las conversaciones legítimas bloqueadas por error, que según el análisis de la Sección 3.5 ninguno de los trabajos de la Tabla 3.3 cuantifica, además del tiempo hasta el bloqueo, la CPU, la latencia y la inferencia. Por otro lado, las métricas sin conexión quedan por debajo de las puntuaciones cercanas al 100 % que comunican algunos trabajos con particiones aleatorias, aunque no son comparables con ellas, y al medir los falsos bloqueos se hace visible un coste que esos trabajos no cuantifican.

6.5.7. Alcance de los resultados

Varias condiciones acotan lo que los resultados permiten concluir, y se retoman junto al resto de limitaciones del enfoque en el Capítulo 7. El rendimiento sin conexión difiere de forma apreciable entre las dos generaciones del conjunto de datos, por la saturación del entorno, y los resultados en tiempo de ejecución proceden de una única batería de diez ejecuciones, con una dispersión grande entre ejecuciones.

La detección en tiempo de ejecución depende de la política de mitigación con la que se midió, y la mitigación sesga la exhaustividad por flujo. Las fases de ataque se generan sin tráfico legítimo concurrente y sin la variante del DDoS sin límite de tasa. El criterio de etiquetado incluye a la víctima como actor. La asociación entre IP y MAC de referencia procede de Mininet. Y todo se ha medido en una red emulada, en un único equipo.

Capítulo 7

Conclusiones y trabajo futuro

Este capítulo cierra la memoria. Más concretamente, la Sección 7.1 resume lo que se ha hecho y lo que los resultados permiten concluir, y contrasta el trabajo con los objetivos fijados en el Capítulo 1. La Sección 7.2 reúne las limitaciones del enfoque, y la Sección 7.3, las líneas de trabajo futuro que se derivan de ellas.

7.1. Conclusiones

El trabajo ha construido y evaluado un sistema de extremo a extremo que combina SDN y aprendizaje automático para detectar y mitigar en tiempo de ejecución ataques de *scanning*, *spoofing* y DDoS. Sobre una red emulada con Mininet [16] y controlada con Ryu [17], el sistema genera un conjunto de datos propio de 254.283 flujos etiquetados, entrena y compara tres modelos con una validación que agrupa por fase, y despliega el mejor, Random Forest, en el controlador. Este clasifica cada flujo en tiempo de ejecución y bloquea las conversaciones de ataque con reglas OpenFlow [9] de descarte temporales, tras confirmarlas en tres sondeos.

Los resultados del Capítulo 6 permiten extraer las conclusiones siguientes:

- **La clasificación multiclase es viable, con un rendimiento moderado y dependiente del entorno.** Random Forest obtiene un F1 macro de 0,826 ± 0,016 con validación cruzada agrupada por fase, frente a 0,758 del árbol de decisión y 0,586 de la regresión logística. El *scanning* se distingue mejor (F1 de 0,882) que el *spoofing* (0,823) y el DDoS (0,773), y el error dominante es clasificar un ataque como tráfico normal. Con una segunda generación del conjunto de datos, más afectada por la saturación del entorno, el F1 macro baja a 0,779.
- **En tiempo de ejecución, el modelo detecta los ataques mientras llevan tráfico, y la detección por flujo depende de la mitigación.** Con los flujos de las diez ejecuciones reunidos, el F1 macro

por flujo es de 0,791 (0,756 ± 0,059 como media de las ejecuciones), por debajo del 0,826 medido sin conexión. La diferencia se concentra en el DDoS: antes del primer bloqueo se detecta el 98,6 % de sus flujos, y después la mayoría de los flujos de DDoS son reglas que el bloqueo ha dejado sin tráfico y que el modelo clasifica como normales.

- **La mitigación es eficaz sobre los ataques, a costa de bloqueos erróneos.** De las 395 conversaciones de ataque se bloquearon 390 (exhaustividad de 0,987), y el primer bloqueo llega, de media, entre 2,0 y 2,2 segundos después del primer flujo de ataque observado. Se bloquearon también 28 de las 123 conversaciones legítimas de las pruebas de tráfico normal (22,8 %), cada una durante 20 segundos.

- **El coste para el controlador es moderado en este entorno.** La CPU media de Ryu está entre el 3,5 % y el 6,7 % según el tipo de tráfico, y la latencia media del plano de control, entre 8,4 y 15,8ms.

- **Medir la detección y la mitigación por separado es necesario.** La detección por flujo y el bloqueo por conversación dan cifras distintas porque miden cosas distintas, y el propio bloqueo altera el tráfico que el controlador observa a continuación. Con una sola medida, una de las dos lecturas quedaría oculta.

- **La evaluación sin fuga de información y el cálculo idéntico en entrenamiento y en tiempo de ejecución son decisiones de diseño con consecuencias.** Agrupar por fase reduce el riesgo de las puntuaciones infladas de las particiones aleatorias, y compartir el cálculo de características y de etiquetas entre fases hace que lo medido sin conexión sea representativo de lo que ve el controlador.

La Tabla 7.1 contrasta el trabajo con los objetivos específicos del Capítulo 1. Todos se han abordado. En el OBJ_7, el mecanismo limita el impacto de un falso positivo en el tiempo, pero no lo evita: el corte de 20 segundos afecta a una parte relevante de las conversaciones legítimas, como se discute en la sección 6.5.

Objetivo	Dónde se aborda	Resultado
OBJ_1	Capítulo 3	Revisión de SDN, de su seguridad y del aprendizaje automático aplicado, con siete trabajos relacionados y siete carencias identificadas.
OBJ_2	Secciones 4.2 y 5.2	Entorno con Mininet, Ryu y OpenFlow 1.3, e instalación automatizada con un solo comando a partir de una máquina Ubuntu limpia.
OBJ_3	Secciones 4.3 y 6.2	Conjunto de datos propio de 254.283 flujos en 904 fases, con tráfico normal y de las tres familias de ataque, etiquetado por flujo.
OBJ_4	Secciones 4.4 y 5.4	22 características por flujo, cuatro de ellas de ventana temporal, calculadas con el mismo procedimiento al entrenar y en tiempo de ejecución.
OBJ_5	Secciones 4.5 y 6.3	Tres modelos comparados con validación cruzada agrupada por fase; Random Forest es el mejor (F1 macro de 0,826).
OBJ_6	Secciones 4.6 y 5.5	Clasificación en tiempo de ejecución de cada flujo en el controlador, con 14.788 flujos clasificados en las pruebas.
OBJ_7	Secciones 4.6 y 6.4.2	Bloqueo por conversación con reglas de descarte de 20s y tres confirmaciones; 390 de 395 conversaciones de ataque bloqueadas.
OBJ_8	Secciones 4.7 y 6.4	Detección por flujo y mitigación por conversación medidas por separado, junto con la CPU, la latencia y la inferencia.
OBJ_9	Secciones 7.2 y 7.3	Limitaciones del enfoque y líneas de trabajo futuro.

Tabla 7.1: Objetivos específicos y resultado de cada uno.

7.2. Limitaciones

Las limitaciones del trabajo se agrupan según el aspecto al que afectan. Varias de ellas ya se señalaron donde procedía, y aquí se reúnen.

Datos y etiquetado.

- Los resultados proceden de dos generaciones del conjunto de datos, que difieren de forma apreciable (F1 macro de 0,826 frente a 0,779). El modelo desplegado se entrena con la primera, que muestra menos signos de saturación del entorno, pero la elección se tomó conociendo los dos resultados. Con dos generaciones no se puede estimar con rigor la dispersión entre ellas, para lo que harían falta más tiradas de unas seis horas cada una.

- El criterio de etiquetado incluye a la víctima como actor, de modo que un flujo entre la víctima y un *host* ajeno al ataque también se etiqueta como ataque. Es una simplificación del etiquetado, no de la medición.

- Las fases de ataque se generan sin tráfico legítimo concurrente, lo que probablemente simplifica el problema respecto de una red real.
- La asociación entre IP y MAC de referencia, que usa la característica de coincidencia, se toma de Mininet. En una red real procedería de otra fuente, como DHCP o un inventario de equipos.

Modelo y detección.

- La exhaustividad del DDoS y del *spoofing* es la más baja sin conexión (0,730 y 0,763), y los ataques que se pierden se clasifican como tráfico normal. No se ha determinado la causa, aunque la comparación entre las dos generaciones apunta a que, en el DDoS, influye cómo registra el entorno los flujos de ataque.
- El número de características seleccionadas, 15, es un umbral de diseño y no se ha optimizado: no se ha comparado el rendimiento con otros valores. Además, la selección se basa en la importancia que asigna un Random Forest, por lo que podría favorecer a los modelos basados en árboles frente a la regresión logística; no se ha medido ese efecto.
- No se ha medido en qué características se apoya el modelo para cada clase, porque la importancia se calculó para el conjunto de las clases. Por eso no se sabe cómo se comportaría ante un DDoS lento y sostenido, de menor tasa que los evaluados: es una vía de evasión plausible que no se ha probado.
- Solo se han comparado tres modelos clásicos, con la configuración descrita en el Capítulo 4 y sin una búsqueda de hiperparámetros.

Mitigación.

- El principal coste práctico son los falsos bloqueos: algo más de una de cada cinco conversaciones legítimas sufre un corte de 20 segundos en las pruebas de tráfico normal. Subir el umbral de confirmación o acortar la duración de la regla podría reducirlos, a costa de retrasar la respuesta o de más rebloqueos, pero solo se ha evaluado una combinación: tres confirmaciones y 20 segundos.
- La exhaustividad de la mitigación mide cobertura, no supresión continua: una conversación cuenta como bloqueada si se le aplicó un descarte en algún momento. Como las reglas duran 20 segundos, un ataque prolongado se bloquea, se desbloquea y se vuelve a bloquear.
- La detección en tiempo de ejecución no es independiente de la política de mitigación. Cuanto antes se bloquea una conversación, menos flujos

de ese ataque llega a observar el controlador, y en el DDoS los que observa tras el bloqueo son en su mayoría reglas sin tráfico, que el modelo clasifica como normales. Las cifras de detección de la fase 3 deben leerse junto con el umbral con el que se obtuvieron.

Evaluación.

- La batería de la fase 3 es estocástica: cada ejecución sortea atacantes, víctimas y variantes, y el F1 macro por ejecución varía entre 0,667 y 0,828. Por eso se repite diez veces, y una ejecución aislada no es representativa.
- En la batería de la fase 3 el DDoS se lanza sin la variante sin límite de tasa, que sí está en el conjunto de datos, porque satura la máquina que aloja la emulación mientras el modelo clasifica en tiempo de ejecución.
- Los tiempos de inferencia sin conexión (lotes de decenas de miles de flujos) y en tiempo de ejecución (lotes de pocos flujos) no son comparables entre sí, porque la diferencia se debe al coste fijo de cada llamada al modelo y a que sin conexión la predicción usa todos los núcleos del equipo.
- Los resultados no se han comparado con los de otros trabajos, porque cada uno emplea un conjunto de datos, un protocolo y unas métricas distintos (Sección 3.4).

Entorno.

- La red emulada y el controlador comparten una única máquina, y en momentos puntuales se saturan: en la primera generación, 42 de las 675 fases de ataque (6,2 %) no registraron ningún flujo de su clase, con tramos de hasta 35s sin filas, y los episodios son más frecuentes tras un DDoS sin límite de tasa. Se pierden muestras de ataque, y la diferencia entre las dos generaciones del conjunto de datos se asocia a esta saturación.
- Todo se ha medido en una red emulada, en un único equipo, con un solo controlador, una topología fija de 5 *switches* y 16 *hosts* y enlaces de 10 Mbps. Los resultados, y en particular las medidas de CPU y de latencia, no permiten extrapolar el comportamiento a redes mayores, a varios controladores ni a infraestructura física.
- No se han considerado ataques contra el propio controlador ni contra el canal entre este y los *switches*, que la literatura señala como una amenaza específica de SDN (Sección 3.2).

7.3. Trabajo futuro

Las líneas siguientes se derivan de las limitaciones anteriores y de las carencias del estado del arte que el trabajo no ha cubierto.

- **Más generaciones y más ejecuciones.** Generar varias tiradas más del conjunto de datos y repetir la batería de la fase 3 permitiría estimar con rigor la dispersión entre generaciones y entre ejecuciones.
- **Un entorno sin saturación.** Ejecutar el controlador y la red emulada en máquinas distintas, o con más recursos, y vigilar durante la generación indicadores como las fases sin flujos de su clase, para reducir la saturación observada y su efecto sobre el conjunto de datos.
- **Ataques lentos.** Evaluar ataques de DDoS de baja tasa y medir la importancia de las características por clase, para saber en qué se apoya el modelo y hasta qué punto los detecta.
- **Falsos bloqueos.** Medir el compromiso entre el umbral de confirmación y el retraso de la respuesta con otros valores. También medir el efecto de un corte de 20 segundos sobre aplicaciones concretas, y no solo sobre conversaciones.
- **Otras respuestas de mitigación.** Comparar el descarte con alternativas que recoge la literatura, como limitar la tasa con *meters* de OpenFlow (Sección 3.3) o retrasar o redirigir el tráfico sospechoso a un análisis más profundo (Sección 3.2), que no asumen una detección perfecta.
- **Tráfico más realista.** Generar ataques con tráfico legítimo concurrente, y revisar el criterio de etiquetado para separar los flujos del atacante de los de la víctima, de modo que se pueda medir su efecto sobre los errores.
- **Referencia de identidad realista.** Obtener la asociación entre IP y MAC de una fuente propia de una red real, como DHCP o un inventario, en lugar de tomarla de Mininet.
- **Entornos más amplios.** Repetir la evaluación con otras topologías, con varios controladores y, si es posible, con infraestructura física, que la literatura señala como poco habitual (Sección 3.1).
- **Otros modelos.** Evaluar otros algoritmos, con búsqueda de hiperparámetros, y su coste en el controlador.

Anexo A

Código de las fases 1 y 2

Este anexo reúne los fragmentos de código de las fases 1 y 2 a los que remite el Capítulo 5. Se reproducen tal como figuran en el código fuente.

A.1. Etiquetado por flujo

La función siguiente, de `controller/sdn_monitor.py`, aplica el criterio de etiquetado por actores de la Sección 4.3. Las etiquetas de una fase normal y del tráfico de transición (`warmup`) se devuelven tal cual; dentro de una fase de ataque, los flujos que no involucran a ningún actor se etiquetan como normales. El controlador de defensa importa esta misma función.

Listado A.1: Etiqueta de un flujo dentro de una fase de ataque.

```
    @staticmethod
    def _label_for_flow(phase_label, actors, ip_src, ip_dst, arp_spa,
        arp_tpa):
        """Etiqueta de una fila concreta dentro de una fase de ataque.

        Devuelve la etiqueta del ataque solo si el flujo involucra a
            algún
        actor (atacante, víctima o identidad suplantada), y "normal"
            en
        caso contrario, de modo que el tráfico de fondo simultáneo al
        ataque no se etiqueta como ataque. Es el criterio estándar en
        datasets del área (CICIDS y similares etiquetan por pareja
        origen/destino del ataque, no por franja temporal).

        Sin actores (fase normal) se conserva la
        etiqueta de la fase tal cual.
        """
        if not actors or phase_label in ("normal", "warmup"):
            return phase_label
        for ip in (ip_src, ip_dst, arp_spa, arp_tpa):
            if ip and ip in actors:
                return phase_label
        return "normal"
```

A.2. Ventana temporal

La clase siguiente, del módulo `feature_windows.py`, calcula las características de patrón entre flujos. La usan tanto el preprocesado, sobre el conjunto de datos completo, como el clasificador en tiempo de ejecución, flujo a flujo a medida que llegan las estadísticas.

Listado A.2: Ventana deslizante temporal compartida por el preprocesado y la detección en tiempo de ejecución.

```
class WindowTracker:
    """Cuenta eventos y valores distintos por clave dentro de una ventana
    temporal de tamaño fijo.

    La ventana mira siempre hacia atrás: cada llamada a add() registra un
    evento, descarta los que ya han caducado y devuelve el estado
    actualizado de esa clave.

    Ejemplo, para contar puertos distintos por origen en 5 segundos:
        tracker = WindowTracker(window_seconds=5)
        for src, port, ts in events:          # en orden temporal
            total, distinct = tracker.add(key=src, value=port, now=ts)

    """

    def __init__(self, window_seconds: float):
        self.window = float(window_seconds)
        self._events = defaultdict(deque)               # key -> deque [(timestamp, value)]
        self._value_counts = defaultdict(lambda: defaultdict(int))  # key -> {value: repeticiones}
        self._distinct = defaultdict(int)               # key -> valores distintos en la ventana

    def _prune(self, key, now: float) -> None:
        """Descarta los eventos de 'key' anteriores a la ventana.

        Mantiene actualizado el recuento de valores distintos: un valor
        deja de contar solo cuando caduca su última aparición.
        """
        dq = self._events[key]
        vc = self._value_counts[key]
        while dq and (now - dq[0][0]) > self.window:
            _, old_value = dq.popleft()
            vc[old_value] -= 1
            if vc[old_value] == 0:
                del vc[old_value]
                self._distinct[key] -= 1

    def add(self, key, value, now: float):
        """Registra un evento y devuelve (total, distintos) para 'key'
        dentro de la ventana, contando este mismo evento."""
        self._prune(key, now)
        dq = self._events[key]
        vc = self._value_counts[key]
        if vc[value] == 0:
```

```
            self._distinct[key] += 1
        vc[value] += 1
        dq.append((now, value))
        return len(dq), self._distinct[key]
```

A.3. Validación cruzada agrupada por fase

La función siguiente, de `ml/evaluate.py`, evalúa un modelo con `GroupKFold`. El escalado y la selección de características se ajustan dentro de cada partición, solo con sus datos de entrenamiento.

Listado A.3: Validación cruzada agrupada por fase, con escalado y selección dentro de cada partición.

```
def cross_validate_model(model_ctor, X, y, groups, n_splits=N_CV_FOLDS):
    """GroupKFold: entrena y evalúa el modelo en n_splits particiones,
    sin que ninguna fase quede repartida entre train y test dentro de
    un mismo fold. Devuelve las predicciones "out-of-fold" (cada fila
    predicha exactamente una vez) y el F1 macro de cada fold."""
    gkf = GroupKFold(n_splits=n_splits)
    oof_pred = np.full(len(y), -1)
    per_fold_metrics = []
    per_flow_inference_times = []

    for fold_i, (train_idx, test_idx) in enumerate(gkf.split(X, y,
        groups=groups), start=1):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        scaler = StandardScaler()
        X_train_s = pd.DataFrame(scaler.fit_transform(X_train),
            columns=X.columns)
        X_test_s = pd.DataFrame(scaler.transform(X_test), columns=X.
            columns)

        selected = select_features_for_fold(X_train_s, y_train)
        X_train_sel, X_test_sel = X_train_s[selected], X_test_s[
            selected]

        model = model_ctor()
        model.fit(X_train_sel, y_train)

        t0 = time.time()
        y_pred = model.predict(X_test_sel)
        inference_time = time.time() - t0
        per_flow_inference_times.append(inference_time / len(
            X_test_sel))

        oof_pred[test_idx] = y_pred
        fold_f1 = f1_score(y_test, y_pred, average="macro",
            zero_division=0)
        per_fold_metrics.append(fold_f1)
        print(f"    fold {fold_i}/{n_splits}: F1 = {fold_f1:.4f} "
              f"(train={len(train_idx)}, test={len(test_idx)})")

    return oof_pred, per_fold_metrics, per_flow_inference_times
```

Anexo B

Código de la fase 3

Este anexo reúne los fragmentos de código de la fase 3 a los que remite el Capítulo 5. Se reproducen tal como figuran en el código fuente.

B.1. Clasificación en lote

El método siguiente, de `controller/live_classifier.py`, clasifica todos los flujos de una respuesta de estadísticas en una sola llamada al modelo. Las características de ventana se calculan flujo a flujo y en orden, porque el módulo de ventana necesita la secuencia.

Listado B.1: Clasificación en lote de los flujos de una respuesta de estadísticas.

```
    def predict_batch(self, raws, now=None):
        """Clasifica varios flujos en una sola llamada al modelo.

        Es lo que evita que el controlador se atasque: llamar a predict()
        por flujo tiene un coste fijo por llamada que, multiplicado por
        todos los flujos de cada sondeo, deja al controlador atrás. Una
        sola llamada sobre la matriz completa los clasifica en unos pocos
        milisegundos. Las características de ventana sí se calculan flujo a
        flujo y en orden (el WindowTracker necesita secuencia), pero eso es
        barato. Devuelve (label, infer_ms_por_flujo, window_feats) por
        flujo, en el orden de 'raws'.
        """
        if not raws:
            return []
        rows, windows = [], []
        for raw in raws:
            # build_feature_row alimenta los WindowTracker y devuelve el
            # vector ya escalado y recortado a las features seleccionadas.
```

```
19              rows.append(self.build_feature_row(raw, now)[0])
20              windows.append(dict(self._last_window))
21          X = np.vstack(rows)
22          t0 = time.perf_counter()
23          preds = self.model.predict(X)
24          total_ms = (time.perf_counter() - t0) * 1000.0
25          labels = self.le_y.inverse_transform(preds)
26          # Tiempo de inferencia repartido por flujo (para las métricas)
                .
27          per_flow_ms = total_ms / len(raws)
28          return [(labels[i], per_flow_ms, windows[i]) for i in range(
                len(raws))]
```

B.2. Mitigación

La función siguiente, de `controller/sdn_defense.py`, decide si se bloquea una conversación y, en su caso, instala el descarte en todos los *switches*.

Listado B.2: Decisión y aplicación del bloqueo de una conversación.

```
1       def _mitigate(self, eth_src, eth_dst, now):
2           """Decide si bloquear la conversación eth_src -> eth_dst y, si
3           procede, instala el DROP en todos los switches. Devuelve True
                si se
4           ha instalado ahora.
5
6           Se bloquea por par de MACs, no por IP origen, por varios
                motivos:
7             - El modelo dice "este flujo pertenece a un ataque", no "
                  este
8               host es el atacante", y el dataset etiqueta como ataque
                    también
9               las respuestas de la víctima. Bloquear la IP origen cortar
                    ía a
10              las víctimas; bloquear el par solo corta la respuesta
                    concreta
11              hacia el atacante, que es inofensiva.
12            - En spoofing la IP origen es falsa, así que bloquearla
                  dejaría
13              fuera al host suplantado. La MAC origen es la del
                    remitente
14              real.
15            - Un falso positivo corta una conversación, no un host
                  entero.
16            - En todos los switches, porque bloquear solo en el que
                  reportó
17              el flujo no corta el tráfico que no pasa por él.
18
19          Solo bloquea si el par se clasifica como ataque en
20          MITIGATION_CONFIRMATIONS sondeos distintos dentro de
21          CONFIRM_WINDOW_S."""
22          if not eth_src or not eth_dst:
23              return False
24          pair = (eth_src, eth_dst)
25          if now < self.mitigated.get(pair, 0):
26              return False          # ya bloqueado y la regla sigue
                    vigente
27          rnd = int(now / POLL_INTERVAL)
```

```
28          rounds = self._attack_rounds.setdefault(pair, {})
29          rounds[rnd] = now
30          for r in [r for r, t in rounds.items() if now - t >
                CONFIRM_WINDOW_S]:
31              del rounds[r]
32          if len(rounds) < MITIGATION_CONFIRMATIONS:
33              return False
34          for dp in list(self.datapaths.values()):
35              try:
36                  parser = dp.ofproto_parser
37                  dp.send_msg(parser.OFPFlowMod(
38                      datapath=dp, priority=MITIGATION_PRIORITY,
39                      match=parser.OFPMatch(eth_src=eth_src, eth_dst=
                            eth_dst),
40                      instructions=[],        # sin acciones = DROP en
                            OpenFlow 1.3
41                      idle_timeout=0, hard_timeout=DROP_TIMEOUT, cookie=
                            DROP_COOKIE,
42                  ))
43              except Exception as e:
44                  self.logger.warning("[defense] no se pudo instalar
                        DROP en dpid=%s: %s",
45                                      getattr(dp, "id", "?"), e)
46          self.mitigated[pair] = now + DROP_TIMEOUT
47          rounds.clear()
48          self.logger.info("[defense] DROP %s -> %s (%ds)", eth_src,
                eth_dst, DROP_TIMEOUT)
49          return True
```

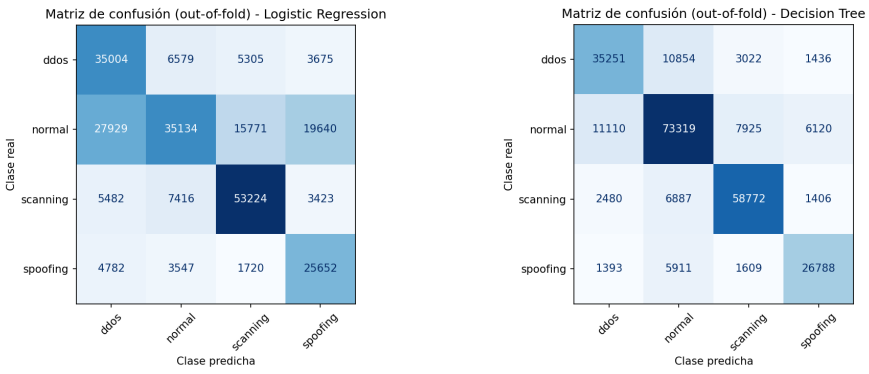
Anexo C

Resultados complementarios

Este anexo reúne las tablas y figuras de la experimentación (Capítulo 6) que complementan las del cuerpo del capítulo. Proceden de los ficheros que generan la fase 2 y la fase 3, y se reproducen sin modificar.

C.1. Matrices de confusión de los otros modelos

La Figura C.1 muestra las matrices de confusión de la regresión logística y del árbol de decisión con la misma validación que la de Random Forest (Figura 6.1).



(a) Regresión logística.

(b) Árbol de decisión.

Figura C.1: Matrices de confusión de la regresión logística y del árbol de decisión con validación cruzada agrupada por fase. Las clases aparecen en orden alfabético.

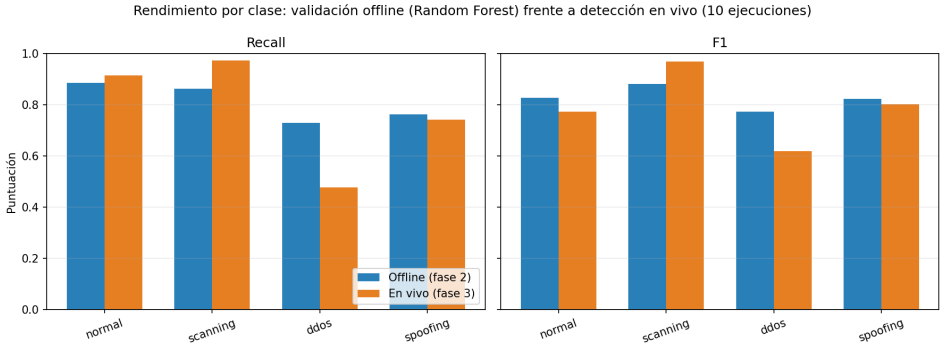


Figura C.2: Exhaustividad (*recall*) y F1 por clase de Random Forest sin conexión (fase 2) y en tiempo de ejecución (fase 3).

C.2. Rendimiento sin conexión y en tiempo de ejecución por clase

La figura C.2 representa gráficamente las columnas de exhaustividad y F1 de la Tabla 6.4. No se dibuja la precisión, porque depende de la proporción de clases, que en la batería no es la del conjunto de datos.

C.3. Resultados por ejecución de la batería

La Tabla C.1 da, para cada una de las diez ejecuciones de la batería de la fase 3, las métricas macro de la detección por flujo. Su media y su desviación típica poblacional son las que se dan en la Sección 6.4, junto al valor calculado con todos los flujos reunidos.

Ejecución	Flujos	Prec. macro	Exh. macro	F1 macro
1	1.383	0,872	0,815	0,809
2	1.527	0,734	0,660	0,667
3	1.449	0,756	0,681	0,696
4	1.389	0,841	0,806	0,787
5	1.626	0,852	0,764	0,795
6	1.329	0,786	0,663	0,696
7	1.432	0,872	0,818	0,827
8	1.766	0,870	0,834	0,828
9	1.281	0,854	0,765	0,761
10	1.606	0,755	0,675	0,697
Media	1.478,8	0,819	0,748	0,756
Desviación	143,0	0,052	0,068	0,059

Tabla C.1: Métricas macro de la detección en tiempo de ejecución en cada ejecución de la batería. Prec.: precisión. Exh.: exhaustividad.

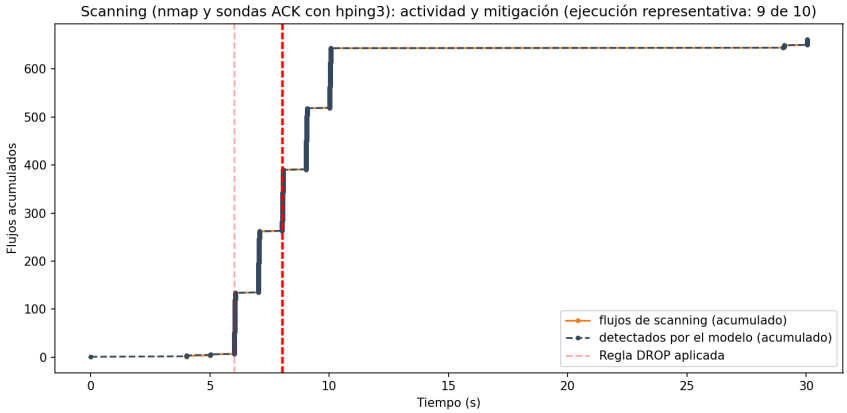


Figura C.3: Prueba de *scanning*: flujos de ataque acumulados y flujos detectados por el modelo, con las reglas de descarte instaladas.

C.4. Ejecución representativa de cada tipo de tráfico

Las figuras siguientes corresponden a la ejecución representativa de la batería, la novena, cuyo F1 macro (0,761) es el más cercano a la media (0,756). Las líneas verticales discontinuas marcan los instantes en que se instala una regla de descarte. El DDoS se muestra en la Figura 6.4 del Capítulo 6.

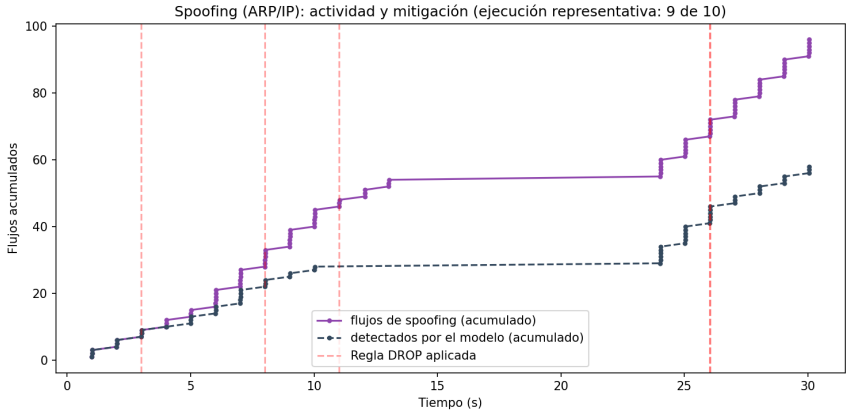


Figura C.4: Prueba de *spoofing*: flujos de ataque acumulados y flujos detectados por el modelo, con las reglas de descarte instaladas.

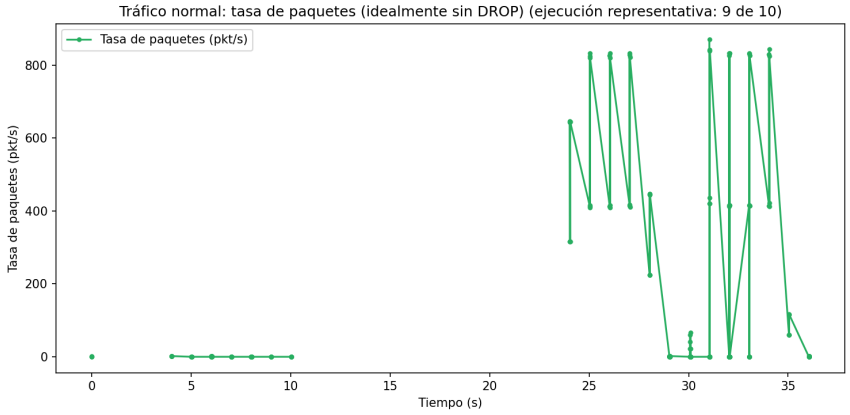


Figura C.5: Prueba de tráfico normal: tasa de paquetes y reglas de descarte instaladas. Al no haber ataque, toda regla de descarte sería un falso bloqueo; en esta ejecución no se produjo ninguno, aunque en cada una de las otras nueve hubo entre 1 y 7. Entre los 10s y los 24 s no se registran flujos, un hueco que no aparece en las demás ejecuciones (en ellas, el mayor es de 5s) y que es compatible con la saturación puntual del entorno (Sección 6.2.2).

C.5. CPU y latencia del plano de control

La Figura C.6 muestra, en la misma ejecución representativa, la CPU del proceso de Ryu a lo largo de cada prueba y la latencia del plano de control de cada bloqueo.

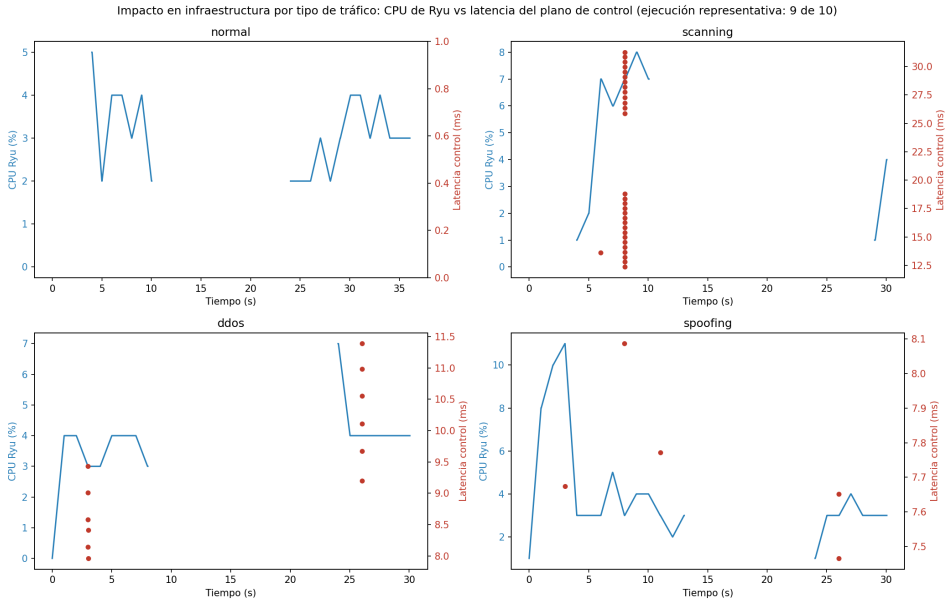


Figura C.6: CPU de Ryu y latencia del plano de control por tipo de tráfico en la ejecución representativa. En las pruebas de ataque, el tramo central sin datos corresponde a la vigencia de las primeras reglas de descarte, durante la que no aparecen flujos nuevos; en la de tráfico normal, al hueco descrito en la Figura C.5.

Glosario

Este glosario define los términos y las siglas que se emplean en la memoria, con el sentido que tienen en ella. Las herramientas software se describen por su papel en el proyecto.

ACK Indicador de la cabecera TCP que marca un segmento como confirmación de datos recibidos. En el trabajo se usan sondas que solo llevan este indicador como una de las variantes de *scanning*.

API *Application Programming Interface*. Interfaz que un programa ofrece para que otros lo utilicen.

Aprendizaje automático Conjunto de técnicas que construyen un modelo a partir de ejemplos etiquetados, en lugar de programar reglas a mano. En este trabajo, el modelo clasifica cada flujo como tráfico normal o como uno de los tres tipos de ataque.

Árbol de decisión Modelo que clasifica mediante una secuencia de preguntas sobre las características, organizadas en forma de árbol. Es uno de los tres modelos comparados.

ARP *Address Resolution Protocol*. Protocolo que, en una red local, asocia una dirección IP con la dirección MAC del equipo que la usa. Es el protocolo que se falsifica en el *spoofing* ARP.

Batería de pruebas Conjunto de cuatro pruebas de 30s, una por cada tipo de tráfico (normal, *scanning*, *spoofing* y DDoS), que se lanza sobre la red emulada con el modelo desplegado para evaluar la fase 3. Se repite diez veces.

Característica Magnitud calculada sobre un flujo que sirve de entrada al modelo, como la duración, la tasa de paquetes o el número de puertos distintos sondeados por un origen. Cada flujo se describe con 22.

Clase Categoría que el modelo asigna a un flujo. Son cuatro: tráfico normal, *scanning*, *spoofing* y DDoS.

Conjunto de datos Tabla de flujos etiquetados que se genera en la fase 1 y con la que se entrenan y evalúan los modelos en la fase 2. En el trabajo se emplea este término y no el anglicismo *dataset*.

Controlador Programa que ejerce el plano de control de una red SDN: recibe información de los *switches* y les instala reglas. En el trabajo es una aplicación de Ryu.

Conversación Par ordenado formado por las direcciones MAC de origen y de destino de un flujo. Es la unidad que bloquea el controlador, solo en ese sentido de la comunicación, y la unidad en la que se mide la mitigación.

CPU *Central Processing Unit*. Procesador. En el trabajo se mide el porcentaje de CPU que consume el proceso del controlador.

DDoS *Distributed Denial of Service*. Ataque de denegación de servicio en el que múltiples orígenes saturan los recursos de la víctima hasta degradar o interrumpir su servicio.

Descarte (regla DROP) Regla OpenFlow cuyas instrucciones no incluyen ninguna acción de salida, por lo que el *switch* descarta los paquetes que coinciden con ella. Es el mecanismo con el que el sistema bloquea una conversación.

DHCP *Dynamic Host Configuration Protocol*. Protocolo que asigna direcciones IP a los equipos de una red.

DoS *Denial of Service*. Ataque que degrada o interrumpe un servicio saturando sus recursos. Cuando procede de múltiples orígenes es un DDoS.

ECTS *European Credit Transfer and Accumulation System*. Sistema europeo de créditos universitarios. El Trabajo de Fin de Grado equivale a 12 créditos.

Emulación Ejecución de una red virtual en la que los equipos y los *switches* son software real, de modo que el tráfico generado es tráfico de red real. Se distingue de la simulación, en la que el tráfico es el resultado de un modelo.

En tiempo de ejecución Dicho de la clasificación y la mitigación que se realizan sobre el tráfico mientras este se genera, con sondeos de estadísticas cada segundo. No implica una respuesta instantánea.

Entrada de flujo Regla de la tabla de flujos de un *switch* OpenFlow, formada por campos de coincidencia, prioridad, contadores, instrucciones, *timeouts* y *cookie*.

Exactitud *Accuracy*. Proporción de flujos clasificados correctamente sobre el total. Depende de la clase mayoritaria, por lo que no se usa para elegir el modelo.

Exhaustividad *Recall*. Proporción de los elementos de una clase que el modelo identifica correctamente. Se calcula por clase y en forma macro.

F1 Media armónica de la precisión y la exhaustividad. El **F1 macro** es la media aritmética del F1 de las cuatro clases, de modo que todas pesan lo mismo.

Falso bloqueo Conversación legítima que el sistema bloquea por error. Se contabiliza por conversación.

Falso positivo Flujo que el modelo clasifica como ataque sin serlo.

Fase (del sistema) Cada una de las tres partes en que se organiza el sistema: generación del conjunto de datos (fase 1), entrenamiento y evaluación de los modelos sin conexión (fase 2) y detección y mitigación en tiempo de ejecución (fase 3).

Fase de tráfico Intervalo de entre 10 y 25s en el que se genera un único tipo de tráfico durante la creación del conjunto de datos. Es la unidad de agrupación de la validación cruzada.

Flujo Conjunto de paquetes que coinciden con una misma entrada de la tabla de flujos de un *switch*. En el trabajo, cada flujo observado se describe con 22 características, se etiqueta y es la unidad de clasificación del modelo.

Fuga de información Situación en la que el modelo se entrena o se prepara con información de los datos con los que luego se evalúa, lo que infla las métricas. Para evitarla, los flujos se agrupan por fase de tráfico.

`GroupKFold` Variante de la validación cruzada de scikit-learn que reparte los datos en particiones sin separar los elementos de un mismo grupo. Aquí, el grupo es la fase de tráfico.

Host Equipo terminal conectado a la red, como una víctima, un atacante o un origen de tráfico normal.

`hping3` Herramienta que construye y envía paquetes TCP, UDP e ICMP a medida. Se usa para los ataques DDoS, las sondas ACK y la suplantación de IP.

ICMP *Internet Control Message Protocol*. Protocolo de mensajes de control y diagnóstico de IP, como el que emplea `ping`.

IoT *Internet of Things*. Internet de las cosas: red de dispositivos conectados que intercambian datos.

IP *Internet Protocol*. Protocolo de red cuyas direcciones identifican a los equipos. Su dirección de origen es la que se falsifica en el *spoofing* de IP.

iPerf Herramienta que genera transferencias TCP y UDP con una tasa configurable. Se usa para el tráfico normal.

KNN *k-Nearest Neighbors*. Clasificador de k vecinos más próximos, que asigna a cada muestra la clase mayoritaria entre las más parecidas del conjunto de entrenamiento.

Latencia del plano de control Tiempo que transcurre desde que el controlador empieza a procesar una respuesta de estadísticas hasta que envía el bloqueo. Solo existe para los flujos que provocan un bloqueo.

LSTM *Long Short-Term Memory*. Tipo de red neuronal recurrente diseñada para aprender dependencias en secuencias.

MAC *Media Access Control*. Dirección que identifica a una interfaz de red dentro de una red local.

Matriz de confusión Tabla que cruza la clase real de cada flujo con la clase que predice el modelo, y permite ver qué clases se confunden entre sí.

Mininet Emulador que crea una red virtual de *hosts* y *switches* en una sola máquina.

Nmap Herramienta de exploración de redes. Se usa para el escaneo de puertos.

Open vSwitch *Switch* virtual de software, con soporte de OpenFlow, que actúa como *switch* de la red emulada.

OpenFlow Protocolo con el que el controlador programa la tabla de flujos de los *switches*. En el trabajo se emplea la versión 1.3 de la especificación.

Packet-in Mensaje OpenFlow con el que un *switch* envía un paquete al controlador, por ejemplo cuando no coincide con ninguna entrada.

Plano de aplicación, de control y de datos Los tres niveles de la arquitectura SDN: las aplicaciones de red, el controlador y los dispositivos que reenvían los paquetes, respectivamente.

Ponderación de clases Ajuste del entrenamiento que penaliza más los errores en las clases minoritarias, sin duplicar ni eliminar filas.

Precisión *Precision*. Proporción de los elementos que el modelo asigna a una clase que pertenecen realmente a ella.

Random Forest Modelo formado por un conjunto de árboles de decisión cuyas predicciones se combinan. En el trabajo se usa con 200 árboles y es el modelo que se despliega.

Regresión logística Modelo lineal de clasificación. Es el más sencillo de los tres modelos comparados.

REST *Representational State Transfer*. Estilo de interfaz web basado en recursos y peticiones HTTP. Es la interfaz *northbound* más extendida en SDN.

ROC *Receiver Operating Characteristic*. Curva que representa la tasa de verdaderos positivos frente a la de falsos positivos al variar el umbral de decisión. El área bajo ella resume el rendimiento del clasificador.

Ryu Controlador SDN escrito en Python que admite OpenFlow 1.3.

Scanning Sondeo sistemático de equipos y puertos para descubrir qué servicios hay expuestos, habitualmente como paso previo a otros ataques.

Scapy Biblioteca de Python para construir y enviar paquetes. Se usa para el envenenamiento ARP.

scikit-learn Biblioteca de Python para aprendizaje automático. Se usa para los modelos, la validación y las métricas.

Script Programa escrito en un lenguaje interpretado que contiene instrucciones para realizar una tarea o automatizar un proceso.

SDN *Software-Defined Networking*. Redes Definidas por Software: arquitectura que separa el plano de control del plano de datos y centraliza lógicamente el control en un controlador programable.

Sondeo Petición periódica de estadísticas de flujo que el controlador dirige a los *switches*. En el trabajo se realiza cada segundo.

Spoofing Suplantación de identidad mediante la falsificación de direcciones. Se consideran el *spoofing* ARP, que asocia la IP de un equipo legítimo a la MAC del atacante, y el *spoofing* de IP, que falsifica la dirección IP de origen.

SSL *Secure Sockets Layer*. Protocolo de cifrado de comunicaciones, predecesor de TLS.

SVM *Support Vector Machine*. Máquina de vectores soporte: clasificador que separa las clases con el hiperplano de máximo margen.

Switch Dispositivo de reenvío de la red. En una red SDN, reenvía los paquetes según las entradas de flujo que le instala el controlador.

SYN Indicador de la cabecera TCP con el que se inicia una conexión. Los paquetes SYN se emplean en los ataques DDoS y en el *spoofing* de IP.

Table-miss Entrada de flujo que se aplica a los paquetes que no coinciden con ninguna otra y que puede enviarlos al controlador, descartarlos o dirigirlos a otra tabla.

TCP *Transmission Control Protocol*. Protocolo de transporte orientado a conexión.

Tiempo de inferencia Tiempo que tarda el modelo en clasificar un flujo.

Timeout Tiempo tras el cual el *switch* elimina una entrada de flujo. El *idle timeout* la elimina cuando no ha coincidido con ningún paquete durante ese tiempo, y el *hard timeout*, transcurrido un tiempo máximo desde su instalación, haya o no coincidido con paquetes.

TLS *Transport Layer Security*. Protocolo de cifrado de comunicaciones. Puede proteger el canal entre el *switch* y el controlador.

UDP *User Datagram Protocol*. Protocolo de transporte sin conexión.

Umbral de confirmación Número de sondeos distintos, dentro de la ventana de confirmación, en que una conversación debe salir clasificada como ataque para que se bloquee. En el trabajo son tres sondeos en una ventana de 5s.

Validación cruzada agrupada Evaluación que divide los datos en particiones y entrena con unas para probar con otras, de modo que los elementos de un mismo grupo queden siempre en el mismo lado. Aquí se agrupa por fase de tráfico, con cinco particiones.

Ventana temporal Intervalo deslizante de 5s que mira solo hacia atrás, se reinicia en cada fase y sobre el que se calculan las cuatro características que describen patrones entre flujos, como los puertos distintos sondeados por un mismo origen.

Bibliografía

[1] Yassine Maleh et al. «A comprehensive survey on SDN security: threats, mitigations, and future directions». *Journal of Reliable Intelligent Environments* 9.2 (2023), págs. 201-239. DOI: 10.1007/s40860-022-00171-8.

[2] Amandeep Kaur, C Rama Krishna y Nilesh Vishwasrao Patil. «A comprehensive review on Software-Defined Networking (SDN) and DDoS attacks: Ecosystem, taxonomy, traffic engineering, challenges and research directions». *Computer Science Review* 55 (2025), pág. 100692. DOI: 10.1016/j.cosrev.2024.100692.

[3] Noe M Yungaicela-Naula et al. «Towards security automation in software defined networks». *Computer Communications* 183 (2022), págs. 64-82. DOI: 10.1016/j.comcom.2021.11.014.

[4] Sunil Gowda y RB Dayananda. «Detection and prevention of arp attack in software defined networks». *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE. 2023, págs. 1-5. DOI: 10.1109/ICCCNT56998.2023.10307123.

[5] James F. Kurose y Keith W. Ross. *Redes de computadoras: un enfoque descendente. 7.ª ed.* Pearson Educación, 2017.

[6] Mudassar Hussain et al. «Software-defined networking: Categories, analysis, and future directions». *Sensors* 22.15 (2022), pág. 5551. DOI: 10.3390/s22155551.

[7] Abdullah Ahmed Bahashwan et al. «A systematic literature review on machine learning and deep learning approaches for detecting DDoS attacks in software-defined networking». *Sensors* 23.9 (2023), pág. 4441. DOI: 10.3390/s23094441.

[8] Nick McKeown et al. «OpenFlow: enabling innovation in campus networks». *ACM SIGCOMM computer communication review* 38.2 (2008), págs. 69-74.

[9] Open Networking Foundation. *OpenFlow Switch Specification, Version 1.3.0 (Wire Protocol 0x04)*. ONF TS-006, 25 de junio. 2012.

[10] Ramiz Salama, Chadi Altrjman y Fadi Al-Turjman. «Network security in architectures for software defined networking (SDN)». *NEU Journal for Artificial Intelligence and Internet of Things* 4.1 (2025), págs. 37-44. DOI: 10.32955/neuaiit202541960.

[11] Mahmoud Said El Sayed et al. «A flow-based anomaly detection approach with feature selection method against ddos attacks in sdns». *IEEE Transactions on Cognitive Communications and Networking* 8.4 (2022), págs. 1862-1880. DOI: 10.1109/TCCN.2022.3186331.

[12] Moustapha Awwalou Diouf et al. «Software security in software-defined networking: A systematic literature review». *arXiv preprint* (2025). DOI: 10.48550/arXiv.2502.13828. eprint: 2502.13828.

[13] Muhammad Shoaib Farooq, Shamyla Riaz y Atif Alvi. «Security and privacy issues in software-defined networking (SDN): A systematic literature review». *Electronics* 12.14 (2023), pág. 3077. DOI: 10.3390/electronics12143077.

[14] Sait Melih Doğan, Aynur Koçak y Mustafa Alkan. «Detection and mitigation of cyber-attacks in software defined networks using machine learning/deep learning: a systematic literature review, research challenges and future directions». *International Journal of Information Security* 24.5 (2025), pág. 209. DOI: 10.1007/s10207-025-01114-z.

[15] Abdulsalam O Alzahrani y Mohammed JF Alenazi. «ML-IDSDN: Machine learning based intrusion detection system for software-defined network». *Concurrency and Computation: Practice and Experience* 35.1 (2023), e7438. DOI: 10.1002/cpe.7438.

[16] Bob Lantz, Brandon Heller y Nick McKeown. «A network in a laptop: rapid prototyping for software-defined networks». *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*. 2010, págs. 1-6.

[17] Nippon Telegraph and Telephone Corporation. *Ryu 4.34 Documentation*. https://ryu.readthedocs.io/en/latest/. Documentación oficial de Ryu.

[18] Kadir Hasan et al. «Real-time DDoS detection in software-defined networks using machine learning». *2024 27th International Conference on Computer and Information Technology (ICCIT)*. IEEE. 2024, págs. 453-458. DOI: 10.1109/ICCIT64611.2024.11022460.

[19] Kawthar Rasoul ALesawi y Aymen Hassan Alawadi. «Effectual real-time DDoS detection and mitigation based on SDN and machine learning using feature selection technique». *AIP Conference Proceedings*. Vol. 3408. AIP Publishing, 2026, pág. 040165. DOI: 10.1063/5.0330582.

[20] Kyaw Swar Linn et al. «A Realistic SDN-Based Framework for DDoS Detection Using Phase-Based Traffic and Machine Learning». *2026 International Conference on Technical Education (ICTechED)*. IEEE. 2026, págs. 1-6. DOI: 10.1109/ICTechED69022.2026.11682896.

[21] Jiayue Lai. «Machine Learning-Based Network Detection Research for SDNs». *ITM Web of Conferences*. Vol. 70. EDP Sciences. 2025, pág. 01015. DOI: 10.1051/itmconf/20257001015.

[22] Fabian Pedregosa et al. «Scikit-learn: Machine learning in Python». *Journal of Machine Learning Research* 12 (2011), págs. 2825-2830.

[23] LaTeX Project. *LaTeX Documentation*. https://www.latex-project.org/help/documentation/. Documentación oficial de LaTeX.

[24] Canonical Ltd. *Ubuntu Desktop*. https://ubuntu.com/desktop. Sitio oficial de Ubuntu Desktop. Versión utilizada: 22.04.5 LTS.

[25] Python Software Foundation. *Python 3.10 Documentation*. https://docs.python.org/3.10/. Documentación oficial de Python. Versión utilizada: 3.10.12. 2026.

[26] Eventlet Contributors. *Eventlet 0.35.2 Documentation*. https://eventlet.readthedocs.io/en/v0.35.2/. Documentación oficial de Eventlet. 2024.

[27] Ben Pfaff et al. «The design and implementation of Open vSwitch». *12th USENIX symposium on networked systems design and implementation (NSDI 15)*. 2015, págs. 117-130.

[28] The Scapy community. *Scapy Documentation*. https://scapy.readthedocs.io/. Documentación oficial de Scapy. Versión utilizada: 2.7.0. 2026.

[29] Kali Linux. *hping3*. https://www.kali.org/tools/hping3/. Documentación del paquete hping3 en Kali Linux. 2025.

[30] Gordon Lyon. *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Guía oficial de Nmap. 2009. URL: https://nmap.org/book/toc.html.

[31] iPerf. *iPerf - iPerf3 and iPerf2 User Documentation*. https://iperf.fr/iperf-doc.php. Documentación de iPerf.

[32] Wes McKinney et al. «Data structures for statistical computing in Python.» *scipy* 445.1 (2010), págs. 51-56.

[33] Charles R Harris et al. «Array programming with NumPy». *Nature* 585.7825 (2020), págs. 357-362. DOI: 10.1038/s41586-020-2649-2.

[34] Joblib Developers. *Joblib Documentation*. https://joblib.readthedocs.io/en/stable/index.html. Documentación oficial de Joblib. 2026.

[35] John D Hunter. «Matplotlib: A 2D graphics environment». *Computing in science & engineering* 9.3 (2007), págs. 90-95.

[36] Giampaolo Rodola. *psutil Documentation*. `https://psutil.readthedocs.io/stable/`. Documentación oficial de psutil. 2026.

[37] Microsoft. *Visual Studio Code Documentation*. `https://code.visualstudio.com/docs`. Documentación oficial de Visual Studio Code. Versión utilizada: 1.133.0.

[38] Git Development Team. *Git Documentation*. `https://git-scm.com/docs`. Documentación oficial de Git. Versión utilizada: 2.34.1.

[39] GitHub, Inc. *GitHub Docs*. `https://docs.github.com/`. Documentación oficial de GitHub.

[40] Universidad Francisco de Vitoria. *Cuánto cobra un ingeniero informático en España en 2026: junior, mid y senior*. `https://www.ufv.es/cuanto-cobra-un-ingeniero-informatico-en-espana-en-2026-junior-mid-y-senior-preguntas-grados/`. 2026.

[41] Isiaka Babatunde Sadiku et al. «Effect of traditional and software-defined networking on performance of computer network». *Scientific Journal of Informatics* 9.2 (2022), págs. 111-122. DOI: `10.15294/sji.v9i2.31315`.

[42] Quadri Waseem et al. «Software-defined networking (SDN): a review». *2022 5th international conference on information and communications technology (ICOIACT)*. IEEE. 2022, págs. 30-35. DOI: `10.1109/ICOIACT55506.2022.9972067`.

[43] Neelam Gupta et al. «A comparative study of software defined networking controllers using mininet». *Electronics* 11.17 (2022), pág. 2715. DOI: `10.3390/electronics11172715`.